LIBXS

LIBXS is a portable C library providing building blocks for memory operations, numerics, synchronization, and more — with a focus on performance and minimal dependencies. Targets x86-64, AArch64, and RISC-V; requires only a C89 compiler. Originally developed as part of LIBXSMM.

Functionality

Domain	Header	Description
Core	`libxs.h`	Initialization, verbosity, data types
Registry	`libxs_reg.h`	Thread-safe key-value store
Predict	`libxs_predict.h`	Fingerprint-guided parameter prediction
Malloc	`libxs_malloc.h`	Pool allocator (steady state: no syscalls)
Memory	`libxs_mem.h`	Byte compare, matrix copy/transpose
String	`libxs_str.h`	Edit distance, substring, similarity
N-gram	`libxs_ngram.h`	Variable-order n-gram counts and backoff
Token	`libxs_token.h`	Reversible metatokens and lexical IDs
Timer	`libxs_timer.h`	High-resolution timing via calibrated TSC
CPUID	`libxs_cpuid.h`	CPU feature detection (x86-64, AArch64, RISC-V)
Utils	`libxs_utils.h`	ISA gates, bit-scan, SIMD helpers
Sync	`libxs_sync.h`	Atomics, locks, TLS, barrier, ring buffer, singleton
GEMM	`libxs_gemm.h`	GEMM dispatch and batches (pointer, index, strided)
SYRK	`libxs_syrk.h`	Symmetric rank-k and rank-2k updates (SYRK/SYR2K)
TAS	`libxs_tas.h`	Tall-and-skinny GEMM as a batch of panels
Math	`libxs_math.h`	Matrix compare, fingerprints, setdiff, compensated sums, modular math, FP16/BF16
Hash	`libxs_hash.h`	CRC32, Adler-32, string hashing
Pack	`libxs_pack.h`	Bit packing at fixed or per-group widths
Perm	`libxs_perm.h`	Shuffle, kd-tree, Hilbert/Morton curves
Hist	`libxs_hist.h`	Thread-safe histogram, running statistics
Mix	`libxs_mix.h`	Fixed-share mixture over expert banks
MHD	`libxs_mhd.h`	Read/write MetaImage (MHD/MHA) files
RNG	`libxs_rng.h`	Pseudo-random generation (SplitMix64)

See also: Fortran Interface, Scripts.

Build

```
make
```

The library is compiled for the build host's highest SSE level (up to SSE4.2) by default but dynamically dispatches to the best ISA available at runtime (up to AVX-512). Use `SSE=0` to compile natively for the build host. The library is built into `lib/<target>`, e.g., `lib/x86_64` by default or `lib/<cpu>` with `SSE=0`, and `lib/` refers to the latest build.

Variable	Default	Description
GNU	1	GNU GCC-compatible compiler (0: Intel)
DBG	0	Debug build
SYM	0	Include debug symbols (-g)
SSE	1	x86 baseline: 0=native, 1=host's SSE level
LIBARCH	(auto)	Name of the build folder (`lib/<target>`)
STATIC	(both)	1: static library only, 0: shared library only
FORTRAN	(auto)	0: no Fortran module, even if a compiler is found
OMP	0	1: compile with OpenMP (the library does not use it)

Tests and samples:

```
make tests
make samples
```

CMake is also supported:

```
cmake -S . -B build
cmake --build build
cmake --install build --prefix $HOME/libxs
```

The C library uses OpenMP SIMD when supported by the compiler, without an OpenMP runtime dependency. The Fortran module's internal OpenMP parallelism is opt-in (`LIBXS_FORTRAN_OMP=ON`). Tests and samples enable their own OpenMP by default; set `LIBXS_SAMPLES_OMP=OFF` to disable it. Both options are available only when their corresponding targets are built. On macOS, Homebrew's `libomp` is detected via `brew --prefix libomp`; use `OpenMP_ROOT` for another installation.

Option	Default	Description
LIBXS_FORTRAN	(auto)	Build the Fortran module
LIBXS_FORTRAN_OMP	OFF	Enable OpenMP in the Fortran module
LIBXS_SAMPLES_OMP	ON	Enable OpenMP in samples and tests
LIBXS_INSTALL_HEADER_ONLY	OFF	Also install the header-only implementation
BUILD_TESTING	OFF	Build the tests (requires BLAS)
BUILD_SHARED_LIBS	OFF	Build the shared library

Installation

Install into a chosen prefix:

```
make -j $(nproc) install PREFIX=$HOME/libxs
```

This installs headers, the Fortran module, the static and shared libraries, the header-only source tree, pkg-config files, and CMake package files under `PREFIX`. A given `DESTDIR` is prepended to `PREFIX` (staged installation).

Out-of-tree builds are also supported:

```
mkdir /tmp/libxs-build && cd /tmp/libxs-build
make -j $(nproc) -f /path/to/libxs/Makefile
```

Usage

Library — link against `libxs.a` (or `.so`) and include the desired headers:

```
#include <libxs/libxs_mem.h>
#include <libxs/libxs_timer.h>
```

Both builds install pkg-config files (`libxs.pc`) and a CMake package (target `libxs::libxs`). A static library needs `--static` to also list its dependencies (pthread, libm, libdl, librt):

```
cc app.c $(pkg-config --cflags libxs) $(pkg-config --libs --static libxs)
```

```
find_package(libxs REQUIRED)
target_link_libraries(app PRIVATE libxs::libxs)
```

Header-only (explicit) — include `libxs_source.h` (no separate library needed). Safe to include from multiple translation units:

```
#include <libxs/libxs_source.h>
```

Header-only (implicit) — compile with `-DLIBXS_SOURCE` and any LIBXS public header automatically includes the implementation. No special include order is required. When used through LIBXSTREAM without a pre-built library (`-DLIBXSTREAM_SOURCE`), `LIBXS_SOURCE` is implied automatically. Either way, the application still links `-lpthread -lm -ldl` (and `-lrt` with older C libraries).

Fortran — `USE LIBXS` is part of the library, and `libxs.mod` is installed next to the headers (`include/libxs`). `USE LIBXS_JIT` adds GEMM dispatch with the kernel providers filled in; the application compiles `libxs_jit.F` itself (see Kernel Providers and the Fortran Interface):

```
USE :: LIBXS, ONLY: libxs_memcmp
```

Environment — read by the library at initialization or exit:

Variable	Default	Description
LIBXS_VERBOSE	0	Verbosity level; empty means 1, negative means all (debug build: all)
LIBXS_PACK	2	Level 0 of libxs_vpack (at the first call): 1 fastest, 2 smaller, 3 smallest (see Pack)
LIBXS_PTRACE	0	Linux: any process may attach a debugger (not only an ancestor)
LIBXS_SIGNAL	(verbose)	Report at SIGABRT/SIGSEGV, then pass the signal on (on if verbose); 0 leaves handlers alone
LIBXS_DUMP_BUILD		Print the build state (GNU Make build): >0 at exit, <0 at startup
LIBXS_DUMP_FILES		At exit, print the listed files (`,`;`:`-separated, `PID` expands); a number N prints the PID and waits N seconds

`LIBXS_DUMP_FILE` is an alias of `LIBXS_DUMP_FILES`. Both dumps are printed by the first LIBXS process on the system only.

Samples

Sample	Description
tokenizer	Reversible configurable metatokenizer
converse	Extractive summarization via tokenized fprints
registry	Registry dispatch microbenchmark
predict	Fingerprint-guided parameter prediction
rosetta	Hierarchical type discovery on opaque data
setdiff	Deterministic set-difference tolerance
shuffle	Shuffling strategies comparison
pack	Bit packing at fixed vs per-group width
scratch	Pool allocator vs system malloc
spatial	Stratification, pair counting, kd-tree
fprint	Fingerprint structure/geometry tests
bracket	Sharp edges from bracket units (Adam, GEMM)
memory	Byte comparison, matrix copy, transpose
ozaki	Ozaki-scheme low-precision GEMM
gemm	Batched DGEMM (strided, pointer, grouped)
syrk	Symmetric rank-k/2k update with validation
sync	Lock implementation microbenchmarks

License

BSD 3-Clause

LIBXS Domains

Core: Initialization and Common Types

Header: `libxs.h`

Library initialization, verbosity, the element-type enumeration shared by the other modules, and the common macros of `libxs_macros.h` (included by every LIBXS header). Environment variables (e.g., `LIBXS_VERBOSE`) are listed in the Usage section of the overview.

Initialization

```
void libxs_init(void);
void libxs_finalize(void);
```

`libxs_init` is optional: functions that need the library initialize it on first use, and library builds initialize at load time where the compiler supports constructors. Call it to take the setup cost at a chosen point; repeated calls have no effect. `libxs_finalize` releases internal memory such as the default memory pool; it is optional as well and runs at unload where supported.

Verbosity

```
int libxs_get_verbosity(void);
void libxs_set_verbosity(int level);
```

Level	Output
0	None
1	Errors
2	Warnings
3	Information and reports (e.g., memory pool statistics)
negative	Everything

The initial level comes from `LIBXS_VERBOSE` (an empty value means 1), and is the maximum in debug builds when the variable is unset.

One warning is printed by `libxs_init` even when `LIBXS_VERBOSE` is unset: a sequential OpenBLAS together with an OpenMP runtime (unless `OMP_NUM_THREADS=1`), since such an OpenBLAS may corrupt results when called from several threads (unless it was built with `USE_LOCKING=1`). Link the OpenMP or pthread build of OpenBLAS, or set `LIBXS_VERBOSE=0` to mute the warning.

Element Types

```
typedef enum libxs_data_t {
  LIBXS_DATATYPE_F64 = 0 | (8 << 4),
  LIBXS_DATATYPE_F32 = 1 | (4 << 4),
  LIBXS_DATATYPE_C64 = 2 | (16 << 4),
  LIBXS_DATATYPE_C32 = 3 | (8 << 4),
  LIBXS_DATATYPE_I64 = 4 | (8 << 4),
  LIBXS_DATATYPE_U64 = 5 | (8 << 4),
  LIBXS_DATATYPE_I32 = 6 | (4 << 4),
  LIBXS_DATATYPE_U32 = 7 | (4 << 4),
  LIBXS_DATATYPE_I16 = 8 | (2 << 4),
  LIBXS_DATATYPE_U16 = 9 | (2 << 4),
  LIBXS_DATATYPE_I8  = 10 | (1 << 4),
  LIBXS_DATATYPE_U8  = 11 | (1 << 4),
  LIBXS_DATATYPE_UNKNOWN = 12
} libxs_data_t;
```

Real, complex (C64 is a pair of `double`, C32 a pair of `float`), signed and unsigned integer types. Each enumerator carries the size of its type.

Macro	Description
`LIBXS_TYPESIZE(ENUM)`	Size of the type in bytes
`LIBXS_TYPEORDER(ENUM)`	Enumerator without the size
`LIBXS_DATATYPE(TYPE)`	Enumerator of a C type: `double`, `float`, `int`, `short`, or `char`

```
const char* libxs_typename(libxs_data_t datatype);
```

Name of the type: `"f64"`, `"f32"`, `"c64"`, `"c32"`, `"i64"`, `"u64"`, `"i32"`, `"u32"`, `"i16"`, `"u16"`, `"i8"`, `"u8"`, or `"void"` if unknown.

```
int libxs_dvalue(libxs_data_t datatype, const void* value, double* dvalue);
```

Convert the real or integer value at `value` to `double`. Returns `EXIT_SUCCESS`, or `EXIT_FAILURE` for a NULL `value`, a complex, or an unknown type.

Version

Macro	Description
`LIBXS_VERSION_MAJOR`, `_MINOR`, `_UPDATE`, `_PATCH`	Version components
`LIBXS_VERSION_NUMBER`	`MAJOR * 10000 + MINOR * 100 + UPDATE`
`LIBXS_VERSION_CHECK(COMP, MAJOR, MINOR, UPDATE)`	Compare the version, e.g., `LIBXS_VERSION_CHECK(<, 2, 0, 0)`
`LIBXS_VERSION_GE(MAJOR, MINOR, UPDATE)`	Version is at least the given one
`LIBXS_VERSION2(MAJOR, MINOR)`, `LIBXS_VERSION3(MAJOR, MINOR, UPDATE)`	Encode a version number (also for compiler versions)
`LIBXS_BUILD_DATE`	Build date as `YYYYMMDD`

The values come from `libxs_version.h`, which the build generates (`make` or CMake) alongside the headers; it also defines `LIBXS_VERSION` (string) and `LIBXS_BRANCH`. `libxs_macros.h` includes it wherever it is found, which needs `__has_include` outside of the LIBXS build itself. Without it, the components default to 0, except in a header-only build, which compiles all of LIBXS and hence reports 9999.99.99 so that no version gate excludes code.

```
const char* libxs_version(int* major, int* minor, int* update, int* patch);
```

Version of the library as built, which can differ from the macros an application was compiled with (e.g., a newer shared library). Returns the version string, and stores the components into the arguments that are not NULL. From Fortran (`USE LIBXS`), the arguments are OPTIONAL and the result is a `TYPE(C_PTR)` to the string.

Common Macros

Macro	Description
`LIBXS_MIN(A, B)`, `LIBXS_MAX(A, B)`	Minimum, maximum
`LIBXS_ABS(A)`, `LIBXS_SIGN(A)`	Absolute value, sign (-1, 0, 1)
`LIBXS_DELTA(A, B)`	Absolute difference
`LIBXS_CLMP(VALUE, LO, HI)`	Clamp to `[LO, HI]`
`LIBXS_UPDIV(N, MULT)`	`N / MULT` rounded up
`LIBXS_UP(N, MULT)`	`N` rounded up to a multiple of `MULT`
`LIBXS_UP2(N, NPOT)`, `LIBXS_LO2(N, NPOT)`	Round up/down to a multiple of a power of two
`LIBXS_UP2POT(N)`, `LIBXS_LO2POT(N)`	Round up/down to a power of two
`LIBXS_ISPOT(A)`	Non-zero if `A` is a power of two
`LIBXS_MOD2(A, NPOT)`	`A` modulo a power of two
`LIBXS_DELIMS`	Default list separators `",;:"`; the first character is the default output separator
`LIBXS_CACHELINE`, `LIBXS_ALIGNMENT`	Cache-line size and default alignment (64 bytes)

Arguments may be evaluated more than once.

Platform symbols: `LIBXS_PLATFORM_X86`, `LIBXS_PLATFORM_AARCH64`, or `LIBXS_PLATFORM_RV64` is defined for the target, and `LIBXS_BITS` is the width of a pointer.

API Declarations for Dependent Libraries

A library depending on LIBXS can reuse its declaration scheme, which covers a shared or static build of the library, its users, and header-only use:

```
#if defined(MYLIB_SOURCE_H)
## define MYLIB_BUILD_KIND LIBXS_APIKIND_INLINE
#elif defined(MYLIB_BUILD)
## define MYLIB_BUILD_KIND LIBXS_APIKIND_EXPORT
#else
## define MYLIB_BUILD_KIND LIBXS_APIKIND_IMPORT
#endif
#define MYLIB_API        LIBXS_API_DECL(MYLIB_BUILD_KIND)
#define MYLIB_API_INTERN LIBXS_API_DECL_INTERN(MYLIB_BUILD_KIND)
```

`LIBXS_API_DECL(KIND)` decorates a public function, `LIBXS_API_DECL_INTERN(KIND)` a function that is not exported. Variables use `LIBXS_APIVAR_DECL_PUBLIC(DECL, KIND)` in a header with `LIBXS_APIVAR_DECL_PUBLIC_DEF(DECL, KIND)` in the source file, or the `_PRIVATE` and `_PRIVATE_DEF` counterparts, and `LIBXS_APIVAR_DECL_DEFINE(DECL, KIND)` declares and defines a private variable at once.

Header-Only Use

Including `libxs_source.h`, or compiling with `-DLIBXS_SOURCE`, provides the implementation without linking a library (see the Usage section of the overview). `LIBXS_BUILD` must not be defined in this case.

CPU Identification

Header: `libxs_cpuid.h`

Portable CPU feature detection for x86-64, AArch64, and RISC-V targets. Returns an ISA level that can be compared numerically — higher values indicate more capable instruction sets. x86 levels use thermometer ordering: higher numeric value implies all features of lower levels. AVX10/256 ranks below AVX512 (no 512-bit vectors). `LIBXS_X86_AVX512_INT8` and above need AVX10.2, and AMX CPUs without it report as `LIBXS_X86_AVX512_AMX`.

ISA Constants

Constant	Value	Architecture
`LIBXS_TARGET_ARCH_UNKNOWN`	0	Unknown / unsupported
`LIBXS_TARGET_ARCH_GENERIC`	1	Portable scalar baseline
`LIBXS_X86_GENERIC`	1002	x86-64 baseline
`LIBXS_X86_SSE3`	1003	SSE3
`LIBXS_X86_SSE42`	1004	SSE4.2
`LIBXS_X86_AVX`	1005	AVX
`LIBXS_X86_AVX2`	1006	AVX2 + FMA
`LIBXS_X86_AVX10_256`	1030	AVX10.1/256: all features, 256-bit max
`LIBXS_X86_AVX512`	1100	AVX-512 (F + CD + DQ + BW + VL + VNNI)
`LIBXS_X86_AVX512_AMX`	1105	AVX-512 + AMX (TILE + INT8 + BF16); Sapphire/Granite Rapids
`LIBXS_X86_AVX512_INT8`	1110	AMX + AVX10.2: 512-bit VPDPBUUD/BSSD
`LIBXS_X86_AVX10_512`	1200	AVX10.2/512 + AMX: full feature set
`LIBXS_AARCH64`	2001	ARMv8.1 baseline
`LIBXS_AARCH64_SVE128`	2201	SVE 128-bit
`LIBXS_AARCH64_SVE256`	2301	SVE 256-bit
`LIBXS_AARCH64_SVE512`	2401	SVE 512-bit
`LIBXS_RV64_MVL128`	3001	RISC-V RVV 128-bit
`LIBXS_RV64_MVL256`	3002	RISC-V RVV 256-bit
`LIBXS_RV64_MVL128_LMUL`	3003	RISC-V RVV 128-bit (non-unit LMUL)
`LIBXS_RV64_MVL256_LMUL`	3004	RISC-V RVV 256-bit (non-unit LMUL)
`LIBXS_X86_ALLFEAT`	1999	x86 sentinel (all features)
`LIBXS_AARCH64_ALLFEAT`	2999	AArch64 sentinel (all features)

Constant	Value	Architecture
LIBXS_RV64_ALLFEAT	3999	RISC-V sentinel (all features)

Types

```
typedef struct libxs_cpuid_t {
  char model[256];   /* CPU model name from OS */
  int constant_tsc;  /* non-zero if TSC is invariant */
} libxs_cpuid_t;
```

Functions

```
int libxs_cpuid(libxs_cpuid_t* info);
```

Detect the ISA level of the current platform. Optionally fills `info` with the CPU model name and TSC capability. Returns a constant from the table above.

```
const char* libxs_cpuid_name(int id);
```

Returns a human-readable string for the given ISA constant, e.g., `"avx2"`.

```
int libxs_cpuid_id(const char* name);
```

Translates a name (e.g., `"avx512"`) to the corresponding ISA constant.

```
int libxs_cpuid_vlen(int id);
```

Returns the SIMD vector length in bytes for the given ISA level (0 for scalar-only targets).

```
int libxs_cpuid_amx_enable(void);
```

Request AMX tile state (XTILE_DATA) from the OS. Must be called before any AMX tile instruction. Returns `EXIT_SUCCESS` if tiles are ready (or were already enabled), `EXIT_FAILURE` on unsupported platform or OS refusal. Not called automatically by `libxs_cpuid`.

GEMM: Matrix Multiplication

Header: `libxs_gemm.h` Fortran: `USE LIBXS` (libxs/libxs.f)

Batched general matrix-matrix multiplication (GEMM) and symmetric rank-k/2k updates. Operations are expressed as C := alpha * op(A) * op(B) + beta * C, where op() is an optional transpose. Kernels are dispatched via MKL JIT, LIBXSMM, or BLAS; a built-in default kernel (auto-vectorized) is used as fallback.

Kernel Providers

LIBXS does not link a kernel provider. MKL JIT, LIBXSMM, and BLAS are linked by the application, and LIBXS receives their entry points as function pointers (`libxs_gemm_backend_t`). One LIBXS build hence serves applications with or without MKL or LIBXSMM, and the provider's version is the one the application links.

C. The dispatch functions are header inlines. They are compiled in the application's translation unit and take the address of what the application's headers declare (`libxs_gemm_backend_init`):

Provider	Enabled by
MKL JIT and BLAS	`-D__MKL`, or `mkl.h` included before `libxs_gemm.h`
LIBXSMM	`-D__LIBXSMM`, or `libxsmm.h` included before `libxs_gemm.h`
BLAS	`-D__BLAS` (Fortran `dgemm_`/`sgemm_`)

```
#include <mkl.h> /* or libxsmm.h */
#include <libxs/libxs_gemm.h>
```

LIBXS_GEMM_JIT is defined if a generator was found. To pass pointers explicitly, fill a libxs_gemm_backend_t and call one of the _rt forms.

Fortran. libxs/libxs_jit.F plays the role of the header inlines. It is not part of the library: the application compiles it with its own sources and flags (preprocessed, hence .F), and uses USE LIBXS_JIT in place of USE LIBXS. The module passes the provider's entry points as procedure pointers (C_FUNLOC) to the dispatch of module LIBXS, where they are OPTIONAL C_FUNPTR arguments.

```
$FC -D__LIBXS [-D__MKL] [-D__LIBXSMM] -I<libxs>/libxs \
  -c <libxs>/libxs/libxs_jit.F
```

-D__LIBXS is the application's flag for using LIBXS; without it, the module falls back to plain BLAS, so that call sites need not change when LIBXS is optional. BLAS is required at link time.

Runtime. Where dynamic linking is supported, the library also looks up dgemm_, sgemm_, the MKL JIT entry points, and libxsmm_dispatch_gemm among the symbols already loaded by the process, and uses them for any backend pointer left NULL. Linking a provider into the application is then sufficient, even without the compile-time detection above.

Types

```
typedef struct libxs_gemm_shape_t {
  libxs_data_t datatype;
  char transa, transb;
  int m, n, k, lda, ldb, ldc;
  double alpha, beta;
} libxs_gemm_shape_t;
```

GEMM shape: problem geometry, transpose flags, and scalar coefficients. Serves as registry key when caching dispatched configurations. Dispatch builds its own key from the members, hence a caller-supplied shape need not be zero-initialized.

```
typedef struct libxs_gemm_config_t {
  libxs_gemm_dblas_t dgemm_blas;
  libxs_gemm_sblas_t sgemm_blas;
  libxs_gemm_djit_t dgemm_jit;
  libxs_gemm_sjit_t sgemm_jit;
  libxs_gemm_xfn_t xgemm;
  void* jitter;
  libxs_gemm_flags_t flags;
  int warmup;
  libxs_gemm_shape_t shape;
} libxs_gemm_config_t;
```

Configuration holding dispatched GEMM kernels. Kernel priority:

1. JIT kernel (dgemm_jit/sgemm_jit + jitter),
2. XGEMM kernel (xgemm),
3. BLAS kernel (dgemm_blas/sgemm_blas) — always non-NULL after dispatch (falls back to built-in auto-vectorized C code).

warmup counts dispatches of this shape and is maintained by dispatch (see LIBXS_GEMM_JIT_WARMUP).

```
typedef enum libxs_gemm_flags_t {
  LIBXS_GEMM_FLAGS_DEFAULT = 0,
  LIBXS_GEMM_FLAG_NOLOCK = 1,
  LIBXS_GEMM_FLAG_OWNJIT = 2,
  LIBXS_GEMM_FLAG_STREAM = 4,
  LIBXS_GEMM_FLAG_REUSE = 8
} libxs_gemm_flags_t;
```

Flags controlling batch synchronization. Set LIBXS_GEMM_FLAG_NOLOCK when no duplicate C pointers exist across the batch. LIBXS_GEMM_FLAG_OWNJIT is set by dispatch and marks the single config owning jitter; aliasing configs (double-dispatch, and every caller copy) leave it clear so the handle is released exactly once.

LIBXS_GEMM_FLAG_STREAM and LIBXS_GEMM_FLAG_REUSE are dispatch hints. The copy flavors (_cpy) read them from the caller's config before populating it, and do not keep them:

Hint	Effect
STREAM	a generated kernel may serve operands that are windows into larger matrices (leading dimension beyond the kernel's extent), which the automatic backend otherwise leaves to BLAS
REUSE	the JIT warm-up is skipped, since the caller calls the kernel often (e.g., a batch)

A kernel is shared per shape, whichever hints generated it: a later dispatch of the same shape receives it with or without hints.

```
typedef enum libxs_gemm_kind_t {
  LIBXS_GEMM_KIND_DEFAULT = 0,
  LIBXS_GEMM_KIND_BLAS = 1,
  LIBXS_GEMM_KIND_JIT = 2
} libxs_gemm_kind_t;
```

Kind of kernel serving a GEMM: built-in code, external BLAS, or a generated kernel (MKL JIT or LIBXSMM); see Backend Query.

Function-pointer and helper types used by the config and the backend:

```
typedef void (*libxs_gemm_dblas_t)(
  const char* transa, const char* transb,
  const int* m, const int* n, const int* k,
  const double* alpha, const double* a, const int* lda,
                       const double* b, const int* ldb,
  const double* beta,        double* c, const int* ldc,
  size_t transa_len, size_t transb_len);
typedef void (*libxs_gemm_sblas_t)(
  const char* transa, const char* transb,
  const int* m, const int* n, const int* k,
  const float* alpha, const float* a, const int* lda,
                      const float* b, const int* ldb,
  const float* beta,        float* c, const int* ldc,
  size_t transa_len, size_t transb_len);
typedef void (*libxs_gemm_djit_t)(void* jitter,
  const double* a, const double* b, double* c);
typedef void (*libxs_gemm_sjit_t)(void* jitter,
  const float* a, const float* b, float* c);
typedef struct libxs_gemm_param_t {
  void* op[4]; const void* a[6]; const void* b[6]; void* c[6];
} libxs_gemm_param_t;
typedef void (*libxs_gemm_xfn_t)(const void*);
typedef struct libxs_xgemm_shape_t {
  int m, n, k, lda, ldb, ldc;
  int a_in_type, b_in_type, out_type, comp_type;
} libxs_xgemm_shape_t;
typedef int (*libxs_blas_nthreads_t)(void);
```

- `libxs_gemm_dblas_t`/`libxs_gemm_sblas_t`: Fortran BLAS `dgemm_`/`sgemm_` including the hidden lengths of transa and transb.
- `libxs_gemm_djit_t`/`libxs_gemm_sjit_t`: MKL JIT kernel (`mkl_jit_get_dgemm_ptr`/`mkl_jit_get_sgemm_ptr`); shape and scalars are part of the jitter.
- `libxs_gemm_param_t`: argument of an XGEMM kernel (`libxs_gemm_xfn_t`), layout-compatible with `libxsmm_gemm_param`; only the primary pointers `a[0]`, `b[0]`, `c[0]` are used.
- `libxs_xgemm_shape_t`: shape passed to `xgemm_dispatch`, layout-compatible with `libxsmm_gemm_shape` (`libxsmm_blasint=int`).
- `libxs_blas_nthreads_t`: number of threads the BLAS uses by itself, e.g., `MKL_Get_Max_Threads` (1 means sequential).

Dispatch

```
libxs_gemm_config_t* libxs_gemm_dispatch(
  libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, int lda, int ldb, int ldc,
  const void* alpha, const void* beta,
  void* registry /* = NULL */);
```

Inline function that selects the backend at compile time (see Kernel Providers). On registry hit, returns a pointer to the cached config (hash probe only). On miss, dispatches a new kernel and stores it. If registry is NULL, an internal registry is used.

The detection itself is exposed, so a caller assembling shapes by hand can reach `libxs_gemm_dispatch_rt` without repeating it:

```
void libxs_gemm_backend_init(libxs_gemm_backend_t* backend);
```

It zero-initializes backend and fills the pointers available at the caller's compile time. Unavailable backends are left NULL, i.e., dispatch takes what the library found at runtime, and the built-in default kernel otherwise.

Config ownership (registry pointer or caller copy) Every dispatch function comes in two flavors. The unsuffixed C form returns a pointer to the registry-owned config; the `_cpy` form populates a caller-owned config instead:

```
int libxs_gemm_dispatch_cpy(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, int lda, int ldb, int ldc,
  const void* alpha, const void* beta,
  void* registry /* = NULL */);

int libxs_gemm_config_cpy(
  libxs_gemm_config_t* config, const libxs_gemm_config_t* cached);
```

Both return nonzero if config was populated (config is left untouched otherwise, hence zero-initialize it before the call). `libxs_gemm_config_cpy` converts a pointer obtained from any dispatch function into a caller-owned config.

Pick the flavor by lifetime, not by convenience:

Flavor	Use when
pointer	the config is cached across many calls and should pick up a kernel once JIT warm-up completes
copy	flags are modified, per-thread state is wanted, or dispatch happens per call anyway

A copy is a snapshot. It is not upgraded in place when warm-up completes later, so re-dispatch to pick up a kernel. Writing `flags` through a registry pointer affects every user of that shape, hence a copy is the safe place for `LIBXS_GEMM_FLAG_NOLOCK`. `LIBXS_GEMM_FLAG_OWNJIT` is cleared in a copy: the JIT handle stays owned by the registry, and `libxs_gemm_release` is a no-op for the copy (release the registry, or the config the registry returned).

In Fortran the plain name covers both flavors, and the first argument selects: pass a config to fill it (copy), omit it to receive the registry-owned pointer. The suffixed names remain and name one flavor each, which is what C has to do throughout:

Operation	C pointer	C copy	Fortran (either)	Fortran pointer	Fortran copy
GEMM	`libxs_gemm_dispatch`	`libxs_gemm_dispatch_cpy`	`libxs_gemm_dispatch`	`libxs_gemm_dispatch_ptr`	`libxs_gemm_dispatch`
SYR2K	`libxs_syr2k_dispatch`	`libxs_syr2k_dispatch_cpy`	`libxs_syr2k_dispatch`	`libxs_syr2k_dispatch`	`libxs_syr2k_dispatch_cpy`
SYRK	`libxs_syrk_dispatch`	`libxs_syrk_dispatch_cpy`	`libxs_syrk_dispatch`	`libxs_syrk_dispatch`	`libxs_syrk_dispatch_cpy`

The result type follows the flavor, so the call tells which one ran:

```
ptr = libxs_syrk_dispatch(LIBXS_DATATYPE_F64, n, k, lda, ldc)
rc = libxs_syrk_dispatch(config, LIBXS_DATATYPE_F64, n, k, lda, ldc)
```

All of the above are inlines (or Fortran wrappers) that detect the backend at the caller's compile time (see Kernel Providers). To supply backend pointers explicitly, use the `_rt` forms, which are the only dispatch symbols exported by the library itself:

Operation	Explicit backend (library symbol)	Caller copy
GEMM	`libxs_gemm_dispatch_rt` (takes shape structs)	`libxs_gemm_dispatch_cpy_rt`

Operation	Explicit backend (library symbol)	Caller copy
SYR2K	`libxs_syr2k_dispatch_rt`	`libxs_syr2k_dispatch_cpy_rt`
SYRK	`libxs_syrk_dispatch_rt`	`libxs_syrk_dispatch_cpy_rt`

The `_cpy_rt` forms take a config as first argument and return nonzero if it was populated, like the `_cpy` inlines.

```
typedef struct libxs_gemm_backend_t {
  libxs_jit_create_dgemm_t jit_create_dgemm;
  libxs_jit_get_dgemm_t   jit_get_dgemm;
  libxs_jit_create_sgemm_t jit_create_sgemm;
  libxs_jit_get_sgemm_t   jit_get_sgemm;
  libxs_xgemm_dispatch_t  xgemm_dispatch;
  libxs_gemm_dblas_t dgemm_blas;
  libxs_gemm_sblas_t sgemm_blas;
  libxs_blas_nthreads_t blas_nthreads;
} libxs_gemm_backend_t;

libxs_gemm_config_t* libxs_gemm_dispatch_rt(
  const libxs_gemm_shape_t* shape,
  const libxs_gemm_shape_t* kernel_shape,
  const libxs_gemm_backend_t* backend,
  void* registry);
```

Non-inline function that accepts backend and shape structs. shape: full problem shape (registry key, stored in config). kernel_shape: actual kernel dimensions (may differ, e.g., tight ldc for scratch). NULL means same as shape. If kernel_shape differs from shape, the kernel is looked up under kernel_shape first (double-dispatch), avoiding redundant code generation. backend: function pointers for backends. A NULL backend, or a NULL member, takes what the library found at runtime, and the built-in default otherwise. blas_nthreads tells whether the BLAS is threaded (e.g., `MKL_Get_Max_Threads`) where the library cannot find out itself (static link). Same registry semantics as above. The BLAS pointers take the two hidden lengths of transa and transb as trailing `size_t` arguments (always 1), which a BLAS compiled from Fortran expects; a C BLAS ignores them. `LIBXS_GEMM_BACKEND` can restrict the starting point of the runtime fallback chain: 0 = automatic/default, 1 = MKL JIT, 2 = LIBXSMM, 3 = BLAS/MKL, 4 = built-in fallback. Choices 1-3 still fall through to lower-priority backends if the requested backend is not supplied or cannot dispatch the shape.

Automatic selection takes a generated kernel only for resident operands, i.e., leading dimensions that match the kernel's own extent; a larger leading dimension reaches BLAS. `LIBXS_GEMM_BACKEND=1` or `=2` requests the respective generator regardless.

Among the generators, automatic selection tries the MKL JIT first and LIBXSMM second, which in addition requires `alpha` of one and `beta` of zero or one. A shape that neither generator serves reaches BLAS.

Backend callback signatures (MKL-compatible):

```
jit_create_dgemm: int(void** jitter, int layout, int transa,
                      int transb, int m, int n, int k,
                      double alpha, int lda, int ldb,
                      double beta, int ldc)
                  Return: MKL_JIT_SUCCESS (0) or
                          MKL_NO_JIT (1) on success,
                          MKL_JIT_ERROR (2) on failure.

jit_get_dgemm:    void*(void* jitter)
                  Return kernel function pointer.

xgemm_dispatch:   libxs_gemm_xfn_t(
                      libxs_xgemm_shape_t shape /* by value */,
                      unsigned int gemm_flags,
                      unsigned int prefetch_flags)
                  Layout-compatible with libxsmm_dispatch_gemm.
                  shape: m,n,k,lda,ldb,ldc then a_in_type,
                         b_in_type, out_type, comp_type.
                  gemm_flags: bit 0 = transa, bit 1 = transb,
                         bit 2 = beta==0.
                  Dispatch is attempted only for alpha==1 with
                  beta in {0,1}; other scalars skip this backend.
```

```
USE LIBXS_JIT
rc = libxs_gemm_dispatch(config, LIBXS_DATATYPE_F64,
     &  'N', 'N', m, n, k, lda, ldb, ldc, alpha, beta)
```

The LIBXS_JIT adapter module provides the same dispatch API but fills in BLAS, and MKL JIT or LIBXSMM when compiled with `-D__MKL` or `-D__LIBXSMM`. The application compiles `libxs_jit.F` itself (see Kernel Providers).

Without `-D__LIBXS`, `libxs_jit.F` compiles to a BLAS-only module LIBXS_JIT that needs no LIBXS library. It offers `libxs_init`, `libxs_finalize`, `libxs_gemm_dispatch` (config flavor), `libxs_gemm_call` (config and one-shot form), `libxs_syrk`, and `libxs_syr2k`. These call DGEMM, DSYRK, and DSYR2K (double precision only), and the SYRK/SYR2K forms take the shape directly:

```
CALL libxs_syrk(datatype, n, k, lda, ldc, uplo, alpha, beta, a, c)
CALL libxs_syr2k(datatype, n, k, lda, ldb, ldc,
     &  uplo, alpha, beta, a, b, c)
```

```
USE LIBXS
rc = libxs_gemm_dispatch(config, datatype, transa, transb,
     &  m, n, k, lda, ldb, ldc, alpha, beta,
     &  jit_create_dgemm=..., jit_get_dgemm=...,
     &  dgemm_blas=..., registry=...)
```

All backend arguments are OPTIONAL C_FUNPTR (named arguments): `jit_create_dgemm`, `jit_get_dgemm`, `jit_create_sgemm`, `jit_get_sgemm`, `xgemm_dispatch`, `dgemm_blas`, `sgemm_blas`, and `blas_nthreads`; `registry` is an OPTIONAL C_PTR. Returns nonzero on success (dispatch produced a callable config). The config is populated from the registry-owned copy.

```
rc = libxs_gemm_config_cpy(config, cached) ! TYPE(C_PTR) :: cached
```

Converts a pointer obtained from a pointer-flavor dispatch into a caller-owned config, like `libxs_gemm_config_cpy` in C.

Backend Query

```
libxs_gemm_kind_t libxs_gemm_backend_kind(
  const libxs_gemm_backend_t* backend /* = NULL */,
  const libxs_gemm_shape_t* shape /* = NULL */);
```

Tells which kind of kernel a dispatch settles on once warmed up: `LIBXS_GEMM_KIND_JIT` (MKL JIT or LIBXSMM), `LIBXS_GEMM_KIND_BLAS` (external BLAS), or `LIBXS_GEMM_KIND_DEFAULT` (built-in code). The query honors `LIBXS_GEMM_BACKEND` and `LIBXS_GEMM_JIT_MAX`, but it neither generates a kernel nor consults a registry, hence it is cheap enough to decide per call. Without a shape, it answers for any shape; without a backend, for the library's own. JIT means a generator is tried, which may still refuse a particular shape. For shapes with resident operands (leading dimensions equal to the extents), JIT for the largest shape holds for all smaller ones.

```
libxs_gemm_backend_t backend;
libxs_gemm_backend_init(&backend);
if (LIBXS_GEMM_KIND_JIT == libxs_gemm_backend_kind(&backend, NULL)) {
  /* generated kernels are available */
}
```

At compile time, `LIBXS_GEMM_JIT` is defined if `libxs_gemm_backend_init` provides a generator, which likewise requires mkl.h or libxsmm.h to be included before libxs_gemm.h.

```
USE LIBXS_JIT
kind = libxs_gemm_backend_kind() ! any shape
kind = libxs_gemm_backend_kind(LIBXS_DATATYPE_F64, 'N', 'N',
     &  m, n, k, lda, ldb, ldc, alpha, beta)
```

With `USE LIBXS`, the backend pointers are OPTIONAL named arguments as for the dispatch.

Single-Kernel Call

```
void libxs_gemm_call(
  const libxs_gemm_config_t* config,
  const void* a, const void* b, void* c);
```

Call the dispatched GEMM kernel (JIT > XGEMM > BLAS fallback). The caller must ensure config is non-NULL (dispatch succeeded).

```
CALL libxs_gemm_call(config, a, b, c)
CALL libxs_gemm_call(datatype, transa, transb, m, n, k,
     &  lda, ldb, ldc, alpha, beta, a, b, c)
```

In Fortran, the second form dispatches and calls in one step (`libxs_gemm_call_direct`); it accepts the OPTIONAL backend arguments and `registry` of the dispatch. `a`, `b`, and `c` are `TYPE(C_PTR)`.

Release

```
void libxs_gemm_release(libxs_gemm_config_t* config);
```

Release resources (e.g., MKL jitter handle) held by config, and clear the JIT kernel such that config falls back to XGEMM/BLAS. A handle can be shared by several configs (double dispatch), hence only the owning config (LIBXS_GEMM_FLAG_OWNJIT, set by dispatch) releases it and a handle is released exactly once. A dispatched config is owned by the registry, i.e., releasing it affects every user of that registry: release at teardown, or rely on libxs_gemm_release_registry.

```
void libxs_gemm_release_registry(libxs_registry_t* registry);
```

Release all configs in a registry, then destroy the registry.

Pointer-Array Batch

```
void libxs_gemm_batch(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config);

void libxs_gemm_batch_task(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Batch of GEMMs from pointer arrays. The _task variant splits work across ntasks threads (tid = 0..ntasks-1).

Index/Strided Batch

```
void libxs_gemm_index(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config);

void libxs_gemm_index_task(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Batch of GEMMs from element-offset index arrays into contiguous buffers. index_base: 0 (C) or 1 (Fortran). index_stride: byte stride between consecutive index entries (sizeof(int) for packed arrays, 0 for constant-stride mode).

```
int libxs_gemm_ntasks(const libxs_gemm_config_t* config,
  int batchsize, int nthreads);
```

Number of tasks worth splitting `libxs_gemm_batch_task` or `libxs_gemm_index_task` into, given nthreads (1 means unsplit); never exceeds batchsize. Unless config->flags carries `LIBXS_GEMM_FLAG_NOLOCK`, a split serializes updates of the same C. Fortran: `libxs_gemm_ntasks(config, batchsize, nthreads)`.

Fortran and OpenMP. When the application compiles the Fortran module with OpenMP, `libxs_gemm_batch` and `libxs_gemm_index` run over an OpenMP team sized by `libxs_gemm_ntasks`, unless called from within a parallel region. The `_task` forms remain for explicit splitting.

SYR2K / SYRK and TAS

Symmetric rank-k updates (SYRK, `libxs_syrk.h`) and tall-and-skinny GEMMs (TAS, `libxs_tas.h`) are built on this dispatch and batch layer, and documented on their own pages.

Compile-Time Tuning

```
LIBXS_GEMM_NWARMUP   JIT warm-up counters (default: 4096)
```

Number of slots for the per-shape reuse counters (must be a power of two). Fewer slots than distinct shapes in flight can delay JIT, but never yield a wrong kernel.

Environment Variables

```
LIBXS_GEMM_BACKEND=N     Select runtime backend chain start:
                         0 auto (default), 1 MKL JIT,
                         2 LIBXSMM, 3 BLAS, 4 built-in fallback.
                         Auto takes a generator only for resident
                         operands, the MKL JIT before LIBXSMM;
                         1 and 2 take the respective one always,
                         except that MKL JIT is never taken for an
                         operand spanning more than 2 GB (its
                         kernels cannot address it); LIBXSMM or
                         BLAS serve such a shape instead.
LIBXS_GEMM_JIT_MAX=N     Arithmetic-intensity threshold for JIT
                         dispatch. JIT/LIBXSMM kernels are only
                         generated when the kernel shape's AI
                         (flops/bytes) is below N (default: 7).
                         Set to 0 to disable JIT entirely.
LIBXS_GEMM_JIT_WARMUP=N  Number of calls a shape must
                         accumulate before JIT compilation is
                         attempted. Shapes called fewer than N
                         times use the BLAS fallback. Reuse is
                         counted per dispatch; the first shape of
                         an empty registry is compiled immediately.
                         Counted in a fixed-size table, not in the
                         registry (see LIBXS_GEMM_NWARMUP); once a
                         shape's JIT has been attempted it is never
                         attempted again, whether it succeeded or
                         was refused by LIBXS_GEMM_JIT_MAX.
                         Set to 0 or 1 to compile immediately
                         on first miss (default: 8, clamped to 253).
LIBXS_GEMM_PRINT=N       Print dispatch info every N-th call
                         to stderr (compile-time gate, enabled
                         by default).
```

```
Set to 0 for a summary at teardown instead
(registry, histogram, and warm-up slots).
Every line carries the origin of the
process, "LIBXS INFO[rid]:" with rid as
libxs_rid,
hence the local rank where it is known and
the process id otherwise.
```

Example (C)

```
#include <libxs/libxs_gemm.h>

/* dispatch once per unique shape, call many times */
libxs_gemm_config_t* cfg = libxs_gemm_dispatch(LIBXS_DATATYPE_F64,
  'N', 'N', m, n, k, lda, ldb, ldc, &alpha, &beta, NULL);

if (NULL != cfg) {
  for (i = 0; i < nmatrices; ++i) libxs_gemm_call(cfg, a[i], b[i], c[i]);
}
```

Example (Fortran)

```
USE :: LIBXS

TYPE(libxs_gemm_config_t) :: config
INTEGER(C_INT) :: rc

rc = libxs_gemm_dispatch(config, LIBXS_DATATYPE_F64,
     &  'N', 'N', m, n, k, lda, ldb, ldc, alpha, beta,
     &  dgemm_blas=C_FUNLOC(DGEMM))

IF (0 /= rc) THEN
  CALL libxs_gemm_call(config, C_LOC(a), C_LOC(b), C_LOC(c))
END IF
! rc /= 0 guarantees libxs_gemm_call will succeed
! (JIT, XGEMM, or BLAS/built-in fallback)
```

Hashing and Checksums

Header: `libxs_hash.h`

Hash functions for buffers, fixed-width keys, and strings. The default hash uses hardware-accelerated CRC32-C (SSE4.2) with a software fallback.

Buffer Hashing

```
unsigned int libxs_hash(const void* data, size_t size,
  unsigned int seed);
```

Produce a 32-bit hash from a byte buffer of the given size. A NULL buffer is accepted only with a size of zero (returns seed).

```
uint64_t libxs_hash64(const void* data, size_t size, uint64_t seed);
```

Produce a 64-bit hash from a byte buffer of the given size. NULL buffer or zero size is accepted (returns seed). Unlike `libxs_hash`, it is not a CRC, so inputs collide only by chance. The value does not depend on the byte order of the host.

Fixed-Width Hashing

```
unsigned int libxs_hash8(unsigned int data);
unsigned int libxs_hash16(unsigned int data);
unsigned int libxs_hash32(uint64_t data);
```

Hash an 8-, 16-, or 32-bit value. The result has the same width (8, 16, or 32 bits).

String Hashing

```
uint64_t libxs_hash_string(const char string[]);
```

64-bit hash of a character string. Short strings (up to 8 bytes) are stored directly, which cannot collide; longer strings use `libxs_hash64`. NULL-string is accepted (returns zero). A short string is stored in the host's byte order, hence its value differs between little- and big-endian hosts.

From Fortran (`USE libxs`), the generic `libxs_hash64` takes a contiguous array of `CHARACTER(C_CHAR)`, `INTEGER(C_INT8_T)`, `INTEGER(C_INT)`, or `INTEGER(C_LONG_LONG)` and an optional seed, and returns `INTEGER(C_INT64_T)`, as `libxs_hash` does for 32 bits. `libxs_hash_string(string)` takes a NUL-terminated `CHARACTER(C_CHAR)` array and returns `INTEGER(C_INT64_T)`. `libxs_hash_iso3309(data, size, seed)` and `libxs_adler32(data, size, seed)` take a `TYPE(C_PTR)`, an `INTEGER(C_SIZE_T)` size, and an `INTEGER(C_INT)` seed (all by value), and return `INTEGER(C_INT)`.

CRC32 (ISO 3309)

```
unsigned int libxs_hash_iso3309(const void* data,
  size_t size, unsigned int seed);
```

CRC-32 using the ISO 3309 polynomial (used by PNG, gzip, and similar formats). Pre-condition and post-XOR with 0xFFFFFFFF are the caller's responsibility. A NULL buffer is accepted only with a size of zero.

Adler-32

```
unsigned int libxs_adler32(const void* data,
  size_t size, unsigned int seed);
```

Adler-32 checksum (the variant used by zlib). The standard initial seed is 1. A NULL buffer is accepted only with a size of zero.

Histogram

Header: `libxs_hist.h`

Histogram with running statistics. Buckets by `vals[0]`; additional values per entry track user-defined statistics. Functions taking a `lock` are thread-safe when one is given; a NULL `lock` means no locking.

Types

```
typedef struct libxs_hist_t libxs_hist_t;  /* opaque */
typedef struct libxs_span_t libxs_span_t;  /* opaque */

typedef void (*libxs_hist_update_t)(double* dst, const double* src, int count);
typedef void (*libxs_hist_fold_t)(void* ctx, double* queue, int nvals, int nsamples);
```

An update function reduces one value into one bucket; the order of updates must not matter. A fold receives a whole batch of samples at once and may keep state of its own, e.g., to relate samples to one another.

```
typedef struct libxs_hist_info_t {
  const int* buckets;   /* per-bucket counts [nbuckets] */
  const double* vals;   /* per-bucket values [nbuckets * nvals] */
  const double* sum;    /* running total per value [nvals] */
  double range[2];      /* [min, max] of vals[0] */
  int nbuckets, nvals, nsamples;
} libxs_hist_info_t;
```

`sum` is the total of the pushed values regardless of the update functions. `nsamples` equals the sum of the bucket counts.

Functions

```
libxs_hist_t* libxs_hist_create(int nbuckets, int nvals,
  const libxs_hist_update_t update[], libxs_hist_fold_t fold, void* ctx);
void libxs_hist_destroy(libxs_hist_t* hist);
```

Create/destroy. `update[nvals]` specifies per-slot accumulation (NULL defaults to avg). `fold` is optional (NULL for none) and receives `ctx` unchanged; the histogram only borrows it, so it must outlive the histogram. Destroy accepts NULL.

```
void libxs_hist_push(libxs_lock_t* lock,
  libxs_hist_t* hist, const double vals[]);
```

Insert one sample. Samples are queued and folded into the buckets in batches, so a histogram is exact while a batch fits and summarizes only once it is full. Re-bins automatically if `vals[0]` exceeds the established range.

```
void libxs_hist_query(libxs_lock_t* lock,
  const libxs_hist_t* hist, libxs_hist_info_t* info);
```

Query statistics, folding whatever is still queued. Repeated calls report the same result rather than folding the same samples twice.

```
void libxs_hist_query_percentile(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[], double percentile);
void libxs_hist_query_median(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[]);
void libxs_hist_query_mode(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[]);
```

Values at a percentile [0..1], at the median (0.5), or of the most populated bucket. All report per-bucket aggregates, blending only towards a populated neighbour. Prefer the mode for a multi-modal distribution; its `vals[0]` is the bucket's upper bound rather than an aggregate.

```
void libxs_hist_print(FILE* ostream, const libxs_hist_t* hist,
  const int prec[], const char fmt[], ...);
```

Print to stream: `fmt` (may be NULL) is a `printf` format for a header line, followed by its arguments. Each non-empty bucket then prints one line with its number, upper bound, count, and values. `prec[k]` (`nvals` entries, or NULL for `%f`) sets the precision of value `k`, and `prec[0]` also that of the bound; a negative entry omits its value, and a negative `prec[0]` omits the bucket lines. A NULL `ostream` prints nothing. Printing takes no lock.

Interval Union

```
libxs_span_t* libxs_span_create(int nsegments);
void libxs_span_destroy(libxs_span_t* span);
void libxs_span_push(libxs_span_t* span, double begin, double end);
double libxs_span_total(const libxs_span_t* span, int* inexact);

void libxs_hist_fold_union(void* ctx, double* queue, int nvals, int nsamples);
```

The time a set of intervals covers, counting overlaps once. `nsegments` bounds how far out of order intervals may arrive, not how many there may be: an interval beginning before what has already been retired is added whole and counted in `inexact`, and the total is then an upper bound.

`libxs_hist_fold_union` is a ready-made `libxs_hist_fold_t` that expects `ctx` to be a `libxs_span_t*` and reads the last two values of a sample as `(begin, end)`; pass it to `libxs_hist_create` to obtain both a distribution of durations and the time they covered from one push.

Values must share an origin a double can hold exactly: subtract an epoch from wall-clock nanoseconds first.

Update Functions

```
void libxs_hist_update_avg(double* dst, const double* src, int count);
void libxs_hist_update_add(double* dst, const double* src, int count);
void libxs_hist_update_min(double* dst, const double* src, int count);
void libxs_hist_update_max(double* dst, const double* src, int count);
```

- `avg` — Welford online mean: `*dst += (*src - *dst) / count`
- `add` — sum: `*dst += *src`
- `min` — `*dst = min(*dst, *src)`
- `max` — `*dst = max(*dst, *src)`

Custom callbacks use the same signature; `count` enables online algorithms without external state.

Memory Allocation

Header: `libxs_malloc.h`

Pool-based memory allocator designed for steady-state performance: after an initial warm-up phase, allocations are served from a recycled pool without system calls.

Types

```
typedef struct libxs_malloc_info_t {
  size_t size;  /* allocated size in bytes */
} libxs_malloc_info_t;

typedef struct libxs_malloc_pool_hist_t {
  size_t count;         /* allocations in this bucket */
  size_t nreuses;       /* chunk reused without reallocation */
  size_t ngrows;        /* chunk reallocated (too small) */
  size_t nevicts_age;   /* evictions triggered by age */
  size_t nevicts_limit; /* evictions triggered by memory limit */
} libxs_malloc_pool_hist_t;
```

```
typedef struct libxs_malloc_pool_info_t {
  size_t used;      /* memory currently in use (sum of requested sizes) */
  size_t size;      /* total allocated memory (sum of actual chunk sizes) */
  size_t peak;      /* peak of the total allocated memory */
  size_t nactive;   /* pending (not yet freed) allocations */
  size_t nmallocs;  /* total allocation count */
  libxs_malloc_pool_hist_t hist[6]; /* per-bucket histogram */
} libxs_malloc_pool_info_t;
```

The histogram uses 6 logarithmic buckets keyed on requested size: <1M, 1-4M, 4-16M, 16-64M, 64-256M, >=256M. Each bucket tracks how allocations in that size class were served (reuse vs. grow) and how many were evicted by the pool's memory-pressure heuristics.

```
typedef struct libxs_malloc_pool_t libxs_malloc_pool_t; /* opaque */

typedef void* (*libxs_malloc_fn)(size_t size);
typedef void  (*libxs_free_fn)(void* pointer);

typedef void* (*libxs_malloc_xfn)(size_t size, const void* extra);
typedef void  (*libxs_free_xfn)(void* pointer, const void* extra);
```

Pool Management

```
libxs_malloc_pool_t* libxs_malloc_pool(libxs_malloc_fn malloc_fn, libxs_free_fn free_fn);
```

Create a memory pool. If `malloc_fn` and `free_fn` are both NULL, the standard `malloc`/`free` are used. Both must be NULL or both must be non-NULL.

```
libxs_malloc_pool_t* libxs_malloc_xpool(libxs_malloc_xfn malloc_fn, libxs_free_xfn free_fn,
  int max_nthreads);
```

Create a memory pool with extended allocator functions that receive a per-thread extra argument (see `libxs_malloc_arg`). The `max_nthreads` parameter determines the size of the internal per-thread argument table (indexed by `libxs_tid`). Both function pointers must be non-NULL and `max_nthreads` must be positive; otherwise NULL is returned.

```
void libxs_malloc_arg(libxs_malloc_pool_t* pool, const void* extra);
```

Set the per-thread extra argument for an extended pool (created by `libxs_malloc_xpool`). The pointer is stored at the calling thread's slot (`libxs_tid() % max_nthreads`) and passed to the registered `malloc_xfn`/`free_xfn` on subsequent allocations and frees from this thread. No-op for standard pools. Access is lock-free since each thread writes only its own slot.

```
void libxs_free_pool(libxs_malloc_pool_t* pool);
```

Destroy the pool and release all associated memory. Accepts NULL.

```
int libxs_malloc_pool_info(const libxs_malloc_pool_t* pool, libxs_malloc_pool_info_t* info);
```

Query aggregate pool statistics including the per-bucket histogram. A NULL `pool` selects the default pool.

```
void libxs_malloc_pool_print(FILE* ostream, const char prefix[],
  const libxs_malloc_pool_t* pool);
```

Print pool statistics and per-bucket histogram to `ostream`. Each non-empty bucket is printed on its own line showing count, reuse, grow, and eviction numbers. The `prefix` string (may be NULL) is prepended to each line. No output is produced if the pool has zero allocations or `ostream` is NULL.

```
libxs_malloc_pool_t* libxs_default_pool(void);
```

Access the default host memory pool (created at `libxs_init`). Printed automatically at `libxs_finalize` when `LIBXS_VERBOSE` >= 3 (or negative), as is every pool destroyed by `libxs_free_pool`.

General Allocation

```
void* libxs_malloc(libxs_malloc_pool_t* pool, size_t size, int alignment);
```

Allocate `size` bytes from the given `pool`. `LIBXS_MALLOC_AUTO` (0) uses automatic alignment with inline metadata. `LIBXS_MALLOC_NATIVE` (1) preserves the allocator's native pointer (out-of-band metadata via registry). Values greater than 1 are interpreted as explicit alignment in Bytes (inline metadata). A NULL `pool` selects the default pool. Returns NULL on failure or if `size` is zero.

```
void libxs_free(void* pointer);
```

Return memory to its originating pool (derived internally). Accepts NULL.

```
int libxs_malloc_info(const void* pointer, libxs_malloc_info_t* info);
```

Query the size of an allocation made with `libxs_malloc`. The pool is derived internally.

Environment

`LIBXS_MALLOC_LIMIT` bounds the pool in MB. Beyond it, an allocation first evicts a cached chunk, which returns its memory to the system. A negative value removes the bound. Unset, the bound follows the pool's high-water mark plus headroom. A bound below the high-water mark does not reduce what is held; it only causes more eviction. `LIBXS_VERBOSE=3` prints the pool report, including the high-water mark and the bound.

`LIBXS_MALLOC_SHARE` caps the derived bound at a percentage of the memory the process may use (default 50). With several MPI ranks on a node (`libxs_nranks`), each rank receives its part of the share. It does not apply when `LIBXS_MALLOC_LIMIT` is set.

`LIBXS_MALLOC_EVICT_AGE` evicts a cached chunk that was not used for this many ticks of 1024 allocations from its pool (default 8). A negative value disables eviction by age, leaving the bound above as the only eviction. A larger age reallocates less often, at the cost of holding idle memory longer. The pool report (`LIBXS_VERBOSE=3`) counts both kinds of eviction per size class.

`LIBXS_MALLOC_NATIVE` selects where `libxs_malloc` keeps its metadata. Unset, `LIBXS_MALLOC_NATIVE` allocations keep the native pointer (out-of-band metadata) and `LIBXS_MALLOC_AUTO` uses inline metadata. A non-zero value makes `LIBXS_MALLOC_AUTO` keep the native pointer as well; zero makes both use inline metadata.

Fixed-Size Pool

A lightweight fixed-size pool for scenarios where the element size is known at initialization time.

```
void libxs_pmalloc_init(size_t size, size_t* num,
  void* pool[], void* storage);
```

Partition `storage` into `*num` chunks of `size` bytes and register them in `pool`.

```
void* libxs_pmalloc(void* pool[], size_t* num);
void* libxs_pmalloc_lock(void* pool[], size_t* num,
  libxs_lock_t* lock);
```

Pop one chunk from the pool. The `_lock` variant uses a caller-provided lock; the plain variant uses an internal lock.

```
void libxs_pfree(void* pointer, void* pool[], size_t* num);
void libxs_pfree_lock(void* pointer, void* pool[],
  size_t* num, libxs_lock_t* lock);
```

Push a chunk back into the pool.

Math Utilities

Header: `libxs_math.h`

Matrix Difference

```
typedef struct libxs_matdiff_t {
  double norm1_abs, norm1_rel;  /* one-norm */
  double normi_abs, normi_rel;  /* infinity-norm */
  double normf_rel;             /* Frobenius-norm (relative) */
  double linf_abs, linf_rel;    /* max difference (abs/rel at same element) */
  double l2_abs, l2_rel, rsq;   /* L2-norm and R-squared */
  double l1_ref, min_ref, max_ref, avg_ref, var_ref;  /* reference stats */
  double l1_tst, min_tst, max_tst, avg_tst, var_tst;  /* test stats */
  double diag_min_ref, diag_max_ref;   /* diagonal min/max (reference) */
  double diag_min_tst, diag_max_tst;   /* diagonal min/max (test) */
  double v_ref, v_tst;          /* values at max-diff location */
  double grade;                 /* componentwise grade (matdiff_grade) */
  double w;                     /* cumulative weight for online mean */
  int m, n, i, r;               /* location and reduction count */
} libxs_matdiff_t;
```

The fields linf_abs, linf_rel, and v_ref/v_tst always refer to the same element. For complex types, statistics use modulus; differences use complex absolute error.

The `w` field accumulates the element count (m*n) across reductions, so avg_ref/avg_tst of a reduction are the element-weighted means.

```
int libxs_matdiff(libxs_matdiff_t* info,
  libxs_data_t datatype, int m, int n,
  const void* ref, const void* tst,
  const int* ldref, const int* ldtst);
```

Compute scalar differences between two matrices. Supports `F64`, `F32`, `I64`, `I32`, `U32`, `I16`, `U16`, and `I8`, plus `LIBXS_DATATYPE_C64` and `LIBXS_DATATYPE_C32` (interleaved complex; dimensions refer to complex elements). Any other type (`U8`, `U64`) returns `EXIT_FAILURE`.

```
int libxs_matdiff_grade(libxs_matdiff_t* info,
  libxs_data_t datatype, int m, int n,
  const void* ref, const void* tst, const void* bound,
  const int* ldref, const int* ldtst, const int* ldbnd);
```

Everything `libxs_matdiff` calculates, plus `grade`, which `libxs_matdiff` leaves zero. The caller supplies `bound`, the element-wise upper bound on the exact result (for a GEMM: `|alpha||A||B| + |beta||C|`). The grade is dimensionless (units of roundoff), so the threshold stays with the caller: a componentwise-stable implementation stays at or below a modest multiple of the number of terms summed. `LIBXS_DATATYPE_F64` and `F32` only; any other type is reported rather than silently left ungraded.

```
double libxs_matdiff_epsilon(const libxs_matdiff_t* input);
```

Combined error margin from absolute and relative norms. If the environment variable `LIBXS_MATDIFF` names a file, the margin is appended to it as one line; if it names a directory, the line goes to `libxs_matdiff.log` inside it. Further space-separated words in `LIBXS_MATDIFF` are appended to the line, followed by the command line.

```
int libxs_matdiff_combine(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
```

Combine two single-matrix infos (`ref=NULL`) into a meta-diff. Per-side statistics are exact; `linf_abs` is the mean shift, `l2_abs` a statistical bound.

```
void libxs_matdiff_reduce(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
void libxs_matdiff_clear(libxs_matdiff_t* info);
```

Worst-case reduction of matdiff results. Initialize with `libxs_matdiff_clear`.

```
double libxs_matdiff_posdef(const libxs_matdiff_t* info);  /* inline */
```

Returns the smallest test-side diagonal element (positive = necessary condition for positive definiteness met). Zero if no diagonal data.

Environment variables read by `libxs_matdiff`:

- `LIBXS_DUMP`: unset, empty, or `0` disables it (default). Otherwise the reference and test matrices are written as MHD files `<prefix>-<address>-ref.mhd` and `<prefix>-<address>-tst.mhd`. The prefix is the value itself unless it starts with a digit or `-`, in which case it is `libxs_dump`. A value starting with `-` writes only matrices that differ, and `-2` (or higher) additionally prints a passed/failed line per comparison. A number whose magnitude exceeds 1 (e.g., `2`) writes the data reshaped into a near-square two-dimensional array. Files are written only in builds with MHD support (the default configuration).
- `LIBXS_MATDIFF_SHUFFLE`: a nonzero integer (or an empty value) visits the elements in a co-prime shuffled order rather than column-major order; unset or `0` keeps the default order.

From Fortran (`USE libxs`), `libxs_matdiff(info, datatype, m[, n, ref, tst, ldref, ldtst, bound, ldbnd, rc])` takes `TYPE(C_PTR)` matrices and optional leading dimensions; passing `bound` selects `libxs_matdiff_grade`, and `rc` receives the status. `libxs_matdiff_reduce`, `libxs_matdiff_clear`, `libxs_matdiff_epsilon`, and `libxs_matdiff_combine` are bound directly.

Multiset Distance

Order-independent distance between two vectors treated as multisets. Counts elements that cannot be matched 1-to-1 within the given tolerance. The result satisfies metric properties: symmetry, non-negativity, and identity of indiscernibles.

Complex values (C64, C32) are matched by the modulus of their difference. The tolerance is inclusive (less-than-or-equal).

```
int libxs_setdiff(libxs_data_t datatype,
  const void* a, int na,
  const void* b, int nb, double tol);
```

Supports all `libxs_data_t` types except `U64`. Returns the number of unmatched elements, or -1 for unsupported types. The distance is at least abs(na - nb) when the vector lengths differ.

```
int libxs_setdiff_min(libxs_data_t datatype,
  const void* a, int na,
  const void* b, int nb, double* tol);
```

Minimizes the multiset distance over all tolerances and returns the minimum unmatched count. The pointer tol (may be NULL) receives the smallest tolerance that achieves this minimum. Supported types: F64, F32, C64, C32 only (integer types are not meaningful here; returns max(na, nb) with tol set to zero).

Polynomial Fingerprint

Structural fingerprint for n-dimensional data based on Macaulay bracket (truncated-power) polynomials. For 1-D data, the fingerprint records per-derivative-order norms of the forward finite differences, normalized to the unit interval [0,1]. Higher dimensions are handled hierarchically: the innermost dimension is fingerprinted first, each child fingerprint is collapsed to a Sobolev self-norm scalar, and those scalars form the 1-D input for the next outer dimension.

Because of this normalization, datasets of different lengths (or shapes) produce directly comparable fingerprints. Order 0 reflects value magnitude, order 1 slope, order 2 curvature, and so on.

Uses: compare datasets across resolutions or shapes (`libxs_fprint_diff`), judge smoothness and compressibility (`libxs_fprint_decay`), discover the element type of opaque data (`LIBXS_DATATYPE_UNKNOWN`), fingerprint streams or merge partial results (`libxs_fprint_partial`, `libxs_fprint_join`), rate roughness against a smooth reference (`libxs_fprint_bernstein`), and measure an exact distance between two sample sets (`libxs_fprint_bernstein_diff`).

```
#define LIBXS_FPRINT_MAXORDER 8

typedef struct libxs_fprint_t {
  double l2[LIBXS_FPRINT_MAXORDER + 1];
  double l1[LIBXS_FPRINT_MAXORDER + 1];
  double linf[LIBXS_FPRINT_MAXORDER + 1];
  double mean[LIBXS_FPRINT_MAXORDER + 1];
  double acc_sq[LIBXS_FPRINT_MAXORDER + 1];
  double acc_abs[LIBXS_FPRINT_MAXORDER + 1];
  double acc_sum[LIBXS_FPRINT_MAXORDER + 1];
  double tail[LIBXS_FPRINT_MAXORDER + 1];
  double bernstein[LIBXS_FPRINT_MAXORDER + 1];
  int order, n;
  int nk[LIBXS_FPRINT_MAXORDER + 1];
  int nbernstein;
  libxs_data_t datatype;
} libxs_fprint_t;
```

Three norm families and a signed mean are computed per derivative order k = 0..order:

Field	Description
`l2[k]`	L2 norm of the k-th finite difference
`l1[k]`	Mean absolute value (total variation)
`linf[k]`	Maximum absolute value (worst-case decay)
`mean[k]`	Signed mean (sum / count, preserves sign/phase)
`acc_sq[k]`	Accumulator: sum of raw squared differences
`acc_abs[k]`	Accumulator: sum of raw absolute differences
`acc_sum[k]`	Accumulator: sum of raw signed differences
`tail[k]`	Last value at derivative level k (for partial)
`nk[k]`	Count of differences accumulated at order k
`bernstein[k]`	Bernstein derivative norm (if requested)
`nbernstein`	Orders held in bernstein (0: none)

All norms are normalized to the unit interval (h = 1/(n-1)).

The `order` field records how many derivative orders were used; `n` is the extent of the fingerprinted dimension. The `datatype` field records the element type — either the type passed by the caller, or the type discovered by probing when the caller passes LIBXS_DATATYPE_UNKNOWN (see Type Discovery below).

To recover raw (unnormalized) finite-difference magnitudes, use libxs_fprint_raw (see Generalized Binomial and Distance section).

```
double libxs_fprint_decay(const libxs_fprint_t* info);
```

Geometric-mean per-order decay ratio:

```
r = (l2[K] / l2[0])^(1/K) / (n - 1)
```

A ratio well below 1 indicates smooth (compressible) data, and a ratio between 1 and 2 indicates noise; 1e30 is returned when no ratio can be computed (order == 0, l2[0] == 0, or n < 2).

```
typedef enum libxs_fprint_flags_t {
  LIBXS_FPRINT_DEFAULT  = 0,
  LIBXS_FPRINT_SORT     = 1,
  LIBXS_FPRINT_AUTOCORR = 2,
  LIBXS_FPRINT_PERAXIS  = 4,
  LIBXS_FPRINT_WITH_BERNSTEIN = 8
} libxs_fprint_flags_t;

int libxs_fprint(libxs_fprint_t* info,
  libxs_data_t datatype, const void* data,
  int ndims, const size_t shape[], const size_t stride[],
  int order, int axis, int smooth, unsigned int flags);
```

Build a fingerprint from data described by shape and stride arrays. For 1-D data, pass ndims=1, shape={n}, stride=NULL. For higher dimensions, shape[0] is the innermost extent and shape[ndims-1] the outermost. When stride is NULL, contiguous storage is assumed (stride[0]=1, stride[k]=product of shape[0..k-1]). The requested order is clamped to min(order, extent-1, LIBXS_FPRINT_MAXORDER).

The flags parameter is a bitwise OR of preprocessing options:

- `LIBXS_FPRINT_SORT`: sort values before differencing (permutation-invariant fingerprint).
- `LIBXS_FPRINT_AUTOCORR`: replace the signal with its autocorrelation before differencing (shift-invariant).
- `LIBXS_FPRINT_PERAXIS`: per-axis mode. Differentiates along dimension `axis` only and takes the per-order maximum of each norm across all positions in the remaining dimensions.
- `LIBXS_FPRINT_WITH_BERNSTEIN`: also store the libxs_fprint_bernstein norms of the samples as differenced (1-D only, at the cost of that call); libxs_fprint_diff then compares them as well.

Without LIBXS_FPRINT_PERAXIS, hierarchical mode is used (sweeps the innermost dimension first, collapses via Sobolev self-norms). The axis parameter is only consulted when LIBXS_FPRINT_PERAXIS is set.

The smooth parameter specifies a box-filter radius applied before differencing (0 = off). Each output sample becomes the average of the 2*smooth+1 nearest input samples (clamped at boundaries).

For 1-D data without LIBXS_FPRINT_PERAXIS, axis is ignored.

Supported types: F64, F32, and all integer libxs_data_t types (I64, I32, U32, I16, U16, I8, U8 — promoted to double internally).

With datatype LIBXS_DATATYPE_UNKNOWN, the element type is discovered (see Type Discovery below).

```
int libxs_fprint_partial(libxs_fprint_t* info,
  libxs_data_t datatype, const void* data, int n, int order);
```

Streaming (incremental) fingerprint accumulation. Processes data one chunk at a time and maintains exact junction derivatives across calls via the tail buffer. First call: pass a zero-initialized info. Subsequent calls: pass the same info with the next contiguous chunk. Only 1-D contiguous data is supported. The normalized fields (l2, l1, linf, mean) are updated after each call so that libxs_fprint_diff can be used at any intermediate point.

For a single chunk of n elements, produces the same result as libxs_fprint with ndims=1, shape={n}, stride=NULL.

```
int libxs_fprint_join(libxs_fprint_t* output,
  const libxs_fprint_t* a, const libxs_fprint_t* b);
```

Approximate merge of two finalized fingerprints. Combines a and b into output using the raw accumulators (acc_sq, acc_abs, acc_sum). Junction derivatives between the two regions are not recomputed, so the result is approximate with error proportional to order / min(a->n, b->n). The linf field is the maximum of the un-scaled maxima from both sides, re-scaled to the combined length. Returns EXIT_FAILURE if the combined n is below 2.

```
double libxs_fprint_diff(
  const libxs_fprint_t* a, const libxs_fprint_t* b,
  const double* weights);
```

Weighted Sobolev distance between two fingerprints:

```
d = sqrt( sum_k weights[k] * [(a->l2[k] - b->l2[k])^2 + (a->mean[k] - b->mean[k])^2] )
```

The number of orders compared is min(a->order, b->order) + 1. If weights is NULL, default weights w(k) = 1/k! are used. Where both fingerprints hold Bernstein norms, w(k) * (a->bernstein[k] - b->bernstein[k])^2 is added for those orders; streamed and joined fingerprints hold none.

```
#define LIBXS_FPRINT_BANDED 1024

int libxs_fprint_bernstein(double norms[], libxs_data_t datatype,
  const void* data, int n, int order);
```

L2 norms on [0,1] of the derivatives of the Bernstein approximant of the n samples: norms[k] = $||B^{(k)}||$ for k = 0..order. The order is clamped to LIBXS_FPRINT_MAXORDER, and orders beyond the degree n - 1 give zero. For any data, norms[k] <= l2[k] of `libxs_fprint` on the same samples, so the ratio norms[k] / l2[k] rates roughness independently of the amplitude: near 1 for smooth data, small for edges and noise.

The call costs far more than `libxs_fprint` for large n. Above LIBXS_FPRINT_BANDED samples, weights below 10^{-17} of their largest are skipped. The Fortran generic `libxs_fprint_bernstein(norms, data[, order])` takes REAL(C_DOUBLE) or REAL(C_FLOAT) data and fills norms(0:order).

```
int libxs_fprint_bernstein_diff(double* dist, libxs_data_t datatype,
  const void* a, int na, const void* b, int nb, int order,
  const double weights[]);
```

Exact Sobolev distance between the Bernstein approximants of two sample sets of any lengths: dist = sqrt(Sigma$_k$ w(k) $||$(B_a - B_b)$^{(k)}||^2$) for k = 0..order, with w(k) = 1/k! if weights is NULL. Unlike `libxs_fprint_diff`, which compares summaries, it is a metric on the data themselves (zero only for equal samples), and it needs both sample sets. The Fortran generic `libxs_fprint_bernstein_diff(dist, a, b[, order][, weights])` takes REAL(C_DOUBLE) or REAL(C_FLOAT) samples.

From Fortran (`USE libxs`), the generic `libxs_fprint(info, data[, order, axis, smooth, flags])` takes a one-dimensional REAL(C_DOUBLE), REAL(C_FLOAT), INTEGER(C_INT), or INTEGER(C_INT8_T) array, and `libxs_fprint(info, datatype, data, ndims, shape[, stride, order, axis, smooth, flags])` a `TYPE(C_PTR)` with an explicit shape; order defaults to LIBXS_FPRINT_MAXORDER and the other options to zero. `libxs_fprint_partial(info, data[, order])` takes the same four array types, `libxs_fprint_diff(a, b[, weights])` returns the distance, and `libxs_fprint_join(output, a, b)` is bound directly.

Type Discovery for Opaque Data

To identify the element type of an untyped buffer, call `libxs_fprint` with `LIBXS_DATATYPE_UNKNOWN`, ndims=1, and shape[0] set to the byte count. Every type whose width divides the byte count is fingerprinted, and the one with the smallest decay ratio is written to info->datatype; ties go to the smaller width, and float interpretations whose largest magnitude is below 1e-37 are skipped. EXIT_FAILURE is returned if no candidate yields a finite ratio. The choice is a best guess: if `libxs_fprint_decay` of the result is above 1, the buffer showed no structure under any type.

Golden Section Search

```
typedef enum libxs_gss_flags_t {
  LIBXS_GSS_DEFAULT        = 0,
  LIBXS_GSS_EVAL_ENDPOINTS = 1
} libxs_gss_flags_t;

typedef enum libxs_gss_status_t {
  LIBXS_GSS_STATUS_DEFAULT      = 0,
  LIBXS_GSS_STATUS_ENDPOINT_MIN = 1,
  LIBXS_GSS_STATUS_FLAT_MIN     = 2,
  LIBXS_GSS_STATUS_LEFT_REFINED = 4,
  LIBXS_GSS_STATUS_MAXITER      = 8,
  LIBXS_GSS_STATUS_NO_BRACKET   = 16
} libxs_gss_status_t;

typedef struct libxs_gss_info_t {
  double unimodality;
  double xmin, fmin;
  double x0, x1;
  int evaluations;
  int iterations;
  int status;
} libxs_gss_info_t;
```

Status	Meaning
LIBXS_GSS_STATUS_ENDPOINT_MIN	The minimum lies at x0 or x1
LIBXS_GSS_STATUS_FLAT_MIN	Another sample is within ftol of the minimum
LIBXS_GSS_STATUS_LEFT_REFINED	libxs_bisect_min refined the left edge
LIBXS_GSS_STATUS_MAXITER	maxiter iterations were exhausted
LIBXS_GSS_STATUS_NO_BRACKET	libxs_bisect_min: neither endpoint reaches fmin

Both searches require x0 <= x1 and maxiter > 0. Function values that differ by at most ftol count as equal; with ftol <= 0, only identical values do.

```
double libxs_gss_min(
  double (*fn)(double x, const void* data),
  const void* data,
  double x0, double x1, double* xmin, int maxiter,
  int flags, double ftol,
  libxs_gss_info_t* info);
```

Minimizes a unimodal function fn on the interval [x0, x1]. The callback receives x and an opaque context pointer. Returns f(x*) where x* is the minimizer; xmin (may be NULL) receives x*. The search stops when the bracket collapses to machine precision or after maxiter iterations. LIBXS_GSS_EVAL_ENDPOINTS also evaluates fn at x0 and x1. For step-like objectives such as tolerance thresholds, use libxs_bisect_min instead.

The optional info pointer (may be NULL) receives status flags, iteration and evaluation counts, the final x/f pair, the final bracket, and a unimodality score in [0, 1] (1.0: samples consistent with a single valley; lower: irregular or multimodal).

```
double libxs_bisect_min(
  double (*fn)(double x, const void* data),
  const void* data,
  double x0, double x1, double fmin, double* xmin, int maxiter,
  double ftol, libxs_gss_info_t* info);
```

Bisects the interval [x0, x1] to find the left edge of a known minimum level fmin, for monotone threshold or plateau-edge searches. The right endpoint is expected to reach fmin; otherwise info->status includes LIBXS_GSS_STATUS_NO_BRACKET. Returns fn at the found point, and xmin (may be NULL) receives that point.

Generalized Binomial and Distance

```
double libxs_binom(double t, int k);
```

Generalized binomial coefficient C(t, k) for real-valued t: C(t, k) = t * (t-1) * ... * (t-k+1) / k!. Evaluates the Newton basis polynomial at position t. For integer t >= k >= 0, this equals the standard binomial coefficient.

```
double libxs_dist2(const double* a, const double* b, int n);
```

Squared Euclidean distance between two n-dimensional vectors. Returns the sum of squared component differences.

```
double libxs_fprint_raw(const libxs_fprint_t* info,
  int k, double value);
```

Recover unnormalized finite-difference magnitude by dividing by (n-1)^k, undoing the unit-interval scaling. For k == 0, the value is returned unchanged.

Number Theory

```
size_t libxs_gcd(size_t a, size_t b);
size_t libxs_lcm(size_t a, size_t b);
```

GCD (returns 1 for gcd(0,0)) and LCM.

```
int libxs_primes_u32(unsigned int num,
  unsigned int num_factors_n32[], int num_factors_max);
```

Prime factors of `num` in ascending order, repeated by multiplicity (12 yields [2, 2, 3]). Returns the factor count, which may exceed `num_factors_max` (only that many are written; zero queries the count without writing). Returns zero if `num` is a prime other than two.

```
size_t libxs_coprime(size_t n, size_t minco);
size_t libxs_coprime_bias(size_t n, double bias);
size_t libxs_coprime2(size_t n); /* inline: coprime_bias(n, 0) */
```

`libxs_coprime`: co-prime of `n` not exceeding `minco`.

`libxs_coprime_bias`: co-prime of `n` selected by bias in [-1, +1] via a logarithmic mapping (target = n^((1+bias)/2)). bias=-1 selects the smallest non-trivial coprime (maximum displacement), bias=0 selects near sqrt(n) (balanced, same as libxs_coprime2), bias=+1 selects near n/2 (near-alternation). Monotonic: larger bias always yields a larger (or equal) coprime.

`libxs_coprime2`: inline convenience for `libxs_coprime_bias(n, 0)`.

```
unsigned int libxs_remainder(unsigned int a, unsigned int b,
  const unsigned int* limit, const unsigned int* remainder);
```

Smallest multiple of `b` minimizing remainder mod `a`.

```
unsigned int libxs_product_limit(unsigned int product,
  unsigned int limit, int is_lower);
```

Sub-product of prime factors within `limit` (0/1-Knapsack).

Scalar Utilities

```
double libxs_kahan_sum(double value, double* accumulator,
  double* compensation);
```

Kahan compensated summation.

Compensated Arithmetic

```
double libxs_neumaier_sum(double value, double* accumulator, double* compensation);
double libxs_sum2(const double* x, int n);
double libxs_sum2_err(const double* x, int n, double* err);
double libxs_dot2(const double* x, const double* y, int n);
double libxs_dot2_err(const double* x, const double* y, int n, double* err);
```

Function	Result
`libxs_neumaier_sum`	Like `libxs_kahan_sum`, robust for summands of any magnitude order
`libxs_sum2`	Sum of x[n] as if computed in twice the working precision
`libxs_sum2_err`	As `libxs_sum2`; err (may be NULL) bounds the error: the exact sum is in [res - err, res + err]
`libxs_dot2`	Dot product of x[n] and y[n] as if computed in twice the working precision
`libxs_dot2_err`	As `libxs_dot2`, with an error bound as for `libxs_sum2_err`

Function	Result

```
int libxs_dot2_gemm(libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, const void* alpha, const void* a, int lda,
  const void* b, int ldb, const void* beta, void* c, int ldc);
```

Reference GEMM, C = alpha * op(A) * op(B) + beta * C (column-major, BLAS semantics, trans one of `'N'`, `'T'`, `'C'`), with every element computed by `libxs_dot2`. Use it to grade a GEMM computed in working precision. Supports F64, F32, C64, and C32. It is serial: to grade a large product, pass a few rows (m=1 per row). Returns EXIT_FAILURE in a build whose compensation was optimized away (see `LIBXS_PRAGMA_FASTMATH_OFF`).

```
double libxs_two_sum(double a, double b, double* err);      /* inline */
double libxs_fast_two_sum(double a, double b, double* err); /* inline */
double libxs_two_product(double a, double b, double* err);  /* inline */
```

Error-free transformations: each returns the rounded result x and writes the rounding error to err, so that a + b == x + *err (or a * b == x + *err). `libxs_fast_two_sum` is exact only if |a| >= |b|. `libxs_two_product` is exact unless the product underflows.

```
unsigned int libxs_isqrt_u64(unsigned long long x);
unsigned int libxs_isqrt_u32(unsigned int x);
unsigned int libxs_isqrt2_u32(unsigned int x);
```

Exact integer square root, floor(sqrt(x)), without floating point. The isqrt2 variant returns a factor of x.

```
double libxs_pow2(int n);
```

2^n via IEEE-754 exponent. Valid for n in [-1022, 1023]. Returns 0.0 for underflow (subnormals flushed to zero) and +Inf for overflow.

Modular Arithmetic

```
unsigned int libxs_mod_inverse_u32(unsigned int a, unsigned int m);
size_t libxs_mod_inverse(size_t a, size_t m);
```

Modular inverse a^{-1} mod m. Requires gcd(a, m) = 1 and $0 < a$, $1 < m$. The size_t variant supports counts up to 2^63 - 1.

```
unsigned int libxs_barrett_rcp(unsigned int p);
unsigned int libxs_barrett_pow18(unsigned int p);
unsigned int libxs_barrett_pow36(unsigned int p);
```

Barrett reduction constants for modulus `p`.

```
unsigned int libxs_mod_u32(uint32_t x, unsigned int p,
  unsigned int rcp);                          /* inline */
unsigned int libxs_mod_u64(uint64_t x, unsigned int p,
  unsigned int rcp, unsigned int pow18,
  unsigned int pow36);                        /* inline */
```

Fast modular reduction via Barrett. The 32-bit variant is valid for any `x`; the 64-bit variant uses a radix-2^{18} split and is valid for $x < 2^{54}$ and $p < 8192$.

BF16 and FP16 Conversion

```
typedef uint16_t libxs_bf16_t;
typedef uint16_t libxs_f16_t;
libxs_bf16_t libxs_round_bf16_f32(float x);     /* inline */
float libxs_bf16_to_f32(libxs_bf16_t v);        /* inline */
libxs_bf16_t libxs_round_bf16(double x);        /* inline */
double libxs_bf16_to_f64(libxs_bf16_t v);       /* inline */
libxs_bf16_t libxs_u8_to_bf16(unsigned int v); /* inline */
libxs_f16_t libxs_round_f16_f32(float x);       /* inline */
float libxs_f16_to_f32(libxs_f16_t v);          /* inline */
libxs_f16_t libxs_round_f16(double x);          /* inline */
double libxs_f16_to_f64(libxs_f16_t v);         /* inline */
```

`libxs_bf16_t` and `libxs_f16_t` hold the raw encodings of BF16 and IEEE FP16. The `round` functions round to nearest even, and the `to` functions expand exactly to float or double. `libxs_u8_to_bf16` converts an 8-bit unsigned integer exactly. Native `__bf16` (`LIBXS_BF16`) and `_Float16` (`LIBXS_F16`) are used when the compiler provides them, otherwise portable bit manipulation.

```
void libxs_bf16_to_f32_block(float* dst,
  const libxs_bf16_t* src, size_t n);           /* inline */
void libxs_round_bf16_f32_block(libxs_bf16_t* dst,
  const float* src, size_t n);                  /* inline */
void libxs_f16_to_f32_block(float* dst,
  const libxs_f16_t* src, size_t n);            /* inline */
void libxs_round_f16_f32_block(libxs_f16_t* dst,
  const float* src, size_t n);                  /* inline */
void libxs_u8_to_bf16_block(libxs_bf16_t* dst,
  const unsigned char* src, size_t n);          /* inline */
```

Convert n contiguous elements like the scalar functions above, e.g., into scratch ahead of a compute loop.

```
void libxs_dekker_bf16(double val, int ndigits,
  libxs_bf16_t* dst);                       /* inline */
void libxs_dekker_bf16_f32(float val, int ndigits,
  libxs_bf16_t* dst);                       /* inline */
```

Dekker split: decompose a value into `ndigits` BF16 residuals via iterative subtraction. `dst[0]` holds the dominant digit (the BF16 rounding of the input); `dst[1]` holds the residual after subtracting the first digit's contribution, rounded to BF16; and so on. The sum `libxs_bf16_to_f64(dst[0]) + ... + libxs_bf16_to_f64(dst[ndigits-1])` reconstructs the original to about `8 * ndigits` significant bits, so seven digits cover a double. BF16 has the exponent range of float: a value beyond it overflows, and digits below it are lost. The `_f32` variant keeps the residual arithmetic in single precision for float storage/conversion paths.

Memory Operations

Header: `libxs_mem.h`

Byte-level memory macros, pointer alignment queries, buffer comparison, and matrix copy/transposition.

Memory Macros

The `LIBXS_MEM*` macros expand to an inline byte loop rather than a library call, which suits small sizes; prefer `memset`/`memcpy` for large buffers.

```
LIBXS_MEMSET(DST, SRC, SIZE)
```

Set SIZE bytes at DST to the value SRC.

```
LIBXS_MEMZERO(DST)
```

Zero all bytes of *DST (size derived from sizeof(*DST)).

```
LIBXS_MEMCPY(DST, SRC, SIZE)
```

Copy SIZE bytes from SRC to DST.

```
LIBXS_ASSIGN(DST, SRC)
```

Copy sizeof(*SRC) bytes from SRC to DST. Statically asserts sizeof(*SRC) <= sizeof(*DST).

```
LIBXS_MEMSWP(PTR_A, PTR_B, SIZE)
```

Swap SIZE bytes between PTR_A and PTR_B. Asserts PTR_A != PTR_B.

```
LIBXS_VALUE_ASSIGN(DST, SRC)
LIBXS_VALUE_SWAP(A, B)
```

Operate on L-values directly (address-of is taken internally). VALUE_ASSIGN can cast away const qualifiers. VALUE_SWAP statically asserts equal size.

Memory Information

```
int libxs_mem_info(size_t* mem_free, size_t* mem_total);
```

Host memory in bytes: `mem_total` is what the process may use (a cgroup limit applies where present), `mem_free` what is unused. Either pointer may be NULL. Device memory is not considered. Returns `EXIT_SUCCESS` if a size was determined; otherwise the outputs are left untouched.

Offset and Alignment

```
size_t libxs_offset(size_t ndims, const size_t offset[],
  const size_t shape[], size_t* size);
```

Compute the linear offset of an n-dimensional coordinate within a shape. Optionally returns the total linear size of the shape.

```
int libxs_aligned(const void* ptr, const size_t* inc,
  int* alignment);
```

Check whether ptr is SIMD-aligned (optionally considering the next access at ptr + *inc). Optionally writes the pointer alignment in bytes to *alignment.

Comparison

```
unsigned char libxs_diff(const void* a, const void* b,
  unsigned char size);
```

Compare two short buffers. Returns zero when equal, non-zero otherwise. Uses SSE/AVX2/AVX-512 when available.

```
unsigned int libxs_diff_n(const void* a, const void* bn,
  unsigned char elemsize, unsigned char stride,
  unsigned int hint, unsigned int count);
```

Compare a against count elements in bn. Returns the first index that matches a, or count if none match. hint supplies the initial search position.

```
int libxs_memcmp(const void* a, const void* b, size_t size);
```

Boolean variant of the C memcmp. Returns zero when equal. Uses SSE/AVX2/AVX-512 when available.

Matrix Copy and Transposition

```
void libxs_matcopy(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_matcopy_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Copy an m-by-n matrix from in to out with leading dimensions ldi and ldo. If in is NULL the destination is zeroed. The _task variant distributes work across ntasks threads (caller passes its tid).

```
void libxs_otrans(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_otrans_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Out-of-place matrix transposition. The _task variant distributes work across ntasks threads.

```
void libxs_itrans(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo);
void libxs_itrans_task(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

In-place matrix transposition. A shape other than square with `ldi == ldo` needs a buffer of `m*n` elements, taken from the LIBXS memory pool; `libxs_itrans_batch` amortizes it over the batch.

```
void libxs_itrans_batch(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo,
  int index_base, int index_stride,
  const int stride[], int batchsize,
  int tid, int ntasks);
```

Batch of in-place matrix transpositions (per-thread form).

```
typedef enum libxs_mem_op_t {
  LIBXS_MEM_OP_MATCOPY = 0, /* libxs_matcopy_task with a source */
  LIBXS_MEM_OP_MATZERO = 1, /* libxs_matcopy_task without source (zeroing) */
  LIBXS_MEM_OP_OTRANS = 2,  /* libxs_otrans_task */
  LIBXS_MEM_OP_ITRANS = 3   /* libxs_itrans_task (square with ldi == ldo) */
} libxs_mem_op_t;

int libxs_mem_ntasks(libxs_mem_op_t op, int m, int n,
  unsigned int typesize, int nthreads);
```

Number of tasks worth splitting a `_task` call for `op` into, given `nthreads` available: 1 below a minimum size of the operation, otherwise one task per so many bytes, never more than `nthreads`. An `itrans` of another shape runs on task 0 only. In Fortran, compiled with OpenMP, `libxs_matcopy`, `libxs_otrans`, and `libxs_itrans` use it to run over an OpenMP team unless called from within a parallel region. These Fortran routines are not `PURE`; their `_task` forms are.

MetaImage I/O

Header: `libxs_mhd.h`

Read and write image data in the MetaImage (MHD/MHA) format. Files are compatible with ITK, ITK-SNAP, 3D Slicer, ParaView, and similar tools.

Types

```
typedef enum libxs_mhd_element_conversion_hint_t {
  LIBXS_MHD_ELEMENT_CONVERSION_DEFAULT,
  LIBXS_MHD_ELEMENT_CONVERSION_MODULUS
} libxs_mhd_element_conversion_hint_t;
```

Hint controlling element conversion behaviour (default clamping vs. modulus mapping).

```
typedef struct libxs_mhd_element_handler_info_t {
  libxs_data_t type;
  libxs_mhd_element_conversion_hint_t hint;
} libxs_mhd_element_handler_info_t;
```

Destination type and conversion hint, passed to built-in or custom element handlers.

```
typedef int (*libxs_mhd_element_handler_t)(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max,
  size_t index, void* context);
```

Per-element callback for reading or converting image data. The value range (`src_min`, `src_max`) is available for scaling. `index` is the logical element index in write/read traversal order, and `context` is user data supplied by the caller.

```
typedef struct libxs_mhd_info_t {
  size_t ndims;         /* dimensionality */
  size_t ncomponents;   /* interleaved image channels */
  libxs_data_t type;    /* pixel element type */
  size_t header_size;   /* header size in bytes (LOCAL data) */
} libxs_mhd_info_t;
```

Image descriptor populated by `libxs_mhd_read_header` and consumed by read/write.

```
typedef struct libxs_mhd_write_info_t {
  const libxs_mhd_element_handler_info_t* handler_info;
  libxs_mhd_element_handler_t handler;
  void* handler_context;
  const char* extension_header;
  const void* extension;
  size_t extension_size;
  double range[2];
} libxs_mhd_write_info_t;
```

Optional write parameters (conversion, custom handler, extension data, value range). NULL is accepted for all-defaults, and so is a zero-initialized struct. A `range` with `range[0] < range[1]` is mapped onto the destination type by a scaling conversion, in place of the minimum and maximum found in the data, so images written apart share one scale.

Element Helpers

```
int libxs_mhd_element_conversion(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max,
  size_t index, void* context);
```

Built-in element conversion. Scales values when `src_min` and `src_max` are non-NULL; otherwise clamps to the destination type.

```
int libxs_mhd_element_comparison(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max,
  size_t index, void* context);
```

Check an in-memory buffer against file content (element-wise comparison with optional type conversion).

Type Queries

```
const char* libxs_mhd_typename(libxs_data_t type,
  const char** ctypename);
```

Return the MHD element-type name (e.g. `"MET_FLOAT"`) for the given `libxs_data_t`. Optionally writes the C type name. Returns NULL for unknown types.

```
libxs_data_t libxs_mhd_typeinfo(const char elemname[]);
```

Return the `libxs_data_t` for a given MHD element-type string.

Read

```
int libxs_mhd_read_header(const char header_filename[],
  size_t filename_max_length, char filename[],
  libxs_mhd_info_t* info, size_t size[],
  char extension[], size_t* extension_size);
```

Parse an MHD header file. `info->ndims` is an in/out parameter (maximum on input, actual on output). `filename` receives the data-file path (may differ from the header when data is stored separately). Extension data is optional: on input, `*extension_size` is the capacity of `extension`; on output, it is the size of the extension record in the file (at most the capacity is read). Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

```
int libxs_mhd_read(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], const libxs_mhd_info_t* info,
  void* data,
  const libxs_mhd_element_handler_info_t* handler_info,
  libxs_mhd_element_handler_t handler,
  void* handler_context);
```

Read image data into `data`. Optional `handler_info` triggers type conversion; optional `handler` supplies a custom per-element callback (can also serve as a comparison function without allocating a buffer). Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

Write

```
int libxs_mhd_write(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], libxs_mhd_info_t* info,
  const void* data,
  const libxs_mhd_write_info_t* write_info);
```

Write image data in the extended MHD format. `info->header_size` is updated on output. Pass NULL for `write_info` when no conversion or extension is needed. Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

From Fortran (`USE libxs`), the generic `libxs_mhd_write` takes a contiguous array of `REAL(8)`, `REAL(4)`, `INTEGER(4)`, or `INTEGER(8)`, the image extents (first dimension fastest), and optionally the extension as an array of `INTEGER(C_INT8_T)`. It returns zero on success.

```
status = libxs_mhd_write("blocks.mhd", values, [n], extension)
```

When the environment variable `LIBXS_MHD_PNG` is set and the image is two-dimensional, an uncompressed PNG file is written alongside the MHD output (same base name, `.png` extension). Supported channel counts are 1 (grayscale), 3 (RGB), and 4 (RGBA) at 8-bit depth. Source data of any element type is converted to unsigned 8-bit using the built-in element conversion with automatic min/max scaling, or over `write_info->range` if one is fixed.

```
int libxs_png_write(const char filename[],
  const size_t size[], const size_t pitch[],
  size_t ncomponents, libxs_data_t type,
  const void* data,
  const libxs_mhd_write_info_t* write_info);
```

Write the same PNG explicitly and without an MHD file: `size` holds width and height, `pitch` the leading dimension (NULL: same as `size`). With a handler in `write_info`, what the handler returns is scaled; the extension is not written. Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

From Fortran (`USE libxs`), the generic `libxs_png_write` takes a contiguous array of `REAL(8)`, `REAL(4)`, `INTEGER(4)`, or `INTEGER(8)`, the two extents (width first), and optionally the range as two `REAL(8)`. It returns zero on success.

```
status = libxs_png_write("confidence.png", values, [nx, ny], [0.5d0, 1d0])
```

Mix

Header: `libxs_mix.h`

Fixed-share mixture over a bank of experts: a linear pool whose per-expert weights are carried across observations and updated from realized log loss, so the mixture tracks whichever expert is currently right. Experts may abstain per observation.

Not to be confused with `libxs_predict_set_series_bank`, which averages several views of one window with fixed equal weight; this learns the weights.

Types

```
typedef struct libxs_mix_t {
  double* weight;  /* per-slot weight [nslot] */
  int nslot;       /* number of experts */
  double rate;     /* exponent on the likelihood ratio */
  double share;    /* uniform share redistributed per step */
  double relmin;   /* floor for a non-positive ratio (0 disables) */
} libxs_mix_t;
```

The struct is plain data. A caller that already owns a weight array may fill the fields directly and point `weight` at it, which suits a bank living on the stack or inside a larger record; such a view must not be passed to `libxs_mix_destroy`.

Functions

```
int libxs_mix_create(libxs_mix_t* mix, int nslot,
  double rate, double share, double relmin);
void libxs_mix_destroy(libxs_mix_t* mix);
void libxs_mix_reset(libxs_mix_t* mix, const int active[]);
```

`create` starts from a uniform prior and returns `EXIT_SUCCESS` or `EXIT_FAILURE`. `reset` restores a uniform prior over the slots marked active (NULL selects all). `destroy` releases the weights; the struct itself is caller-owned.

```
double libxs_mix_pool(const libxs_mix_t* mix,
  const double prob[], const int active[]);
void libxs_mix_update(libxs_mix_t* mix,
  const double prob[], const int active[], double mixture);
double libxs_mix_observe(libxs_mix_t* mix,
  const double prob[], const int active[]);
```

`prob[k]` is expert `k`'s probability for the outcome under consideration. `active` may be NULL (every expert speaks). `pool` reports the mixture without touching the weights. `update` advances the weights against a mixture the caller supplies. `observe` is `pool` followed by `update` with that pool, and returns the pooled value taken *before* the update.

Prefer `observe`: a pooled value taken after `update` silently understates the code length. Use `update` only when the caller's mixture differs from `pool`'s, for instance because it mixes in experts this bank does not carry or was floored before taking a logarithm.

Abstention and Disabled Slots

Abstention (`active[k] == 0`) is not a probability of zero. The pool renormalizes over the experts that spoke, so a silent expert does not dilute the mixture. An update renormalizes the experts that spoke to sum to one, while a silent expert keeps its absolute weight: its weight relative to the others changes while it is quiet, and the weights as a whole need not sum to one.

A weight of zero disables the slot permanently (share mass does not revive it), which suits the unused slots of a bank sized for the widest configuration.

With `relmin` of zero, an expert that once gave the outcome no probability drops to zero weight for good; a positive `relmin` lets it recover.

Example

```
libxs_mix_t mix;
double prob[3];
if (EXIT_SUCCESS == libxs_mix_create(&mix, 3, 0.15, 0.005, 1e-4)) {
  double bits = 0;
  while (next_observation(prob)) {
    const double p = libxs_mix_observe(&mix, prob, NULL);
    bits -= log(0 < p ? p : 1e-12) / log(2.0);
  }
  libxs_mix_destroy(&mix);
}
```

N-gram

Header: `libxs_ngram.h`

Count-based variable-order n-gram model over unsigned token ids, backed by a registry keyed on the context. Stores every context length up to a configured maximum from one pass of observations, then answers interpolated-backoff probabilities and top-k predictions.

Token id 0 is reserved and treated as absent context: it is never stored and never predicted.

Constants

```
#define LIBXS_NGRAM_ORDER_MAX 6  /* longest context that can be stored */
#define LIBXS_NGRAM_SUCC_MAX  8  /* successors retained per context */
```

Types

```
typedef struct libxs_ngram_succ_t {
  unsigned int id, count;
} libxs_ngram_succ_t;

typedef struct libxs_ngram_entry_t {
  unsigned int total;   /* observations of this context */
  unsigned int nsucc;   /* successors retained */
  libxs_ngram_succ_t succ[LIBXS_NGRAM_SUCC_MAX];
} libxs_ngram_entry_t;
```

The successor list is bounded by a Space-Saving policy that evicts the least frequent entry, so `total` may exceed the sum of the retained counts; `prob` assigns the difference to backoff.

```
typedef struct libxs_ngram_t {
  libxs_registry_t* store;
  unsigned int* unifreq;
  double unifreq_total;
  unsigned int unifreq_size;
  unsigned int backoff_ids[LIBXS_NGRAM_SUCC_MAX];
  int backoff_count, maxorder;
} libxs_ngram_t;
```

Callers own the struct, so stack allocation is fine; the store and backoff tables are heap-managed by create and destroy.

```
typedef struct libxs_ngram_stats_t {
  long keys[LIBXS_NGRAM_ORDER_MAX + 1];       /* per order, index 0 unused */
  double obs[LIBXS_NGRAM_ORDER_MAX + 1];
  long saturated[LIBXS_NGRAM_ORDER_MAX + 1];
  size_t entries, nbytes;
} libxs_ngram_stats_t;
```

Functions

```
int libxs_ngram_create(libxs_ngram_t* model, int maxorder);
void libxs_ngram_destroy(libxs_ngram_t* model);
```

`maxorder` is clamped to `1..LIBXS_NGRAM_ORDER_MAX`. `create` returns `EXIT_SUCCESS`, or `EXIT_FAILURE` if the backing store cannot be created.

```
void libxs_ngram_observe(libxs_ngram_t* model,
  const unsigned int hist[], int hlen, unsigned int succ);
void libxs_ngram_finalize(libxs_ngram_t* model, unsigned int vocab);
```

`observe` increments every context length `1..min(maxorder,hlen)` ending at `hist[hlen-1]`, so one call per position populates all orders. `finalize` builds the global unigram distribution and the most-frequent-token backoff list from the order-1 counts; call it once after all observations and before any query, with `vocab` the largest token id in use.

```
const libxs_ngram_entry_t* libxs_ngram_lookup(const libxs_ngram_t* model,
  const unsigned int hist[], int hlen, int n);
double libxs_ngram_prob(const libxs_ngram_t* model,
  const unsigned int hist[], int hlen, unsigned int next);
int libxs_ngram_predict(const libxs_ngram_t* model,
  const unsigned int hist[], int hlen, unsigned int out_ids[], int k,
  int* order);
int libxs_ngram_stats(const libxs_ngram_t* model,
  libxs_ngram_stats_t* stats);
```

`lookup` returns the record of the length-`n` context ending at `hist[hlen-1]`, or NULL if there is none.

`prob` is interpolated backoff: it blends orders `1..min(maxorder,hlen)` with weight `total/(total+1)` per order over an additive-smoothed unigram prior. After `finalize`, probabilities over ids `1..vocab` sum to one for every history — which makes the model usable as a code-length model and not only as a ranker.

`predict` uses hard backoff instead: the top-k successors of the longest context that has a record, falling back to the global most-frequent list when none matches. A non-NULL `order` receives the context length that produced the result (0 for the global fallback), so a caller can tell a deeply grounded prediction from a guess. Returns the number of ids written to `out_ids`.

`stats` reports per-order key and observation counts plus the live footprint. `nbytes` follows the hash table's power-of-two capacity and grows in steps; `entries` tracks the model size.

Example

```
libxs_ngram_t model;
unsigned int hist[LIBXS_NGRAM_ORDER_MAX];
int hlen = 0;
if (EXIT_SUCCESS == libxs_ngram_create(&model, 3)) {
  unsigned int id, vocab = 0;
  while (next_token(&id)) {
    libxs_ngram_observe(&model, hist, hlen, id);
    if (vocab < id) vocab = id;
    if (LIBXS_NGRAM_ORDER_MAX > hlen) hist[hlen++] = id;
    else {
      memmove(hist, hist + 1, (LIBXS_NGRAM_ORDER_MAX - 1) * sizeof(*hist));
      hist[LIBXS_NGRAM_ORDER_MAX - 1] = id;
    }
  }
  libxs_ngram_finalize(&model, vocab);
  /* ... libxs_ngram_prob / libxs_ngram_predict ... */
  libxs_ngram_destroy(&model);
}
```

Pack

Header: `libxs_pack.h`

Bit packing of integer arrays, at a fixed width or at a width chosen per group of values. Packed data is a stream of 64-bit words, so the packed buffer must be 8-byte aligned.

Fixed Width

```
int libxs_pack(void* packed, const void* values,
  libxs_data_t type, size_t n, int nbits);
int libxs_unpack(void* values, const void* packed,
  libxs_data_t type, size_t n, int nbits);
```

Packs `n` integers of `type` (`LIBXS_DATATYPE_I8` to `I64`, `U8` to `U64`) at `nbits` each (1 to eight times the type size) into ceil(n*nbits/64) words. Bits beyond `nbits` are ignored. Unpacking sign-extends signed types from `nbits` and zero-extends unsigned types. Both return `EXIT_SUCCESS`, or `EXIT_FAILURE` for a type or width out of range, or for a NULL pointer with $n > 0$.

Variable Width

```
size_t libxs_vpack(void* packed, const void* values,
  libxs_data_t type, size_t n, int maxbits, int level);
size_t libxs_vpack_bound(size_t n, int maxbits);
int libxs_vpack_maxbits(libxs_data_t type, double value, int relative);
size_t libxs_vunpack(void* values, const void* packed, size_t npacked,
  libxs_data_t type, size_t n);
```

Packs `n` values of `type` (integers as above, or `LIBXS_DATATYPE_F64` and `F32`) into a stream of words whose size follows the magnitudes, so small values or zeros cost little. Integers are taken as integers of `maxbits` bits (1 to eight times the type size); small magnitudes of either sign stay small. For floating-point types, `maxbits` sets the accuracy: the width of the type (64 or 32, or more) is lossless; fewer bits keep that many leading bits of every value (at least sign, exponent, and one bit), which bounds the error relative to the value; zero or less bounds the absolute error by 2^`maxbits`, and values that cannot be held to it (Inf, NaN, or too large) are kept exactly, as are all values for an error below the smallest normal number.

The `level` selects a trade-off: 1 packs and unpacks fastest, 2 takes less space, and 3 (or more) the least; each step takes more time to pack and to unpack. Level 0 (or less) takes the default, which is 2 unless `LIBXS_PACK` in the environment says otherwise (read at the first call).

Unpacking needs only `n` and the number of words available (`npacked`, checked against the stream in debug builds only), whatever the level. The stream does not record the type: unpack with a type of the same signedness as used for packing. The layout of the stream is internal and may change between versions: packed data is meant to be unpacked by the same library, not to be stored.

`libxs_vpack` returns the number of words written, at most `libxs_vpack_bound(n, maxbits)` for any level, and `libxs_vunpack` the number of words consumed. Both return zero for invalid arguments (including NULL pointers); `libxs_vunpack` also returns zero for an empty stream (`npacked` of zero) and leaves the values untouched.

`libxs_vpack_maxbits` gives the `maxbits` for a value: for integer types, the bits that magnitudes up to `value` need (a sign bit more for signed types); for floating-point types, the `maxbits` that keep the error within `value`, absolute (`relative` of zero) or relative to each value (not for subnormal values), and lossless for a `value` of zero or less. It returns zero for other types.

From Fortran (`USE libxs`), `libxs_vpack(packed, values, maxbits[, level])` and `libxs_vunpack(values, packed)` take contiguous `INTEGER(C_LONG_LONG)`, `REAL(C_DOUBLE)`, or `REAL(C_FLOAT)` values (and `INTEGER(C_LONG_LONG)` words) and return the number of words as `INTEGER(C_SIZE_T)`; `libxs_vpack_bound(n, maxbits)` and `libxs_vpack_maxbits(datatype, value, relative)` are bound directly.

The pack sample measures the trade-offs on synthetic data or on MHD files.

Permutation and Sorting

Header: `libxs_perm.h`

Permutation-based data reordering: deterministic co-prime shuffling (in-place and out-of-place), sorting, space-filling curves, k-d trees, and smoothness-optimized row permutations for matrices.

Shuffling

The shuffle functions use a co-prime stride C to produce a fixed, deterministic permutation of count elements. The mapping is affine:

```
dst[k] = src[(N-1) - ((C*k + offset) mod N)]
```

The stride must be co-prime to count; passing NULL selects libxs_coprime2(count). A nonzero offset selects another member of the affine family. Pass offset=0 for the standard (non-affine) shuffle.

```
int libxs_shuffle(void* inout, size_t elemsize, size_t count,
  const size_t* shuffle, size_t offset,
  const size_t* nrepeat);
```

In-place shuffle of count elements of elemsize bytes each. Needs N/8 bytes of scratch memory. nrepeat controls the number of successive permutation applications (NULL or pointing to 1 means one pass). Returns EXIT_SUCCESS, or EXIT_FAILURE for invalid arguments or if the auxiliary memory cannot be allocated (inout is then unchanged).

```
int libxs_shuffle2(void* dst, const void* src, size_t elemsize,
  size_t count, const size_t* shuffle, size_t offset,
  const size_t* nrepeat);
```

Out-of-place shuffle from src to dst. dst and src must not overlap. If *nrepeat is zero an ordinary copy is performed.

```
size_t libxs_unshuffle(size_t count, const size_t* shuffle);
```

Return the number of libxs_shuffle2 applications needed to restore the original element order for the given count and co-prime stride, assuming offset=0. A nonzero offset can change the cycle length unless C+1 is co-prime to N (never for even N); libxs_unshuffle2 inverts any offset in one pass.

```
int libxs_unshuffle2(void* dst, const void* src, size_t elemsize,
  size_t count, const size_t* shuffle, size_t offset,
  const size_t* nrepeat);
```

Inverse of libxs_shuffle2 for the same shuffle, offset, and nrepeat, in a single pass per repetition (no need to repeat libxs_shuffle2).

```
#define LIBXS_SHUFFLE_INDEX(I, N, C, B)
#define LIBXS_UNSHUFFLE_INDEX(J, N, CINV, B)
```

Index maps for shuffling without moving data. `LIBXS_SHUFFLE_INDEX(I, N, C, B)` is the index libxs_shuffle2 reads for destination I (stride C, offset B). `LIBXS_UNSHUFFLE_INDEX(J, N, CINV, B)` is its inverse, the index libxs_unshuffle2 reads for destination J, with CINV = `libxs_mod_inverse(C, N)`.

General-Purpose Sort

```
typedef int (*libxs_sort_cmp_t)(
  const void* a, const void* b, void* ctx);

void libxs_sort(void* base, int n, size_t size,
  libxs_sort_cmp_t cmp, void* ctx);
```

Sort n elements of the given byte size. The comparator returns negative, zero, or positive (tristate, like qsort_r). The ctx pointer is forwarded to the comparator unchanged.

Built-in comparators:

```
int libxs_cmp_f64(const void* a, const void* b, void* ctx);
int libxs_cmp_f32(const void* a, const void* b, void* ctx);
int libxs_cmp_i32(const void* a, const void* b, void* ctx);
int libxs_cmp_u32(const void* a, const void* b, void* ctx);
```

Prefer the built-in comparators: they enable a radix-sort fast path, whereas custom comparators fall back to a comparison sort.

Index comparators order an array of indices (base) by keys that ctx points to, and leave the keys untouched (e.g., to permute a payload):

```
int libxs_cmp_f64_idx(const void* a, const void* b, void* ctx);
int libxs_cmp_i32_idx(const void* a, const void* b, void* ctx);
```

With a built-in value comparator, ctx controls the mode:

- `ctx == NULL`: sort base in-place.
- `ctx != NULL`: read from ctx, write the sorted result to base (out-of-place, source unchanged).

With a custom comparator, ctx is user state passed to cmp and base is sorted in-place.

Examples:

```
libxs_sort(data, n, sizeof(double), libxs_cmp_f64, NULL);
libxs_sort(dst, n, sizeof(double), libxs_cmp_f64, src);
libxs_sort(perm, n, sizeof(int), libxs_cmp_i32_idx, keys);
libxs_sort(perm, n, sizeof(int), my_indirect_cmp, keys);
```

Space-Filling Curves

```
uint64_t libxs_hilbert(const unsigned int coords[], int ndims);
void libxs_hilbert_decode(uint64_t code, unsigned int coords[], int ndims);
```

N-dimensional Hilbert curve index. Maps ndims coordinates to a 64-bit key with strong locality guarantees. Each coordinate is quantized to floor(64/ndims) bits. coords[k] must be in [0, 2^bits_per_dim). The decode routine maps such a key back to ndims coordinates with the same bit budget.

```
uint64_t libxs_morton(const unsigned int coords[], int ndims);
void libxs_morton_decode(uint64_t code, unsigned int coords[], int ndims);
```

N-dimensional Morton code (Z-order curve). Bit-interleaves ndims coordinates into a 64-bit key. Each coordinate is quantized to floor(64/ndims) bits. The decode routine deinterleaves the key back to ndims coordinates.

```
uint64_t libxs_hilbert_bits(
  const unsigned int coords[], int ndims, int bits_per_dim);
void libxs_hilbert_decode_bits(
  uint64_t code, unsigned int coords[], int ndims, int bits_per_dim);
uint64_t libxs_morton_bits(
  const unsigned int coords[], int ndims, int bits_per_dim);
void libxs_morton_decode_bits(
  uint64_t code, unsigned int coords[], int ndims, int bits_per_dim);
```

Same curves with an explicit number of bits per coordinate (1 to 32, with ndims*bits_per_dim at most 64); the encoders return zero for arguments out of range.

```
int libxs_stratify_morton(
  const unsigned int src_coords[], int src_ndims,
  unsigned int dst_coords[], int dst_ndims);
int libxs_stratify_hilbert(
  const unsigned int src_coords[], int src_ndims,
  unsigned int dst_coords[], int dst_ndims);
```

Stratify higher-dimensional coordinates into a lower-dimensional layout without slicing. The source coordinates are encoded using the selected source-dimensional space-filling curve, and the resulting key is decoded as target-dimensional coordinates using the same curve family. This gives deterministic rank-preserving embeddings such as 3D to 2D:

```
code = curve(src_coords, 3)          /* 21 bits per coordinate */
dst_coords = inverse_curve(code, 2)  /* 32 bits per coordinate */
```

Each source coordinate keeps floor(64/src_ndims) bits, and each target coordinate receives ceil(src_ndims*floor(64/src_ndim bits, which must not exceed 32. The target dimensionality must be smaller than the source dimensionality. The functions return EXIT_SUCCESS or EXIT_FAILURE.

```
int libxs_stratify_morton_bits(
  const unsigned int src_coords[], int src_ndims, int src_bits,
  unsigned int dst_coords[], int dst_ndims, int dst_bits);
int libxs_stratify_hilbert_bits(
  const unsigned int src_coords[], int src_ndims, int src_bits,
  unsigned int dst_coords[], int dst_ndims, int dst_bits);
```

Same with explicit bits per source and target coordinate (each 1 to 32). Requires dst_ndims < src_ndims, src_ndims*src_bits <= 64, and dst_ndims*dst_bits >= src_ndims*src_bits; otherwise EXIT_FAILURE.

k-d Tree

```
typedef int (*libxs_kdtree_split_t)(
  int* dim, int* pos, double* threshold, const double* pts, int* idx,
  int count, int depth, int nleaves, void* ctx);

typedef struct libxs_kdtree_node_t {
  double threshold;
  int dim, next;
} libxs_kdtree_node_t;

typedef struct libxs_kdtree_config_t {
```

```
  libxs_kdtree_split_t split;
  void* ctx;
  libxs_kdtree_node_t* nodes;
  int min_leaf;
} libxs_kdtree_config_t;
```

Split callback for custom tree construction. Writes chosen split dimension into `*dim` and left-child count into `*pos`, and reorders idx[0..count-1] to place left entries first. `*threshold` arrives as `HUGE_VAL`: left so, the tree places it midway between the two sides along `*dim`; a callback that splits by coordinates other than `pts` writes its own. nleaves gives the number of leaves already assigned (in preorder). Return 0 for a valid split, nonzero to force a leaf.

Config struct: zero-initialize it (`libxs_kdtree_config_t config = { 0 }`) so that unset fields stay off. min_leaf controls minimum node size (0 = split down to 1 element). split=NULL uses round-robin median splits. nodes (partition only) receives the tree in preorder: a split (`dim >= 0`) has its left child next in the array and its right child at index `next`, entries below `threshold` went left; a leaf (`dim < 0`) has its leaf id in `next`. The tree has `2*nleaves-1` nodes, at most `2n-1`.

```
int libxs_kdtree_split_median(int* dim, int* pos, const double* pts,
  int* idx, int count, int depth, int nleaves, void* ctx);
int libxs_kdtree_split_middle(int* dim, int* pos, const double* pts,
  int* idx, int count, int depth, int nleaves, void* ctx);
```

Split rules of the tree itself, passed as `split`: the tree recognizes them and splits internally (called directly, they only return nonzero). `libxs_kdtree_split_median` is the default (split=NULL), the tree that the nearest-neighbor queries search. `libxs_kdtree_split_middle` splits at the middle of the range of the axis with the largest extent, which suits grid coordinates: points below the middle go left, and a node without extent stays a leaf.

```
void libxs_kdtree_build(const double* pts, int* idx, int n,
  int ndims, int stride, const libxs_kdtree_config_t* config);
```

Build a k-d tree in-place. Points are stored row-major: pts[i*stride + k] is coordinate k of point i. The index array idx[0..n-1] must hold the identity {0, 1, ..., n-1} on entry and is rearranged into implicit tree structure. config may be NULL (round-robin median, leaf at 1 element).

```
int libxs_kdtree_partition(const double* pts, int* idx, int n,
  int ndims, int stride, int* assignments,
  const libxs_kdtree_config_t* config);
```

Partition n points into leaves and write leaf IDs (in preorder) into assignments[0..n-1]. Returns the number of leaves. Same tree logic as build, but produces cluster assignments instead of a query-tree index. As for build, idx must hold the identity on entry.

```
int libxs_kdtree_descend(const libxs_kdtree_node_t nodes[],
  const double point[]);
```

The leaf id a point descends to in the nodes partition wrote (`config.nodes`): below a split's threshold left, otherwise right. The point is in the coordinates of the thresholds (those of `pts`, or of the split callback that wrote them). Returns -1 for NULL arguments, or where a node does not lead forward.

```
int libxs_kdtree_nearest(const double* pts, const int* idx,
  const unsigned char* used, int n, int ndims, int stride,
  const double* query, double max_dist2);
```

Find the nearest point to query[0..ndims-1] within squared Euclidean distance max_dist2. Returns the point index or -1. The used array (may be NULL) marks consumed points.

```
int libxs_kdtree_knearest(const double* pts, const int* idx,
  const unsigned char* used, int n, int ndims, int stride,
  const double* query, double max_dist2,
  int out_idx[], double out_dist2[], int k);
```

Find the k nearest points, writing indices to out_idx and their squared distances to out_dist2, both sorted ascending, and returning how many were written (0..k; fewer when n or max_dist2 bounds it).

The result is exact: the same k points a linear scan would pick, in the same order.

Both searches require a tree built with `config == NULL`, or with a config whose `split` is NULL or `libxs_kdtree_split_median` and whose `min_leaf` is at most 1; other trees cannot be searched.

The tree beats a linear scan while n is large against 2^{ndims}. Measured with k = 8, uniformly random points need n of about $64*2^{ndims}$ before the tree is faster. Structured data (a lower intrinsic dimension) pays earlier: 1.26x faster at 37 dimensions and 60,000 points, whereas data without such structure was 3.4x slower than a scan at 28 dimensions and 100,000 points. Whether to prefer a scan depends on the data rather than on ndims alone: time a few queries both ways.

```
int libxs_kdtree_knearest_budget(const double* pts, const int* idx,
  const unsigned char* used, int n, int ndims, int stride,
  const double* query, double max_dist2,
  int out_idx[], double out_dist2[], int k, int budget);
```

As `libxs_kdtree_knearest` (same tree and output), but searched best bin first: the waiting subtree with the smallest lower bound of its distance next, until none can hold a nearer point or `budget` points have been examined (one distance each). With `budget <= 0` the result is exact; with a budget it is the k nearest among the points examined. This is the search for dimensions where the exact one visits most of the tree. Measured with k = 8 (recall: the share of the exact k nearest found, ties counted as found): at 37 dimensions and 60,000 structured points, a budget of 1024 was 13.6x faster than a scan at a recall of 0.95 (4096: 3.5x, 0.99); at 28 dimensions and 100,000 points without such structure, 4096 was 3.9x faster at 0.73 (16384: as fast as a scan, 0.91). Without a budget, `libxs_kdtree_knearest` is the faster exact search.

Convenience wrappers for 2D (interleaved x,y layout):

```
void libxs_kdtree2d_build(double* pts, int* idx, int n);
int libxs_kdtree2d_nearest(const double* pts, const int* idx,
  const unsigned char* used, int n,
  double x, double y, double max_dist2);
```

Spatial Index

```
typedef struct libxs_spatial_t {
  uint64_t* codes;
  void** values;
  int n;
} libxs_spatial_t;

int libxs_spatial_build(libxs_spatial_t* sp,
  const libxs_registry_t* registry);
int libxs_spatial_build_codes(libxs_spatial_t* sp,
  const uint64_t codes[], void* const values[], int n);
int libxs_spatial_nearest(const libxs_spatial_t* sp,
  uint64_t query_code, int k, void** out_values);
void libxs_spatial_destroy(libxs_spatial_t* sp);
```

An index of values sorted by a 64-bit locality code, such as `libxs_hilbert`. `libxs_spatial_build` takes the codes from the first eight bytes of each key of a registry (keys of at least eight bytes), and `libxs_spatial_build_codes` from parallel arrays, which it copies; use the latter when the registry key is not a locality code. Both return EXIT_SUCCESS, or EXIT_FAILURE for invalid arguments, an empty input, or a failed allocation. The values are stored as pointers, not copied.

`libxs_spatial_nearest` writes up to k values whose codes are numerically closest to query_code to out_values and returns how many were written. `libxs_spatial_destroy` releases the index.

Smooth Sorting

```
typedef enum libxs_sort_t {
  LIBXS_SORT_NONE     = 0,
  LIBXS_SORT_IDENTITY = 1,
  LIBXS_SORT_NORM     = 2,
  LIBXS_SORT_MEAN     = 3,
  LIBXS_SORT_GREEDY   = 4,
  LIBXS_SORT_MORTON   = 5,
  LIBXS_SORT_HILBERT  = 6
} libxs_sort_t;
```

```
int libxs_sort_smooth(libxs_sort_t method, int m, int n,
  const void* mat, int ld, libxs_data_t datatype, int* perm);
```

Compute a row permutation that reorders the rows of an m-by-n column-major matrix for smoothness (decaying forward differences between consecutive rows).

Parameter	Description
`method`	Sorting strategy (see constants above)
`m`, `n`	Matrix dimensions (m rows, n columns)
`mat`	Column-major matrix data (read-only)
`ld`	Leading dimension (ld >= m)
`datatype`	Element type (F64, F32, I32, U32, I16, U16, I8, U8)
`perm`	Output array of m ints; perm[i] = source row for position i after reordering

Method	Description
`NONE`	No permutation (early return, perm is not written)
`IDENTITY`	Identity permutation (perm[i] = i)
`NORM`	Sort rows by ascending L1-norm
`MEAN`	Sort rows by ascending row mean (mean over the columns)
`GREEDY`	Nearest-neighbor chain: starting from row 0, each step picks the unvisited row with the smallest Euclidean distance to the current row
`MORTON`	Sort rows by Morton (Z-order) key computed from quantized column values (first 53 columns)
`HILBERT`	Sort rows by Hilbert curve key computed from quantized column values (first 53 columns)

Returns EXIT_SUCCESS or EXIT_FAILURE.

Parameter Prediction

Start Presentation

Header: `libxs_predict.h`

Predict output parameters from input parameters using fingerprint-guided polynomial interpolation and kNN classification. Supports incremental training, model persistence, confidence-gated evaluation, output transforms, and thread-safe operation.

Mode Flags

```
typedef enum libxs_predict_mode_t {
  LIBXS_PREDICT_AUTO        = 0,
  LIBXS_PREDICT_INTERPOLATE = 1,
  LIBXS_PREDICT_CLASSIFY    = 2,
  LIBXS_PREDICT_TEMPORAL    = 4
} libxs_predict_mode_t;
```

ORable flags controlling prediction behavior:

- AUTO (0): fingerprint decides per-output (default).
- INTERPOLATE: force polynomial for all outputs.
- CLASSIFY: force kNN vote for all outputs.
- TEMPORAL: timeseries mode — enables recency weighting (recent neighbors preferred), continuous output (no snap-to-nearest), and local coherence smoothing across horizon steps. Only effective when set_series was called (nseries > 0). For timeseries models without this flag, temporal heuristics auto-enable only when the query falls outside the training bounding box.

Example: `LIBXS_PREDICT_CLASSIFY | LIBXS_PREDICT_TEMPORAL` forces kNN-weighted projection forward with temporal heuristics.

Input Decomposition and Feature Selection

```
typedef enum libxs_predict_decompose_t {
  LIBXS_PREDICT_AUTO_DECOMPOSE = -1,
  LIBXS_PREDICT_RAW     = 0,
  LIBXS_PREDICT_SPREAD  = 1,
  LIBXS_PREDICT_PCA     = 2,
  LIBXS_PREDICT_SETDIFF = 3,
  LIBXS_PREDICT_FISHER  = 4,
  LIBXS_PREDICT_RF      = 5,
  LIBXS_PREDICT_HKNN    = 6
} libxs_predict_decompose_t;
```

Controls input processing and prediction strategy:

- AUTO_DECOMPOSE (-1): choose among the applicable modes at build by trial (each is built on part of the corpus and scored on a held-back part). Read the chosen mode from `libxs_predict_query_t::decompose`.
- RAW (0): no decomposition, standard kNN (default).
- SPREAD: sum/diff modes for anti-correlated pairs (2 series). Forward: sum = (A+B)/2, diff = (A-B)/2. Inverse: A = sum+diff, B = sum-diff. Only effective when nseries == 2.
- PCA: principal component rotation of the input space with dimensionality reduction.
- SETDIFF: supervised feature selection by class separability (libxs_setdiff).
- FISHER: supervised feature weighting by Fisher's discriminant ratio.
- RF: Random Forest per output, 64 trees by default (see `libxs_predict_set_forest`). An integral output spanning at most 128 levels is classified, any other output is regressed. Persisted in save/load.
- HKNN: hierarchical kNN. An output-aware partition replaces the clustering step; inference is kNN within each partition.

PCA/SPREAD decomposition is applied at build time to stored entries and at eval time to user queries. Inverse prediction returns inputs in raw (user) space. Feature selection modes (SETDIFF, FISHER) only set weights at build time. RF replaces the kNN eval path entirely. HKNN replaces only the clustering step (kNN inference is unchanged).

Output Transforms

```
typedef enum libxs_predict_transform_t {
  LIBXS_PREDICT_IDENTITY = 0,
  LIBXS_PREDICT_LOG      = 1,
  LIBXS_PREDICT_SQRT     = 2
} libxs_predict_transform_t;
```

Per-output transforms applied transparently:

- IDENTITY (0): no transform (default).
- LOG: forward = log(x+1), inverse = exp(x)-1.
- SQRT: forward = sqrt(x), inverse = x*x.

The forward transform is applied during push, the inverse during eval. The fingerprint and kNN operate in transformed space where heavy-tailed data is smoother.

Create and Destroy

```
libxs_predict_t* libxs_predict_create(int ninputs, int noutputs);
void libxs_predict_destroy(libxs_predict_t* model);
libxs_lock_t* libxs_predict_lock(libxs_predict_t* model);
```

Create a model for the given input/output dimensionality. Returns NULL on invalid arguments or allocation failure. Destroy accepts NULL. The lock accessor returns a pointer to the model's internal lock for use with push/eval (NULL if model is NULL).

Configuration

```
void libxs_predict_set_mode(libxs_predict_t* model, int mode);
```

Set prediction mode (ORable flags from libxs_predict_mode_t). Default is LIBXS_PREDICT_AUTO.

```
void libxs_predict_set_weights(libxs_predict_t* model,
  const double weights[]);
```

Set per-dimension input weights for distance computation. Larger weight means the dimension contributes more. Pass NULL to reset to uniform weighting. Must be called before libxs_predict_build.

```
void libxs_predict_set_transform(libxs_predict_t* model,
  int output, int transform);
```

Set per-output transform. The output parameter is 0-based, or -1 to set all outputs. Must be called before pushing entries. Transforms are persisted in save/load.

```
void libxs_predict_set_refine(libxs_predict_t* model,
  int iterations);
```

Set the number of forward-inverse-forward refinement iterations applied during eval.

- iterations=0 (default): off, except for a single pass when set_consistency>0 and a per-output confidence is below 0.9.
- iterations<0: a single pass when a per-output confidence is below 0.9.
- iterations>0: always perform that many iterations regardless of confidence.

```
void libxs_predict_set_smooth(libxs_predict_t* model,
  double amount);
```

Control multi-cluster blending at eval time.

- amount<0: auto-determine blending radius from model structure.
- amount=0 (default): no blending.
- amount>0: blend predictions from clusters within (1+amount)*nearest_distance radius. Only smooth-mode outputs are blended; categorical outputs remain unblended.

```
void libxs_predict_set_consistency(libxs_predict_t* model,
  double amount);
```

Set round-trip consistency penalty (0..1). During the refinement loop, the prediction is inverse-mapped back to input space. If the reconstructed input differs from the original query by more than the cluster diameter, the prediction is geometrically inconsistent.

- amount=0 (default): inconsistency only skips refinement (no confidence change).
- amount>0: confidence is actively penalized proportional to the round-trip distance: `conf = floor + (conf - floor) / (1` where floor is set by `libxs_predict_set_floor` (default 0).
- amount=1: halves confidence above the floor when the round-trip distance equals the cluster diameter.

This catches predictions where the model contradicts itself: the output maps back to a different input region than the query came from.

```
void libxs_predict_set_quantile(libxs_predict_t* model,
  double quantile);
```

Set quantile level for prediction intervals (0..0.5). When enabled, `info->lower` and `info->upper` are filled with the q-th and (1-q)-th distance-weighted quantiles of the k nearest neighbors.

- quantile=0 (default): intervals not computed, `info->lower` and `info->upper` are NULL.
- quantile>0 (e.g. 0.1): 10th/90th percentile bounds, scaled by `1/confidence` so sparse regions produce wider intervals and dense regions stay close to the raw neighbor spread.

```
void libxs_predict_set_central(libxs_predict_t* model, int mode);
void libxs_predict_set_neighbors(libxs_predict_t* model, int k);
void libxs_predict_set_reach(libxs_predict_t* model, int nclusters);
void libxs_predict_set_coordinate(libxs_predict_t* model,
  int coordinate);
void libxs_predict_set_floor(libxs_predict_t* model, double floor);
void libxs_predict_set_missing(libxs_predict_t* model, int enable);
void libxs_predict_set_forest(libxs_predict_t* model,
  int ntrees, int max_depth);
```

Setter	Values
`set_central`	Vote of a many-valued output: 1 median, 2 mean, 0 (default) chosen per output at build; negative also prints the choice.
`set_neighbors`	Neighbors the vote reads: k>0 pins it, 0 (default) derives it, k<0 selects it per output at build by trial. Persisted.
`set_reach`	Further clusters a flat vote gathers from, scanned by the nearest any member can be and stopped once none can hold one of the k nearest: n>0 at most n, n<0 decided at build by trial (with or without at most |n|), 0 (default) that trial at 4, `LIBXS_PREDICT_REACH_NONE` none. Set before build; persisted.
`set_coordinate`	Coordinate an input contributes to the distance (`libxs_predict_coord_t`, default `LIBXS_PREDICT_AUTO_COORD`).
`set_floor`	Floor (0..1) the confidence rescalings pull toward; 0 (default) disables them. Persisted.
`set_missing`	Non-zero: `load_csv` admits rows with an empty or unparsable input field as absent (NaN) instead of skipping them (default 0). Call before push.
`set_forest`	`LIBXS_PREDICT_RF` trees per output (ntrees>0, default 64) and depth (max_depth>0 fixed, 0 (default) derived from the entry count, <0 selected per output at build). Call before build; not persisted.

```
typedef enum libxs_predict_coord_t {
  LIBXS_PREDICT_AUTO_COORD = 0,
  LIBXS_PREDICT_RANK       = 1,
  LIBXS_PREDICT_SCORE      = 2
} libxs_predict_coord_t;
```

- AUTO_COORD (0): RANK or SCORE chosen at build by trial, from 8 inputs on; with fewer inputs, RANK.
- RANK: an input is placed by its rank among the stored entries.
- SCORE: an input is placed by what its value says about the output. Needs one discrete output and no series, and applies to RAW and HKNN only. A loaded model built this way reports no corpus.

Absent inputs (NaN, pushed directly or admitted by `set_missing`) are omitted from the distance. Interpolation abstains for a query with an absent input, and build fails under PCA, SPREAD, RF or HKNN when an absence is present.

Timeseries Configuration

```
#define LIBXS_PREDICT_AUTO_WINDOW 0

void libxs_predict_set_series(libxs_predict_t* model,
  int nseries, int window);
void libxs_predict_set_series_deriv(libxs_predict_t* model,
  int nderiv);
void libxs_predict_set_series_aux(libxs_predict_t* model,
  int naux);
void libxs_predict_set_series_bank(libxs_predict_t* model,
  int nbank);
void libxs_predict_set_target(libxs_predict_t* model,
  int target);
void libxs_predict_set_decompose(libxs_predict_t* model,
  int decompose);
void libxs_predict_set_diff(libxs_predict_t* model,
  int order);
```

Declare timeseries structure for automatic sliding-window construction. For a positive window, the model's ninputs must equal nseries * window + nderiv + naux (see `libxs_predict_set_series_deriv` and `libxs_predict_set_series_aux`); noutputs is the forecast horizon. With window = `LIBXS_PREDICT_AUTO_WINDOW` (0), the window is selected at build, and a negative window selects it by trial; in both cases ninputs is an upper bound and `libxs_predict_query_t::window` reports the window in force. A negative window builds candidate windows and keeps the one with the least held-back error over the horizon; its magnitude, when at least 4, is one more candidate. This costs several builds (see `LIBXS_PREDICT_WINDOW_FOLDS`).

- set_series_deriv: append nderiv terminal first-differences of the target window as inputs (default 0). Call before push.
- set_series_aux: naux exogenous per-timestep features (e.g. day-of-year), pushed after the series values and neither windowed nor transformed (default 0). Call before push.
- set_series_bank: average the forecast over nbank views of the window, each halving the lags of the one before (default 1 = off). Call before build; `libxs_predict_query_t::nbank` reports the views built. `libxs_predict_inverse` abstains on a loaded model with views.

When set_series has been called, push(lock, model, values, NULL) accumulates one timestep (nseries + naux values per call: the series first, then the auxiliary features). The build step then constructs all valid sliding windows internally: input = concatenated windows across all series, output = the next horizon values of the target series.

set_target selects which series to predict (0-based, default 0). set_decompose selects input processing mode (see above).

set_diff enables auto-differencing for non-stationary series:

- `set_diff(model, 0)`: auto-detect order from fingerprint decay.
- `set_diff(model, d)`: explicit order d (1 = linear detrend).
- Default is disabled (-1).

Single-series example (sunspots):

```
model = libxs_predict_create(WINDOW, HORIZON);
libxs_predict_set_series(model, 1, WINDOW);
for (t = 0; t < n; ++t)
    libxs_predict_push(NULL, model, &series[t], NULL);
libxs_predict_build(model, 0, 2, 0);
```

Multi-series example (anti-correlated pair):

```
model = libxs_predict_create(WINDOW * 2, HORIZON);
libxs_predict_set_series(model, 2, WINDOW);
libxs_predict_set_target(model, 0);
libxs_predict_set_decompose(model, LIBXS_PREDICT_SPREAD);
for (t = 0; t < n; ++t) {
    double vals[2] = {A[t], B[t]};
    libxs_predict_push(NULL, model, vals, NULL);
}
libxs_predict_build(model, 0, 2, 0);
```

Non-stationary example (trending stock prices):

```
model = libxs_predict_create(WINDOW * 2, HORIZON);
libxs_predict_set_mode(model, LIBXS_PREDICT_TEMPORAL);
libxs_predict_set_series(model, 2, WINDOW);
libxs_predict_set_target(model, 0);
libxs_predict_set_decompose(model, LIBXS_PREDICT_SPREAD);
libxs_predict_set_diff(model, 0);
for (t = 0; t < n; ++t) {
    double vals[2] = {price_A[t], price_B[t]};
    libxs_predict_push(NULL, model, vals, NULL);
}
libxs_predict_build(model, 0, 2, 0);
```

Training

```
int libxs_predict_push(libxs_lock_t* lock,
  libxs_predict_t* model,
  const double inputs[], const double outputs[]);
```

Push one training entry. When set_series was called and outputs is NULL, inputs holds nseries + naux values representing one timestep; sliding windows are constructed at build time. At eval, such a model takes the raw window followed by the naux auxiliary values (nseries * window + naux values). Returns EXIT_SUCCESS or EXIT_FAILURE.

```
int libxs_predict_push_weighted(libxs_lock_t* lock,
  libxs_predict_t* model, const double inputs[], const double outputs[],
  double weight);
```

Push one entry whose kNN vote counts weight times (weight > 0; 1 is exactly push). Only the vote reads the weight, not the polynomial fit, the partition or the probability API. In series mode only weight 1 is accepted. Weights are persisted.

```
int libxs_predict_build(libxs_predict_t* model,
  int nclusters, int order, double quality);
```

Build the model from pushed entries. nclusters=0 selects sqrt(n) clusters automatically. order>0 uses at most that order, order=0 auto-optimizes, order<0 scans up to |order|.

The quality parameter (0..1) controls model compression:

- quality=0: no compression (default).
- quality>0: a leave-one-out pass removes the entries that every output recovers exactly from a unanimous neighborhood. For a classify output every positive value yields the same drop set; an interpolating output reads the magnitude as a residual tolerance. The quality is persisted in save/load.

Quality does not affect confidence. With `libxs_predict_set_floor` > 0, confidence is discounted for underpopulated clusters and for queries far from the cluster center, approaching the floor but never going below it. The floor is persisted in save/load.

```
int libxs_predict_build_task(libxs_predict_t* model,
  int nclusters, int order, double quality, int tid, int ntasks);
```

Per-thread collective form. All tasks call with the same parameters, and every task must enter: a task that does not enter stops the build. The model owns the barrier, so no lock or barrier is passed. Two models may be built by two teams at once; the same model may not be built concurrently.

Evaluation

```
void libxs_predict_eval(libxs_lock_t* lock,
  const libxs_predict_t* model,
  const double inputs[], double outputs[],
  libxs_predict_info_t* info, int nblend);
```

Predict outputs for given inputs. nblend controls multi-cluster blending: nblend>=2 averages that many clusters, 0 and 1 are automatic (the nearest cluster answers unless its confidence calls for more evidence), and nblend<0 pins the answer to the nearest cluster. The info pointer (optional) receives per-output confidence, variance, error bounds, and mode flags.

```
int libxs_predict_shape(const libxs_predict_t* model,
  int* ninputs, int* noutputs);
void libxs_predict_eval_flat(const libxs_predict_t* model,
  const double inputs[], double outputs[], double confidence[],
  double variance[], double lower[], double upper[]);
```

shape reports the model's ninputs and noutputs (either pointer may be NULL; EXIT_FAILURE for a NULL model). eval_flat is eval with the per-output fields copied into caller arrays of noutputs each, every one optional (NULL skips it): the form for a foreign-function caller (ctypes, Fortran) that should not mirror `libxs_predict_info_t`. Without a quantile level, lower and upper receive the prediction.

```
void libxs_predict_inverse(libxs_lock_t* lock,
  const libxs_predict_t* model,
  const double target_outputs[], double inputs[],
  libxs_predict_info_t* info);
```

Inverse prediction: find inputs that produce desired outputs.

```
void libxs_predict_eval_batch(
  const libxs_predict_t* model,
  const double inputs_batch[], double outputs_batch[],
  int count, int nblend);

void libxs_predict_eval_batch_task(
  const libxs_predict_t* model,
  const double inputs_batch[], double outputs_batch[],
  int count, int nblend, int tid, int ntasks);
```

Batch prediction. The task variant distributes queries across threads by slice.

Query and Access

```
void libxs_predict_query(const libxs_predict_t* model,
  libxs_predict_query_t* info);

void libxs_predict_get(const libxs_predict_t* model,
  int index, double inputs[], double outputs[]);
```

query fills a statistics struct (see `libxs_predict_query_t` below). get retrieves the i-th pushed entry (0-based).

`nscan` is the number of candidates a query scans in the worst case (the largest cluster); a model built with a single cluster scans every entry per query.

Probability

```
void* libxs_predict_prob_create(const libxs_predict_t* model);
void libxs_predict_prob_destroy(void* context);
int libxs_predict_prob_commit(libxs_predict_t* model,
  const void* context);

void libxs_predict_prob(libxs_lock_t* lock,
  const libxs_predict_t* model, void* context,
  const double inputs[], const double candidate[], double prob[],
  libxs_predict_prob_info_t* info, int vocabulary, int nblend);
int libxs_predict_prob_observe(libxs_lock_t* lock,
  const libxs_predict_t* model, void* context,
  const double inputs[], int output, const double* candidate,
  double values[], double probs[], int capacity, double* novel,
  libxs_predict_prob_info_t* info, int vocabulary, int nblend);
int libxs_predict_prob_support(const libxs_predict_t* model,
  int output, double values[], int capacity);
```

These score a value the caller supplies, including one the model would never pick. `prob` is a point query for P(y|x); `prob_observe` reports the distribution over one discrete output and then optionally observes the outcome.

- `context != NULL` — adaptive; the context's weights adapt over the stream and the model is not modified.
- `context == NULL` — frozen; the model's stored weights are used and never written, so results do not depend on call order.

`prob_commit` copies a context's weights into the model. Call it before frozen scoring; without it, frozen scoring uses a uniform prior and scores poorly.

`prob_support` returns the attested support of a discrete output (distinct values, sorted ascending), writing up to `capacity` entries and nothing at all when the support is larger.

Calibration

```
int libxs_predict_recalibrate_prob(libxs_predict_t* model,
  const double* inputs, const double* outputs, int nentries);
int libxs_predict_recalibrate_interval(libxs_predict_t* model,
  const double* inputs, const double* outputs, int nentries);
int libxs_predict_interval(const libxs_predict_t* model, int output,
  double lower, double upper, double* lower_out, double* upper_out);
int libxs_predict_probability(const libxs_predict_t* model,
  int output, double confidence, double* probability);
```

A confidence ranks queries; it is not a probability. A forest carries a calibration curve fitted at build.

- probability: translates a confidence from eval into the probability that the reported RF class is correct. Returns EXIT_SUCCESS when a curve was applied, EXIT_FAILURE when the confidence passed through unchanged (still a ranking).
- recalibrate_prob: replaces a forest's curve with one fitted on rows the model was not built from (inputs: nentries * ninputs, outputs: nentries * noutputs, laid out as push takes them), e.g. after the deployment distribution changed. Fails for a model without a forest.
- recalibrate_interval: fits a correction of the prediction interval on such held-out rows. Requires a model built with `set_quantile` > 0; applies to any model kind.
- interval: applies that correction to an interval from eval. lower_out/upper_out always receive a value; EXIT_SUCCESS when corrected, EXIT_FAILURE when passed through unchanged.

Persistence

```
int libxs_predict_save(const libxs_predict_t* model,
  void* buffer, size_t* size);
libxs_predict_t* libxs_predict_load(
  const void* buffer, size_t size);
```

Save: pass buffer=NULL to query required size, then allocate and call again. Load: returns a ready-to-eval model or NULL.

Load accepts every released format version (currently 1 and 2) and defaults fields an older file does not carry. A v1.0.0 flat model stores no global entry order, so it abstains from `libxs_predict_inverse` and from recency weighting, also after saving it again. Version 2 adds a CRC32 trailer, so a truncated or corrupted file is rejected.

CSV Import

```
int libxs_predict_load_csv(libxs_predict_t* model,
  const char filename[], const char delims[],
  const char inputs[], const char outputs[],
  char header[], int header_size, char* delim_out);
```

Load delimited text and push entries. delims=NULL auto-detects separator. inputs and outputs are comma-separated column names (matched case-insensitively against the header line) or 0-based indices, as many as the model's ninputs and noutputs; NULL selects sequential columns. header (optional) receives the header line, delim_out (optional) the delimiter in use. Returns number of entries pushed, or -1 on error.

```
int libxs_predict_load_csv_opts(libxs_predict_t* model,
  const char filename[], const libxs_predict_csv_t* opts);

typedef struct libxs_predict_csv_t {
  const char* delims;
  const char* inputs;
  const char* outputs;
  char* header;
  int header_size;
  char* delim_out;
  int nrows;
  int stride;
  int offset;
} libxs_predict_csv_t;
```

load_csv with the arguments in a struct, for loading a subset of a large file. A zero-initialized struct (or opts=NULL) loads like load_csv with every argument NULL or zero.

- nrows: stop after that many entries were pushed (0: whole file).
- stride: push every stride-th admissible row (0 or 1: every row).
- offset: admissible rows to skip before the first one taken.

With stride and offset, one file splits into disjoint parts that each span the whole file. Unparsable and comment rows are not admissible and do not shift the selection.

Structures

```
typedef struct libxs_predict_info_t {
  const double* values;
  const double* error;
  const double* confidence;
  const double* variance;
  const double* lower;
  const double* upper;
  const int* interpolated;
```

```
  int noutputs;
  int cluster;
  double distance;
} libxs_predict_info_t;
```

Populated by eval when info is non-NULL. confidence holds per-output confidence (0..1), incorporating:

- variety: kNN vote agreement (classify) or 1.0 (interpolation),
- coverage: cluster population and query centrality (when the floor is >0, see libxs_predict_build),
- consistency: round-trip penalty (when set_consistency>0). lower/upper hold per-output prediction intervals (NULL when set_quantile is 0, i.e. disabled). When enabled, bounds are the distance-weighted quantiles from k neighbors scaled by 1/confidence. variance holds per-output neighbor disagreement. cluster gives the assigned cluster index (-1 if blended). distance gives normalized distance to nearest centroid. Confidence is a ranking; see Calibration for a probability.

```
typedef enum libxs_predict_pkind_t {
  LIBXS_PREDICT_PNONE = 0,
  LIBXS_PREDICT_PMASS = 1,
  LIBXS_PREDICT_PDENSITY = 2
} libxs_predict_pkind_t;

typedef struct libxs_predict_prob_info_t {
  const double* prob;
  const double* logprob;
  const double* zscore;
  const int* kind;
  const int* attested;
  const int* support;
  const double* novel;
  double total_logprob;
  int noutputs;
  int cluster;
  double entropy;
} libxs_predict_prob_info_t;
```

Populated by `libxs_predict_prob` and `libxs_predict_prob_observe`, per output where not noted otherwise:

- prob: P(y|x), a mass or a density according to kind (PMASS: discrete output, PDENSITY: continuous output, PNONE: no result, e.g. a blended or RF output).
- logprob: base-2 log of prob; read it rather than prob, which underflows to zero for an improbable candidate.
- zscore: (y - yhat)/h in density mode, 0 in mass mode.
- attested: non-zero if the candidate is in the local neighborhood. support: distinct values in the local support; novel: mass reserved for values outside it (mass mode).
- total_logprob: sum of logprob over outputs, assuming the outputs are conditionally independent (not a calibrated joint likelihood).
- cluster: cluster serving the query (-1 if blended).
- entropy: entropy in bits of the scoring weights, 0 once committed and log2(nexperts) at the uniform prior; tells whether the weights are still settling.

```
typedef struct libxs_predict_query_t {
  double compression;
  int ninputs;
  int noutputs;
  int version;    /* serialization version (current if built here) */
  int neighbors;  /* smallest resolved neighbor count, 0 if none */
  int order;
  int nclusters;
  int nentries;
  int corpus;     /* 0 if the entries cannot be rebuilt from */
  int iterations;
  int diff_order;
  int window;     /* effective sliding window (0 if series mode off) */
  int nbank;      /* window views built (1 when disabled) */
  int nscan;      /* entries in the LARGEST cluster */
  double escan;   /* entries an average query scans */
```

```
  int decompose;  /* decomposition mode in force */
  int coordinate; /* RANK, SCORE, or 0 if only scaled */
  int ncoord;     /* coordinates per stored point */
} libxs_predict_query_t;
```

Populated by query. compression = raw/model size ratio. ninputs/noutputs = the model's shape, which a caller evaluating into its own buffer sizes it from. order = polynomial order used. diff_order = differencing order (0 if disabled).

Environment Variables

All are read at build and default to the behavior without them.

Variable	Effect
`LIBXS_PREDICT_WINDOW_FOLDS`	Held-back splits scored per candidate window when `set_series` is given a negative window (default 3, at least 1).
`LIBXS_PREDICT_DECOMPOSE_FOLDS`	Folds scored per candidate when the decomposition or the coordinate is chosen by trial (default 3 for a series, 1 otherwise; 0 or less keeps the default).
`LIBXS_PREDICT_REFINE`	Overrides `libxs_predict_set_refine` (same values).
`LIBXS_PREDICT_REACH`	Overrides `libxs_predict_set_reach` at build (same values, `none` for `LIBXS_PREDICT_REACH_NONE`).
`LIBXS_PREDICT_RF_FRONTIER`	$N > 0$ builds each forest tree in up to N (at most 64) frontier parts; the trees are the same. Off by default.
`LIBXS_PREDICT_RF_RATE`	Shrinkage of the forest's boosted corrections (default 0.1, applied to regression outputs). Setting it also enables corrections for classification outputs; 0 or less disables them.

Registry

Header: `libxs_reg.h`

Key-value store backed by a hash table with per-thread caching. Operations are thread-safe when a lock is given (e.g., `libxs_registry_lock`).

Configuration Macros

Macro	Default	Description
`LIBXS_REGKEY_MAXSIZE`	64	Maximum key size in bytes
`LIBXS_REGISTRY_NBUCKETS`	64	Initial hash-table buckets (power of two); registries grow dynamically
`LIBXS_REGCACHE_NENTRIES`	16	Thread-local cache entries per thread (power of two; 0 disables caching)
`LIBXS_DESCRIPTOR_MAXSIZE`	`LIBXS_REGKEY_MAXSIZE`	Backward-compatible alias

Types

```
typedef struct libxs_registry_t libxs_registry_t;   /* opaque */

typedef struct libxs_registry_info_t {
  size_t capacity, size, nbytes;
} libxs_registry_info_t;
```

Lifetime

```
libxs_registry_t* libxs_registry_create(void);
```

Create a new registry. Returns `NULL` in case of an error.

```
void libxs_registry_destroy(libxs_registry_t* registry);
```

Destroy a registry and release all entries.

```
libxs_lock_t* libxs_registry_lock(libxs_registry_t* registry);
```

Return a pointer to the registry's internal lock. The returned lock can be passed as the `lock` argument to other registry functions (e.g., to hold the lock across multiple operations).

Iteration

```
void* libxs_registry_begin(const libxs_registry_t* registry,
  const void** key, size_t* cursor);
void* libxs_registry_next(const libxs_registry_t* registry,
  const void** key, size_t* cursor);
```

Enumerate entries. Initialize `*cursor` to 0 before the first `begin` call. Each call returns the value pointer and writes the key pointer to `*key`, or returns NULL when iteration is complete. The key pointer is suitably aligned to be cast to the caller's key type. Only the first `key_size` bytes of `*key` are defined.

```
void* libxs_registry_begin_length(const libxs_registry_t* registry,
  const void** key, size_t* key_size, size_t* cursor);
void* libxs_registry_next_length(const libxs_registry_t* registry,
  const void** key, size_t* key_size, size_t* cursor);
```

Same as `begin`/`next`, and also write the entry's key size to `*key_size` (may be NULL). Use these for a registry that holds keys of different sizes.

Access

```
void* libxs_registry_set(libxs_registry_t* registry,
  const void* key, size_t key_size,
  const void* value_init, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Insert or update a key-value pair. The key must be binary-reproducible (zero-initialize padding). `value_init` may be NULL to defer initialization. Re-registering an existing key reallocates if the new value is larger. When `lock` is NULL no locking is performed (the caller guarantees exclusion); pass `libxs_registry_lock(registry)` to use the registry's internal lock. Returns a pointer to the stored value.

```
void* libxs_registry_get(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Look up a value by key. Returns NULL if not found.

```
unsigned int libxs_registry_hash(const libxs_registry_t* registry,
  const void* key, size_t key_size);

void* libxs_registry_get_hashed(const libxs_registry_t* registry,
  const void* key, size_t key_size, unsigned int hash,
  libxs_lock_t* lock /* = NULL */);

void* libxs_registry_set_hashed(libxs_registry_t* registry,
  const void* key, size_t key_size, unsigned int hash,
  const void* value_init, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Hash a key once and reuse the value across several operations. A hash is only valid for the registry that produced it. `libxs_registry_get/_set` are wrappers that hash internally.

`libxs_registry_get_hashed` may answer a repeated query from the thread-local cache without taking the lock. This requires that the entry is neither removed nor resized while another thread queries it, e.g., a registry that only grows.

```
int libxs_registry_get_copy(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Thread-safe query: copies up to `value_size` bytes of the stored value into `value_out` under the lock. Unlike `libxs_registry_get`, the caller never receives a raw pointer into the registry's internal storage, making this safe under concurrent modifications. Returns non-zero if the key was found, zero otherwise.

```
int libxs_registry_has(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Check whether a key exists. Non-zero if found.

```
size_t libxs_registry_value_size(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Query the stored value size in bytes. Returns 0 if not found.

```
void libxs_registry_remove(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Remove a key-value pair and release its memory.

```
int libxs_registry_extract(libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Atomically retrieve and remove a key-value pair. Copies up to `value_size` bytes of the stored value into `value_out` (may be NULL to discard the value), then removes the entry and releases its memory. The lookup, copy, and removal are performed under a single lock hold, eliminating the race that exists when calling `libxs_registry_get` followed by `libxs_registry_remove` separately. Returns non-zero if the key was found, zero otherwise.

Serialization

```
int libxs_registry_save(const libxs_registry_t* registry,
  void* buffer, size_t* size);
```

Save the registry to a binary buffer. On input, `*size` is the capacity of `buffer`. Pass `buffer = NULL` to query the required size (written to `*size`). On success, `*size` contains the number of bytes written. Returns `EXIT_SUCCESS` or `EXIT_FAILURE`; a buffer that is too small fails and `*size` receives the required size.

```
libxs_registry_t* libxs_registry_load(const void* buffer, size_t size,
  void (*fixup)(void* value, const void* key, size_t key_size,
    size_t value_size, void* udata),
  void* udata /* = NULL */);
```

Load a registry from a binary buffer (previously produced by `libxs_registry_save`). Keys are materialized immediately. When `fixup` is NULL, values remain in the buffer and are accessed on demand via `libxs_registry_get`; the buffer must remain valid for the lifetime of the returned registry. When `fixup` is non-NULL, each value is heap-allocated immediately and the callback is invoked once per entry, allowing the caller to restore pointers or handles by key identity. Returns a new registry or `NULL` on failure.

Status

```
size_t libxs_registry_size(const libxs_registry_t* registry);
```

Number of entries (cheaper than `libxs_registry_info`).

```
int libxs_registry_info(const libxs_registry_t* registry,
  libxs_registry_info_t* info);
```

Query registry capacity, number of entries, and total bytes (table overhead and stored values). Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

Random Number Generator

Header: `libxs_rng.h`

Thread-safe pseudo-random number generation (SplitMix64 with per-thread TLS state).

Matrix Initialization Macros

```
LIBXS_MATRNG(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_SEQ(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_OMP(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
```

Fill a column-major matrix (NROWS x NCOLS, leading dimension LD) with deterministic values. The `_OMP` variant parallelizes with OpenMP; `_SEQ` is serial.

`LIBXS_MATRNG_VERSION` (currently 2) changes whenever the generated values change; record it alongside results so that data from different generators is not mixed.

ESPAN (exponent span) selects the initialization mode:

- `ESPAN == 0`: shuffle mode. A co-prime permutation of the linear index (over LD*NCOLS) gives each element a distinct value in [-|SCALE|, +|SCALE|].
- `ESPAN != 0`: adversarial exponent-span mode for stress-testing floating-point emulation (Ozaki scheme, etc.). Base values are shuffled in [1, 2), and column j is scaled by 2^e with

  ```
  e = sign(ESPAN) * (round(2*|ESPAN|*j / (NCOLS-1)) - |ESPAN|)
  ```

  so column 0 has exponent -ESPAN and the last column +ESPAN, 2*|ESPAN| binades in total. SCALE is ignored in this mode. Use +ESPAN for matrix A and -ESPAN for matrix B so that A*B remains well-conditioned while each operand individually has adversarial exponent range.

In both modes, padding rows [NROWS, LD) are zero-filled.

Example — adversarial DGEMM test with exponent span 512:

```
int espan = 512;
LIBXS_MATRNG(int, double, +espan, a, m, k, lda, 1.0);
LIBXS_MATRNG(int, double, -espan, b, k, n, ldb, 1.0);
```

Functions

```
void libxs_rng_set_seed(unsigned int seed);
```

Set the seed of the calling thread's PRNG state. Each thread maintains independent state via TLS. Unseeded threads start with a deterministic default (seed = 1).

```
unsigned int libxs_rng_u32(unsigned int n);
```

Return a pseudo-random value in [0, n) with uniform distribution, or zero for n <= 1. Thread-safe.

```
double libxs_rng_f64(void);
```

Return a double-precision value in [0, 1) with full 53-bit mantissa resolution. Thread-safe.

```
void libxs_rng_seq(void* data, size_t nbytes);
```

Fill a buffer with pseudo-random bytes. Thread-safe.

String Utilities

Header: `libxs_str.h`

Case-insensitive string search, edit distance, word-level matching and difference, similarity scoring, UTF-8 code point iteration, and value formatting.

Substring Search

```
const char* libxs_stristrn(const char a[], const char b[],
  size_t maxlen);
const char* libxs_stristr(const char a[], const char b[]);
```

Case-insensitive substring search. `stristrn` limits the match length to `maxlen` characters of `b`. Returns a pointer to the first match in `a`, or NULL.

```
int libxs_striequal(const char a[], size_t asize,
  const char b[], size_t bsize);
const char* libxs_strimem(const char a[], size_t asize,
  const char b[], size_t bsize);
```

Same comparisons on byte spans that need no terminator (e.g., a slice of a larger buffer). `striequal` returns non-zero if both spans are equal ignoring case (NULL equals nothing). `strimem` returns a pointer to the first occurrence of `b` within the `asize` bytes of `a`, or NULL (an empty `b` finds nothing).

Edit Distance

```
int libxs_stridist(const char a[], const char b[]);
```

Case-insensitive character-level edit distance between two strings. Returns the minimum number of single-character insertions, deletions, substitutions, or transpositions of adjacent characters to transform `a` into `b` (ignoring case), i.e., the restricted Damerau-Levenshtein distance. Returns -1 for NULL input.

Word Matching

```
int libxs_strimatch(const char a[], const char b[],
  const char delims[], int* count);
```

Word-level fuzzy matching. Counts how many words in `a` (or `b`) have a match in the other string, where two words match if one begins the other (case-insensitive), e.g., `Prod` and `Product`. The result is the smaller of both counts (symmetric). A word starting with `[` ends the words of its string (device IDs). Optional `delims` define word separators (default: space, tab, semicolon, comma, colon, dash). Optional `count` receives the word count of the string with more words. Returns -1 for invalid input.

Word Difference

```
int libxs_stridiff(const char a[], const char b[],
  const char delims[], int tolerance, int* count);
```

Word-level set difference with edit-distance tolerance. Counts how many words in the *smaller* string cannot be matched (within `tolerance` edits) to any word in the larger string. Matching is greedy (each word of the smaller string, in order, takes its cheapest unused partner), case-insensitive, and each word can be consumed at most once.

Parameters:

- `a`, `b` — input strings (NULL or empty returns -1)
- `delims` — word separator characters (NULL uses default: space, tab, semicolon, comma, colon, dash)
- `tolerance` — maximum edit distance (`libxs_stridist`) for two words to be considered a match (0 = exact match only, 1 = allows one edit such as plural/tense inflection)
- `count` — optional output, receives the word count of the larger string

At most 64 words per string are considered. Returns the number of unmatched words from the smaller string. A return value of 0 means every word in the shorter string found a match in the longer one. Returns -1 for NULL or empty input.

Properties:

- Symmetric when the word counts differ: `stridiff(a, b, ...)` equals `stridiff(b, a, ...)` because matching operates from the smaller side (from `a` if both have the same number of words).
- Order-independent: word positions do not affect the result.
- Tolerance=0 gives exact word-level multiset difference.
- Tolerance=1 handles common morphological variation (plurals, verb tenses, e.g., “thread” matches “threads”).

Example — measuring sentence redundancy:

```
int defect, total;
defect = libxs_stridiff(
  "The cache stores frequently accessed data.",
  "Frequently accessed data is kept in memory.",
  NULL, 1, &total);
/* defect=3 (unmatched: "The", "cache", "stores"), total=7 */
/* redundancy = 1 - defect/total = 0.57 */
```

Word Similarity

```
typedef enum libxs_strisimilar_t {
  LIBXS_STRISIMILAR_GREEDY,
  LIBXS_STRISIMILAR_TWOOPT,
  LIBXS_STRISIMILAR_DEFAULT = LIBXS_STRISIMILAR_GREEDY
} libxs_strisimilar_t;

int libxs_strisimilar(const char a[], const char b[],
  const char delims[], libxs_strisimilar_t kind, int* order);
```

Word-level similarity score combining edit distance and word-order analysis.

Strings are split into words using the same delimiters as `libxs_strimatch`. Each word in `a` is matched to a word in `b` by a heuristic for minimum-cost bipartite matching, where the cost of a pair is the character-level edit distance of `libxs_stridist` (case-insensitive, an adjacent transposition counting as one edit). Unmatched words (when the strings have different word counts) contribute their full length. At most 64 words per string are considered, and as with `libxs_strimatch` a word starting with `[` ends the words of its string.

The matching strategy is selected by `kind`:

- `GREEDY` — picks the globally cheapest pair first.
- `TWOOPT` — refines the greedy result by iteratively swapping pairs whenever a swap reduces total cost.

The optional `order` output receives the number of pairwise inversions among matched words (Kendall tau distance), measuring how much the word order differs (0 = same order).

Returns the total edit distance (0 for identical word sets in any order), or -1 for invalid input.

Token Extraction

```
const char* libxs_strtoken(const char str[],
  const char delims[], int index, int* length);
```

Non-destructive access to the `index`-th token in a delimited string. Tokens are separated by characters in `delims` (NULL or empty: `LIBXS_DELIMS`, i.e., comma, semicolon, colon). Leading and trailing whitespace within each token is trimmed. Returns a pointer into `str` at the token start, or NULL if `index` is out of range. Optional `length` receives the trimmed token length.

UTF-8 Code Points

```
size_t libxs_utf8_size(const unsigned char text[], size_t size,
  size_t pos);
unsigned long libxs_utf8_decode(const unsigned char text[],
  size_t size, int* width);
```

`libxs_utf8_size` is **lenient**: it reports the width the lead byte at `text[pos]` claims, clamped to the bytes that remain, and never less than 1 so a scan always advances. It does not inspect continuation bytes. Use it to walk text a code point at a time.

`libxs_utf8_decode` is **strict**: it returns the code point and writes the bytes consumed to `width` (may be NULL). A sequence that is truncated, or whose continuation bytes are not continuation bytes, yields the **lead byte** as the value and a width of 1. Use it when the code point value is wanted.

On well-formed text the two agree, and a scan by either covers the string exactly once. They diverge only on malformed input, where the lenient form skips the claimed span and the strict form skips one byte:

Input	utf8_size	utf8_decode
41	1	U+0041, width 1
C3 A4	2	U+00E4, width 2
E2 80 99	3	U+2019, width 3
C3 78	2	U+00C3, width 1
C3 (alone)	1 (clamped)	U+00C3, width 1

Value Formatting

```
size_t libxs_format_value(char buffer[], int buffer_size,
  size_t nbytes, const char scale[], const char* unit, int base);
```

Format a scalar value with binary scaling: each step divides by 2^base and selects the next character of scale. Example:

```
libxs_format_value(buf, sizeof(buf), nbytes, "KMGT", "B", 10);
```

produces a human-readable byte count such as "128 KB". Returns the value in the selected unit so the caller can decide whether to print the buffer.

Synchronization

Header: libxs_sync.h

Thread-local storage, atomic operations, lock abstractions (spin/mutex/rwlock/atomic), file locking, and stdio synchronization.

Thread-Local Storage

LIBXS_TLS

Storage-class qualifier for thread-local variables. Maps to __thread, __declspec(thread), or thread_local depending on the compiler. Defined as empty when TLS is unavailable or disabled (LIBXS_NO_TLS).

Key-based thread-local values map to POSIX thread keys or Windows TLS slots (defined unless LIBXS_SYNC is 0); return values are those of the platform:

Macro	Description
LIBXS_TLS_TYPE	Key type (pthread_key_t, or DWORD on Windows)
LIBXS_TLS_CREATE(KEYPTR)	Create a key (no destructor)
LIBXS_TLS_DESTROY(KEY)	Delete a key
LIBXS_TLS_SETVALUE(KEY, PTR)	Set the calling thread's value
LIBXS_TLS_GETVALUE(KEY)	The calling thread's value (NULL if not set)

Atomic Operations

The header provides a suite of macros for atomic loads, stores, compare-and-swap, and arithmetic. The implementation selects among GCC builtins (`__atomic_*`), legacy GCC sync builtins, Windows Interlocked intrinsics, or no-op fallbacks depending on the compiler and `LIBXS_SYNC` setting.

Macro	Description
`LIBXS_ATOMIC_LOAD(SRC, KIND)`	Atomic load
`LIBXS_ATOMIC_STORE(DST, VALUE, KIND)`	Atomic store
`LIBXS_ATOMIC_STORE_ZERO(DST, KIND)`	Atomic store of zero
`LIBXS_ATOMIC_CMPSWP(DST, OLDVAL, NEWVAL, KIND)`	Compare-and-swap
`LIBXS_ATOMIC_FETCH_OR(DST, VALUE, KIND)`	Fetch-and-or
`LIBXS_ATOMIC_FETCH_ADD(DST, VALUE, KIND)`	Fetch-and-add
`LIBXS_ATOMIC_FETCH_SUB(DST, VALUE, KIND)`	Fetch-and-subtract
`LIBXS_ATOMIC_ADD_FETCH(DST, VALUE, KIND)`	Add-and-fetch
`LIBXS_ATOMIC_SUB_FETCH(DST, VALUE, KIND)`	Subtract-and-fetch
`LIBXS_ATOMIC_TRYLOCK(DST, KIND)`	Try-lock (returns acquired state)
`LIBXS_ATOMIC_ACQUIRE(DST, NPAUSE, KIND)`	Spin-acquire with backoff
`LIBXS_ATOMIC_RELEASE(DST, KIND)`	Release (store zero)
`LIBXS_ATOMIC_SYNC(KIND)`	Full memory fence

```
typedef enum libxs_atomic_kind_t {
  LIBXS_ATOMIC_SEQ_CST, /* __ATOMIC_SEQ_CST if available */
  LIBXS_ATOMIC_RELAXED  /* __ATOMIC_RELAXED if available */
} libxs_atomic_kind_t;
```

The `KIND` parameter selects the memory order (`LIBXS_ATOMIC_SEQ_CST` or `LIBXS_ATOMIC_RELAXED`) with the GCC builtins (`__atomic_*`). The legacy sync builtins and the Windows Interlocked intrinsics ignore it, and `LIBXS_ATOMIC_SYNC` is always a full fence.

The unqualified operations are 32-bit in some configurations. Select a width explicitly with `LIBXS_ATOMIC(FN, BITS)`, where `BITS` is 8, 16, 32, or 64, e.g., `LIBXS_ATOMIC(LIBXS_ATOMIC_ADD_FETCH, 64)(&counter, 1, LIBXS_ATOMIC_RELAXED)`. `LIBXS_ATOMIC_SIZE(FN)` selects the width of `size_t`.

Macro	Description
`LIBXS_SYNC_PAUSE`	CPU pause hint for spin loops (no-op where unavailable)
`LIBXS_SYNC_YIELD`	Yield the processor to another thread
`LIBXS_NPAUSE_LOCK`	Pauses before a waiting lock yields (default 4096); suits `NPAUSE` of `LIBXS_ATOMIC_ACQUIRE`
`LIBXS_NPAUSE_BARRIER`	Pauses before a waiting barrier task yields (default 1024)
`LIBXS_SYNC_CYCLE(PTR, STATE, NPAUSE)`	Spin until the lowest bit of `*PTR` equals that of `STATE`; pauses double up to `NPAUSE`, then each round yields
`LIBXS_SYNC_CYCLE_EQ(PTR, VALUE, NPAUSE)`	Like `LIBXS_SYNC_CYCLE`, but until `*PTR` equals `VALUE`
`LIBXS_SYNC_CYCLE_ELSE(PTR, STATE, NPAUSE, ELSE)`, `LIBXS_SYNC_CYCLE_EQ_ELSE(...)`	As above; `ELSE` is a statement run after each yield

Lock Abstraction

```
LIBXS_LOCK_TYPE(KIND)
LIBXS_LOCK_INIT(KIND, LOCK, ATTR)
LIBXS_LOCK_DESTROY(KIND, LOCK)
LIBXS_LOCK_ACQUIRE(KIND, LOCK)
LIBXS_LOCK_TRYLOCK(KIND, LOCK)
LIBXS_LOCK_RELEASE(KIND, LOCK)
```

Generic lock interface parameterized by `KIND`. `LIBXS_LOCK_TRYLOCK` succeeded if its result equals `LIBXS_LOCK_ACQUIRED(KIND)`, a value that depends on the backend.

KIND	Backend
`LIBXS_LOCK_SPINLOCK`	Atomic spin-lock
`LIBXS_LOCK_MUTEX`	Pthreads mutex / Windows SRW lock (Windows mutex if targeting pre-Vista)
`LIBXS_LOCK_RWLOCK`	Pthreads reader-writer lock / Windows SRW lock (Windows mutex if targeting pre-Vista)
`LIBXS_LOCK_ATOMIC`	Lightweight atomic lock

The lock kind used by the library is `LIBXS_LOCK`, which defaults to `LIBXS_LOCK_DEFAULT` (`LIBXS_LOCK_ATOMIC`). Define `LIBXS_LOCK` before including LIBXS to choose another kind.

`ATTR` is a pointer to lock attributes, which are created per kind:

```
LIBXS_LOCK_ATTR_TYPE(KIND) attr;
LIBXS_LOCK_ATTR_INIT(KIND, &attr);
LIBXS_LOCK_INIT(KIND, &lock, &attr);
LIBXS_LOCK_ATTR_DESTROY(KIND, &attr);

typedef LIBXS_LOCK_TYPE(LIBXS_LOCK) libxs_lock_t;
```

General-purpose lock type. Instances of `libxs_lock_t` are used by the registry and other library components; users may also create their own.

Reader-writer variants are available for `LIBXS_LOCK_RWLOCK`:

```
LIBXS_LOCK_ACQREAD(KIND, LOCK)
LIBXS_LOCK_RELREAD(KIND, LOCK)
LIBXS_LOCK_TRYREAD(KIND, LOCK)
```

`LIBXS_LOCK_TYPE_ISRW(KIND)` is 1 if the kind provides genuine shared (reader) locking; otherwise the read variants lock exclusively. `LIBXS_LOCK_TYPE_ISPOD(KIND)` is 1 if the lock type is plain data.

Barrier

```
typedef struct libxs_barrier_t libxs_barrier_t; /* POD defined in the header */

void libxs_barrier_init(libxs_barrier_t* barrier, int ntasks);  /* inline */
void libxs_barrier_wait(libxs_barrier_t* barrier);              /* inline */
int libxs_barrier_bcast(libxs_barrier_t* barrier,
  int tid, int root, int value);                                /* inline */
```

Rendezvous over a fixed number of tasks. The caller owns the storage (nothing is allocated inside), and one task calls `libxs_barrier_init` once, before any task waits (`ntasks` below 1 counts as 1). A task is only a number, so the team need not be a thread team.

Every waiting task spins, so the barrier suits a team that meets between stages of work, and a team that over-subscribes the machine slows down.

`libxs_barrier_bcast` waits and returns the `int` value the `root` task passed in (`tid` is the calling task's number). Use it to hand a value from one task to the rest of the team.

Ring

```
typedef enum libxs_ring_kind_t {
  LIBXS_RING_PRIVATE = 0,
  LIBXS_RING_SHARED = 1
} libxs_ring_kind_t;
typedef struct libxs_ring_t libxs_ring_t; /* header placed in caller memory */

size_t libxs_ring_size(size_t capacity);
libxs_ring_t* libxs_ring_init(void* memory, size_t size, libxs_ring_kind_t kind);
void* libxs_ring_reserve(libxs_ring_t* ring, size_t nbytes, int timeout_ms);
void libxs_ring_commit(libxs_ring_t* ring);
const void* libxs_ring_peek(libxs_ring_t* ring, size_t* nbytes, int timeout_ms);
void libxs_ring_release(libxs_ring_t* ring);
void libxs_ring_close(libxs_ring_t* ring);
```

Usage:

```
const size_t size = libxs_ring_size(capacity);
libxs_ring_t* ring = libxs_ring_init(memory, size, LIBXS_RING_SHARED);
void* record = libxs_ring_reserve(ring, nbytes, timeout_ms); /* producer */
libxs_ring_commit(ring);
const void* data = libxs_ring_peek(ring, &nbytes, timeout_ms); /* consumer */
libxs_ring_release(ring);
libxs_ring_close(ring);
```

Records of varying size passed from one producer to one consumer, in order. The caller owns the memory (aligned to a cache line), and the ring keeps no pointer in it, so two processes can map the same memory (`MAP_SHARED`) at different addresses; one side calls `libxs_ring_init` and the other uses the memory as is. `LIBXS_RING_PRIVATE` is for threads of one process, `LIBXS_RING_SHARED` for processes.

A record is written in place: `libxs_ring_reserve` returns room for it, contiguous and aligned to eight bytes, and `libxs_ring_commit` publishes it. The consumer reads it in place as well, until `libxs_ring_release`. A record takes at most half the capacity.

- `libxs_ring_size` returns the bytes of memory needed for `capacity` bytes of records (rounded up to a power of two); a record takes its size rounded up to eight bytes, plus eight.
- `libxs_ring_init` returns NULL if the memory cannot hold a ring.
- `libxs_ring_reserve` returns NULL if there was no room in time, the ring is closed, or the record exceeds half the capacity.
- `libxs_ring_peek` writes the record size to `*nbytes` and returns NULL if no record arrived in time, or if the ring is closed and empty.

A side that has to wait spins briefly and then sleeps (a futex on Linux) until the other side moves; `timeout_ms` of zero does not wait and a negative one waits indefinitely. `libxs_ring_close` ends the waiting on both sides, while the consumer still receives what was committed. A process that dies does not close its side, hence a timeout is how the other side notices.

File Locking

```
LIBXS_FLOCK(FILE)
LIBXS_FUNLOCK(FILE)
```

Per-file locking for thread-safe I/O. Maps to `flockfile`/`funlockfile` on POSIX, `_lock_file`/`_unlock_file` on Windows, or no-ops when synchronization is disabled.

Functions

```
unsigned int libxs_nranks(void);
```

Return the number of MPI ranks on the node of the calling process, as the launcher states it (`MPI_LOCALNRANKS`, `OMPI_COMM_WORLD_LOCAL_SIZE`, or `MV2_COMM_WORLD_LOCAL_SIZE`). Without these, it counts the user's processes on the node that initialized LIBXS (each holds a slot, a locked file in `/tmp`), or one. Such a count grows while processes start, hence it is better queried when needed than right at initialization.

```
unsigned int libxs_nrank(void);
```

Return the MPI rank of the calling process among the ranks on its node (`MPI_LOCALRANKID`, `OMPI_COMM_WORLD_LOCAL_RANK`, `MV2_COMM_WORLD_LOCAL_RANK`, or `SLURM_LOCALID`). Without these, it returns the index of the process' slot (see `libxs_nranks`), which is unique on the node but not necessarily below `libxs_nranks`. The global `PMI_RANK` is the last resort, which is local only if ranks are placed in blocks of `libxs_nranks` per node.

```
unsigned int libxs_rid(void);
```

Return the node-local MPI rank (`libxs_nrank`) if there are several ranks on the node (`libxs_nranks`), and the process ID otherwise.

```
unsigned int libxs_pid(void);
```

Return the process ID of the calling process.

```
unsigned int libxs_tid(void);
```

Return a zero-based, consecutive thread ID for the calling thread. TID = 0 does not necessarily correspond to the main thread.

```
void libxs_stdio_acquire(void);
void libxs_stdio_release(void);
```

Acquire/release a global lock around console output. The macros `LIBXS_STDIO_ACQUIRE()` and `LIBXS_STDIO_RELEASE()` expand to these calls.

```
int libxs_singleton_acquire(const char key[]);
void libxs_singleton_release(int handle);
```

Elect one process per node for a key: of the user's processes on a node, the first to acquire the key owns it until it releases the handle or exits, and every other acquisition fails, including a second one by the owner. A non-negative handle means the calling process owns the key. Unlike `libxs_nrank`, the election does not depend on what the launcher states, so it also holds across jobs and without MPI. On POSIX the lock is a file `/tmp/.libxs.<uid>.<key>`, hence `/tmp` must be local to the node; the key carries no path separator, and an empty key is the one LIBXS uses itself.

SYRK: Symmetric Rank-k Updates

Symmetric rank-2k and rank-k updates (`libxs_syrk.h`) built on top of GEMM dispatch, i.e., the config, backend selection, and registry are those of GEMM.

For small problems (n within LIBXS_GEMM_BM and LIBXS_GEMM_BN, and k <= LIBXS_GEMM_BK), the dispatched kernel handles the full GEMM in one call. Larger problems are split into column panels (see LIBXS_GEMM_PANEL and LIBXS_SYRK_BLAS). Scratch memory is thread-local.

Dispatch

```
libxs_gemm_config_t* libxs_syr2k_dispatch(
  libxs_data_t datatype, int n, int k, int lda, int ldb, int ldc,
  void* registry /* = NULL */);

libxs_gemm_config_t* libxs_syrk_dispatch(
  libxs_data_t datatype, int n, int k, int lda, int ldc,
  void* registry /* = NULL */);

int libxs_syr2k_dispatch_cpy(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, int n, int k, int lda, int ldb, int ldc,
  void* registry /* = NULL */);

int libxs_syrk_dispatch_cpy(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, int n, int k, int lda, int ldc,
  void* registry /* = NULL */);
```

Dispatch a GEMM config for SYR2K/SYRK. The problem shape (n, k, lda, ldb, ldc) is stored in the config and used by the call functions, while the dispatched kernel is the SYRK tile, i.e., the blocking (LIBXS_GEMM_BM/BN/BK) is applied inside. These are inlines and detect the backend at the caller's compile time, exactly like libxs_gemm_dispatch. The `_cpy` flavors populate a caller-owned config (see Config ownership).

To pass backend pointers explicitly, use the runtime forms:

```
libxs_gemm_config_t* libxs_syr2k_dispatch_rt(
  libxs_data_t datatype, int n, int k, int lda, int ldb, int ldc,
  const libxs_gemm_backend_t* backend /* = NULL */,
  void* registry /* = NULL */);

libxs_gemm_config_t* libxs_syrk_dispatch_rt(
  libxs_data_t datatype, int n, int k, int lda, int ldc,
  const libxs_gemm_backend_t* backend /* = NULL */,
  void* registry /* = NULL */);
```

Fortran mirrors both: LIBXS_JIT auto-detects, module LIBXS accepts OPTIONAL backend function pointers.

```
ptr = libxs_syrk_dispatch(LIBXS_DATATYPE_F64, n, k, lda, ldc,
     &  jit_create_dgemm=C_FUNLOC(mkl_cblas_jit_create_dgemm),
     &  jit_get_dgemm=C_FUNLOC(mkl_jit_get_dgemm_ptr),
     &  dgemm_blas=C_FUNLOC(DGEMM))

rc = libxs_syrk_dispatch_cpy(config, LIBXS_DATATYPE_F64,
     &  n, k, lda, ldc)
```

Call

```
void libxs_syr2k(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, const void* b, void* c);

void libxs_syr2k_task(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, const void* b, void* c,
  int tid, int ntasks);
```

C := alpha*(ABˆT + BAˆ T) + beta*C. Only the triangle specified by uplo ('U' or 'L') is written. All dimensions and leading dimensions come from the dispatched config.

```
void libxs_syrk(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, void* c);
```

```
void libxs_syrk_task(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, void* c,
  int tid, int ntasks);
```

C := alpha$A$Aˆ T + beta*C. Only the triangle specified by uplo ('U' or 'L') is written.

The _task variants partition work across ntasks threads. Each thread operates on independent output blocks; no locking is required. Thread-local scratch buffers are used internally.

```
int libxs_syrk_ntasks(const libxs_gemm_config_t* config, int nthreads);
```

Number of tasks worth splitting `libxs_syrk_task` or `libxs_syr2k_task` into, given nthreads (1 means unsplit). Fortran: `libxs_syrk_ntasks(config, nthreads)`.

In Fortran, `libxs_syrk` and `libxs_syr2k` also have a direct form that dispatches and calls in one step:

```
CALL libxs_syrk(config, uplo, alpha, beta, a, c)
CALL libxs_syrk(datatype, n, k, lda, ldc, uplo, alpha, beta, a, c)
CALL libxs_syr2k(config, uplo, alpha, beta, a, b, c)
CALL libxs_syr2k(datatype, n, k, lda, ldb, ldc, uplo, alpha, beta,
     &  a, b, c)
```

The direct form takes an OPTIONAL `registry` but no backend arguments. When the Fortran module is compiled with OpenMP, both forms run over an OpenMP team sized by `libxs_syrk_ntasks`, unless called from within a parallel region.

Compile-Time Tuning

The block sizes used for tiled SYRK/SYR2K can be overridden at compile time via preprocessor defines:

```
LIBXS_GEMM_BM        Row block size    (default: 192)
LIBXS_GEMM_BN        Column block size (default: 32)
LIBXS_GEMM_BK        K-direction block (default: 48)
```

Problems fitting within these limits use a single specialized kernel call (MKL JIT or LIBXSMM when available). The same values form the kernel shape that LIBXS_GEMM_JIT_MAX is applied to.

```
LIBXS_GEMM_PANEL     Smallest SYRK/SYR2K column panel
                     (default: 0, i.e., LIBXS_GEMM_BN)
LIBXS_SYRK_MINTASKS  Fewest threads that split a SYRK/SYR2K
                     against a sequential BLAS SYRK (default: 4)
```

Environment Variables

```
LIBXS_GEMM_BM=N          Row block size for tiled SYRK/SYR2K
                         (default: 192).
LIBXS_GEMM_BN=N          Column block size (default: 32).
LIBXS_GEMM_BK=N          K-direction block size (default: 48).
LIBXS_GEMM_PANEL=N       Smallest column panel of a split SYRK or
                         SYR2K; zero (default) takes
                         LIBXS_GEMM_BN. A larger value yields
                         fewer and wider panels (fewer tasks).
                         The tile sizes above then apply to the
                         diagonal block alone, and only where no
                         BLAS SYRK is available.
LIBXS_SYRK_PRINT=1       Print every double-precision BLAS SYR2K
                         call taken (see LIBXS_SYRK_BLAS) to stderr
                         (compile-time gate, enabled by default).
LIBXS_SYRK_BLAS=0|1      Resolve the BLAS SYRK/SYR2K entry points
                         (double and single precision) and prefer them for any shape wider than
                         one tile; such a shape then compiles no
                         kernel and takes no registry entry. Set to
                         0 to tile (default: 1).
```

The GEMM variables (LIBXS_GEMM_BACKEND, LIBXS_GEMM_JIT_MAX, etc.) apply as well, see GEMM.

Example

```
#include <libxs/libxs_syrk.h>

libxs_registry_t* reg = libxs_registry_create();

/* dispatch once per unique shape */
const libxs_gemm_config_t* cfg = libxs_syr2k_dispatch(
  LIBXS_DATATYPE_F64, n, k, lda, ldb, ldc, reg);

/* call many times (registry hit = hash probe only) */
for (batch = 0; batch < nbatches; ++batch) {
  libxs_syr2k(cfg, 'U', 0.5, 0.0, a[batch], b[batch], c[batch]);
}

libxs_gemm_release_registry(reg);
```

TAS: Tall-and-Skinny GEMM

One column-major GEMM, C := alpha*op(A)*op(B) + beta*C (`libxs_tas.h`), whose arithmetic intensity is low enough for memory bandwidth to bound it: one dimension (M, N, or K) is much larger than the other two. Such a GEMM runs along its longest dimension as a constant-stride GEMM batch of panels plus one remainder:

Longest	Panels	Shared operand
M	rows of op(A) and C	B
N	columns of op(B) and C	A
K	chunks of the inner dimension	none

Along K, each chunk writes a partial C of its own, and the partial results are summed pairwise along a binary tree over the chunks, by the tasks as their chunks complete. The order of summation depends on the number of chunks alone: the result is the same for any number of tasks. Any other GEMM is planned as one call of the whole GEMM. F64 and F32 are supported, with either operand transposed ('N' or 'T').

Panels are dispatched with `LIBXS_GEMM_FLAG_REUSE` (see GEMM): a panel is called often, which skips the JIT warm-up. A panel that is a window into a larger matrix (along M, or a chunk along K) is served by BLAS under the automatic backend, which packs the strided columns such a panel streams; a resident panel (along N) may take a generated kernel.

Plan

```
typedef struct libxs_tas_t {
  libxs_gemm_shape_t shape;           /* the whole GEMM */
  libxs_gemm_config_t panel, rest;    /* full panel, remainder */
  int stride_a, stride_b, stride_c;   /* between panels (elements) */
  size_t rest_a, rest_b, rest_c;      /* offsets of the remainder */
  int npanels, nscratch;              /* panels, partial C (along K) */
  char dim;                           /* 'M', 'N', 'K', or zero */
  void* scratch;                      /* partial C (along K) */
} libxs_tas_t;
```

A plan is caller-owned. It holds the scratch of the partial results (along K), hence it serves one call at a time, and it is released by `libxs_tas_release`. The configs inside are ordinary GEMM configs.

Dispatch

```
int libxs_tas_dispatch(libxs_tas_t* plan,
  libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, int lda, int ldb, int ldc,
  const void* alpha, const void* beta, int panel,
  void* registry /* = NULL */);

int libxs_tas_dispatch_rt(libxs_tas_t* plan,
  const libxs_gemm_shape_t* shape, int panel,
  const libxs_gemm_backend_t* backend, void* registry);
```

Plan a GEMM, and return nonzero if the plan was populated. The inline form detects the backend at the caller's compile time, exactly like libxs_gemm_dispatch; the runtime form takes the backend explicitly. The panel is the extent of a panel along M or N, or of a chunk along K; zero takes `LIBXS_TAS_PANEL` or `LIBXS_TAS_CHUNK`. A GEMM is planned along its longest dimension if its arithmetic intensity (FLOPs per Byte) is below `LIBXS_TAS_MAX` and if this yields at least two panels; otherwise `dim` is zero and the plan is one call.

Call

```
void libxs_tas(const libxs_tas_t* plan,
  const void* a, const void* b, void* c);

void libxs_tas_task(const libxs_tas_t* plan,
  const void* a, const void* b, void* c,
  int tid, int ntasks);

int libxs_tas_ntasks(const libxs_tas_t* plan, int nthreads);

void libxs_tas_release(libxs_tas_t* plan);
```

The operands are those of the whole GEMM. `libxs_tas_task` takes the share of one task: every tid in [0, ntasks) must call, since along K the tasks completing the partial results also sum them into C. `libxs_tas_ntasks` tells how many tasks are worth it (one for a plan of one call), see libxs_gemm_ntasks.

Fortran

```
USE :: LIBXS_JIT  ! auto-detected backends, or LIBXS with OPTIONAL ones
TYPE(libxs_tas_t) :: plan

IF (0 /= libxs_tas_dispatch(plan, LIBXS_DATATYPE_F64, 'N', 'N',
     &  m, n, k, lda, ldb, ldc, alpha, beta)) THEN
  DO i = 1, n_calls ! the plan serves every call of the same shape
    CALL libxs_tas(plan, C_LOC(a), C_LOC(b), C_LOC(c))
  END DO
  CALL libxs_tas_release(plan)
END IF

! one-shot form: plan, call, and release
CALL libxs_tas(LIBXS_DATATYPE_F64, 'N', 'N', m, n, k,
     &  lda, ldb, ldc, alpha, beta, C_LOC(a), C_LOC(b), C_LOC(c))
```

Both forms take an OPTIONAL `panel` and `registry`. In module LIBXS, `libxs_tas_dispatch` also accepts the OPTIONAL backend function pointers of `libxs_gemm_dispatch`. When the module is compiled with OpenMP, both forms run over an OpenMP team sized by `libxs_tas_ntasks`, unless called from within a parallel region; `libxs_tas_task` remains for explicit splitting. Without LIBXS, the fallback of LIBXS_JIT plans every GEMM as one BLAS call.

Compile-Time Tuning

```
LIBXS_TAS_PANEL      Extent of a panel along M or N (default: 256)
LIBXS_TAS_CHUNK      Extent of a chunk along K (default: 4096)
LIBXS_TAS_MAX        Arithmetic intensity (FLOPs per Byte) below
                     which a GEMM is planned along its longest
                     dimension (default: 7, as LIBXS_GEMM_JIT_MAX)
```

Environment Variables

```
LIBXS_TAS_PANEL=N    Extent of a panel along M or N; zero or unset
                     takes the compile-time default.
LIBXS_TAS_CHUNK=N    Extent of a chunk along K; zero or unset takes
                     the compile-time default.
LIBXS_TAS_MAX=X      Arithmetic intensity below which a GEMM is
                     planned along its longest dimension; zero
                     plans every GEMM as one call.
```

The GEMM variables (`LIBXS_GEMM_BACKEND`, `LIBXS_GEMM_JIT_MAX`, etc.) apply to the panels, see GEMM.

Example

```
#include <libxs/libxs_tas.h>

libxs_tas_t plan;
if (0 != libxs_tas_dispatch(&plan, LIBXS_DATATYPE_F64, 'N', 'N',
  m, n, k, lda, ldb, ldc, &alpha, &beta, 0/*panel*/, NULL))
{
  const int ntasks = libxs_tas_ntasks(&plan, omp_get_max_threads());
## pragma omp parallel num_threads(ntasks)
  libxs_tas_task(&plan, a, b, c, omp_get_thread_num(), omp_get_num_threads());
  libxs_tas_release(&plan);
}
```

Timer

Header: `libxs_timer.h`

High-resolution timing via a calibrated TSC when available, with an OS real-time clock fallback.

Types

```
typedef unsigned long long libxs_timer_tick_t;
```

Integer type representing a single timer sample.

```
typedef struct libxs_timer_info_t {
  int tsc;   /* non-zero if a calibrated TSC is used */
} libxs_timer_info_t;
```

Functions

```
int libxs_timer_info(libxs_timer_info_t* info);
```

Query timer properties. `info->tsc` is non-zero when a calibrated cycle counter is in use: RDTSC (x86), CNTVCT_EL0 (AArch64), or MFTB (PPC64), calibrated at initialization if the counter is invariant (Release builds only). Otherwise `info->tsc` is zero and the OS real-time clock is used. Triggers lazy library initialization if necessary. Returns `EXIT_SUCCESS`, or `EXIT_FAILURE` if `info` is NULL.

```
libxs_timer_tick_t libxs_timer_tick(void);
```

Sample the current tick of the monotonic timer. The tick unit is platform-specific; convert with `libxs_timer_duration` or `libxs_timer_ncycles`.

```
libxs_timer_tick_t libxs_timer_ncycles(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);   /* inline */
```

Absolute difference of two ticks (`LIBXS_DELTA`), i.e., `tick1 - tick0` for samples taken in that order. A wrap-around of the counter is not handled.

```
double libxs_timer_duration(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);
```

Convert a pair of ticks to elapsed wall-clock time in seconds.

Tokenizer

Header: `libxs_token.h`

The tokenizer provides reversible metatokens and vocabulary-backed lexemes. These representations serve separate APIs:

- reversible metatokens for prediction and exact text reconstruction;
- vocabulary-backed lexemes for interpretation, matching, and rules.

Metatoken Layout

`libxs_token_t` is one fixed 8-byte physical cell:

```
bit 7     sentence boundary on the final cell
bit 6..4  generic payload kind
bit 3     another cell continues this logical metatoken
bit 2..0  payload bytes in this cell, 1..7
bytes 1..7 payload
```

A logical metatoken may occupy several cells. Long text units therefore do not require a larger struct or an external vocabulary. `libxs_token_span` validates the continuation chain, while `libxs_token_read` reconstructs its payload and metadata.

Macro	Value	Meaning
`LIBXS_TOKEN_BYTES`	8	Size of one cell
`LIBXS_TOKEN_PAYLOAD_BYTES`	7	Payload bytes per cell
`LIBXS_TOKEN_LENGTH_MASK`	`0x07`	Payload length in the control byte
`LIBXS_TOKEN_CONTINUED`	`0x08`	Continuation bit
`LIBXS_TOKEN_KIND_MASK`, `LIBXS_TOKEN_KIND_SHIFT`	`0x70`, 4	Payload kind

Macro	Value	Meaning
LIBXS_TOKEN_SENTENCE	0x80	Sentence boundary

Payload kinds (enum libxs_token_kind_t, 0 to 7): LIBXS_TOKEN_LITERAL, LIBXS_TOKEN_TEXT, LIBXS_TOKEN_NUMBER, LIBXS_TOKEN_SPACE, LIBXS_TOKEN_PUNCT, LIBXS_TOKEN_MARKUP, LIBXS_TOKEN_REFERENCE, and LIBXS_TOKEN_CONTROL.

Encoder-produced cells are zero-initialized, including unused payload bytes. Binary comparison therefore remains deterministic when control metadata matters. For content matching, use libxs_token_payload_equal on physical cells or libxs_token_payload_match on complete logical metatokens. These functions compare declared payload length and bytes while ignoring kind, continuation, sentence flags, unused trailing bytes, and (for logical matching) cell layout. Do not infer payload length from trailing zeros; zero is a valid payload byte.

Payload kinds are independent of segmentation policy: word and syllable settings both emit LIBXS_TOKEN_TEXT. Their boundaries differ, but consumers use the same read and decode APIs.

Tokenizer Configuration

Granularity belongs to the tokenizer, not to each encode call:

```
libxs_tokenizer_t* tokenizer =
  libxs_tokenizer_create(LIBXS_TOKEN_GRANULARITY_WORD);
libxs_token_stream_t stream;

libxs_token_stream_init(&stream);
libxs_token_stream_encode(tokenizer, &stream, text, size);
libxs_tokenizer_set_granularity(tokenizer,
  LIBXS_TOKEN_GRANULARITY_SYLLABLE);
```

Available policies (enum libxs_token_granularity_t) are LIBXS_TOKEN_GRANULARITY_NATIVE (0), _WORD (1), and _SYLLABLE (2). Encoding is otherwise policy-neutral, and every policy preserves all input bytes.

```
libxs_tokenizer_t* libxs_tokenizer_create(int granularity);
void libxs_tokenizer_destroy(libxs_tokenizer_t* tokenizer);
int libxs_tokenizer_set_granularity(libxs_tokenizer_t* tokenizer,
  int granularity);
int libxs_tokenizer_granularity(const libxs_tokenizer_t* tokenizer);
```

destroy accepts NULL. set_granularity returns EXIT_SUCCESS or EXIT_FAILURE; granularity returns the policy, or -1 for NULL.

```
typedef struct libxs_token_stream_t {
  libxs_token_t* data;
  size_t size;
  size_t capacity;
} libxs_token_stream_t;

void libxs_token_stream_init(libxs_token_stream_t* stream);
int libxs_token_stream_reserve(libxs_token_stream_t* stream,
  size_t capacity);
int libxs_token_stream_push(libxs_token_stream_t* stream,
  const libxs_token_t* token);
void libxs_token_stream_release(libxs_token_stream_t* stream);
int libxs_token_stream_encode(const libxs_tokenizer_t* tokenizer,
  libxs_token_stream_t* stream, const unsigned char* text, size_t size);
int libxs_token_stream_decode(const libxs_token_stream_t* stream,
  unsigned char** text, size_t* size);
```

A growable array of cells. release frees the memory, not the struct. encode appends to the stream. All int functions return EXIT_SUCCESS or EXIT_FAILURE.

The stream is self-describing. Decoding does not require the tokenizer or a lexicon:

```
libxs_token_stream_decode(&stream, &decoded, &decoded_size);
```

The decoded bytes are allocated with `malloc` and zero-terminated; the caller releases them with `free`.

For every successful encoding:

```
decode(encode(text)) == text
```

including whitespace, spelling, case, punctuation, numbers, and UTF-8 bytes.

Logical Iteration

Physical cells are traversed by logical span:

```
size_t pos = 0;
while (pos < stream.size) {
  libxs_token_info_t info;
  if (EXIT_SUCCESS != libxs_token_read(stream.data, stream.size, pos,
    NULL, 0, &info)) break;
  pos += info.cells;
}
```

A NULL payload reads metadata only. Otherwise, the capacity must cover `info.length` bytes, or the read fails.

```
typedef struct libxs_token_info_t {
  size_t length;    /* logical payload bytes */
  size_t cells;     /* physical cells */
  int kind;
  int continued;
  int is_sentence;
} libxs_token_info_t;

int libxs_token_read(const libxs_token_t* tokens,
  size_t ntokens, size_t pos, unsigned char* payload, size_t capacity,
  libxs_token_info_t* info);
size_t libxs_token_span(const libxs_token_t* tokens,
  size_t ntokens, size_t pos, size_t* payload_size);
void libxs_token_info(const libxs_token_t* token,
  libxs_token_info_t* info);
int libxs_token_payload_equal(const libxs_token_t* lhs,
  const libxs_token_t* rhs);
int libxs_token_payload_match(const libxs_token_t* lhs,
  size_t nlhs, size_t lhs_pos, const libxs_token_t* rhs,
  size_t nrhs, size_t rhs_pos);
```

- `read` returns `EXIT_SUCCESS` only for a complete, valid chain.
- `span` returns the number of cells of the metatoken at `pos` (0 for an invalid position or chain); `payload_size` (may be NULL) receives its payload size.
- `info` decodes the control byte of a single cell.
- `payload_equal` and `payload_match` return non-zero if the payloads match.

Inline helpers on a single cell (NULL-safe):

```
size_t libxs_token_len(const libxs_token_t* token);           /* payload bytes */
int libxs_token_kind(const libxs_token_t* token);
int libxs_token_is_continued(const libxs_token_t* token);
int libxs_token_is_sentence_end(const libxs_token_t* token);
```

`info.kind` describes text, number, whitespace, punctuation, markup, literal, reference, or control payload. `info.is_sentence` is carried by the final cell of a logical metatoken.

Lexical API

`libxs_lexeme_t` is an 8-byte (`LIBXS_LEXEME_BYTES`) lexical occurrence:

```
typedef struct libxs_lexeme_t {
  unsigned int id;
  unsigned short length;
  unsigned short flags;
} libxs_lexeme_t;
```

`libxs_lexeme_stream_encode` normalizes words, maps them through a persistent lexicon, applies caller-owned normalization and classification rules, and emits stable IDs. The result is not reversible: use metatokens for exact text reconstruction or raw-text likelihood.

```
int libxs_lexeme_stream_encode(libxs_lexicon_t* lexicon,
  libxs_lexeme_stream_t* stream, const unsigned char* text, size_t size,
  const libxs_lexrule_t* rules, int nrules,
  const libxs_lexnorm_t* norms, int nnorms, int create);
```

Words are lowercased, then rewritten by `norms` (may be NULL), numbers become `<num>`, and punctuation is kept. Class flags come from built-in detection and then `rules` (may be NULL). A non-zero `create` interns unknown words into the lexicon; otherwise they receive id 0. The lexemes are appended to `stream`. Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

```
typedef struct libxs_lexnorm_t {
  char from[LIBXS_LEXEME_MAXBYTES + 1];
  char to[LIBXS_LEXEME_MAXBYTES + 1];
} libxs_lexnorm_t;
```

Maps normalized `from` text to `to`. `to` may hold several words separated by `LIBXS_LEXNORM_SEPARATOR` (space), e.g., "is not": only the first emitted lexeme carries the source length, the others carry length zero. Normalized text is at most `LIBXS_LEXEME_MAXBYTES` (63) bytes.

Lexeme flags (`libxs_lexeme_t::flags`):

Flag	Meaning
`LIBXS_LEXEME_WORD`	Word
`LIBXS_LEXEME_NUMBER`	Number
`LIBXS_LEXEME_PUNCT`	Punctuation
`LIBXS_LEXEME_MARKUP`	Markup
`LIBXS_LEXEME_SENTENCE`	Sentence-ending punctuation
`LIBXS_LEXEME_QUESTION`	Question mark or question word (default rules)
`LIBXS_LEXEME_STOP`	Stop word (default rules)
`LIBXS_LEXEME_ENTITY`	Uppercase-initial word (default rules)
`LIBXS_LEXEME_BREAK`	Preceded by whitespace (starts a source word)

```
typedef struct libxs_lexeme_info_t {
  unsigned int id;
  size_t length;
  unsigned int flags;
  int is_word, is_number, is_punct, has_break, is_sentence;
  int is_question, is_stop, is_entity, is_markup;
} libxs_lexeme_info_t;

void libxs_lexeme_info(const libxs_lexeme_t* lexeme,
  libxs_lexeme_info_t* info);
size_t libxs_lexeme_word_next(const libxs_lexeme_t* lexemes,
  size_t nlexemes, size_t pos);
int libxs_lexeme_is_word_char(const unsigned char* text, size_t size,
  int* length);

size_t libxs_lexeme_len(const libxs_lexeme_t* lexeme);           /* inline */
int libxs_lexeme_has_break(const libxs_lexeme_t* lexeme);        /* inline */
int libxs_lexeme_is_sentence_end(const libxs_lexeme_t* lexeme);  /* inline */
```

- `info` decodes all flags of a lexeme.
- `word_next` returns the number of lexemes of the source word at `pos` (up to the next lexeme with `LIBXS_LEXEME_BREAK`).
- `is_word_char` returns non-zero if `text[0]` starts a word character (UTF-8 letters included, encoded punctuation excluded); `length` (may be NULL) receives its byte count. Use it instead of `ctype` when scanning words.

Lexeme streams mirror metatoken streams (`libxs_lexeme_stream_t` with `data`, `size`, `capacity`):

```
void libxs_lexeme_stream_init(libxs_lexeme_stream_t* stream);
int libxs_lexeme_stream_reserve(libxs_lexeme_stream_t* stream,
  size_t capacity);
int libxs_lexeme_stream_push(libxs_lexeme_stream_t* stream,
  const libxs_lexeme_t* lexeme);
void libxs_lexeme_stream_release(libxs_lexeme_stream_t* stream);
```

Lexicon

```
libxs_lexicon_t* libxs_lexicon_create(void);
void libxs_lexicon_destroy(libxs_lexicon_t* lexicon);
unsigned int libxs_lexicon_size(const libxs_lexicon_t* lexicon);
unsigned int libxs_lexicon_id(libxs_lexicon_t* lexicon,
  const char* text, int length, unsigned int flags, int create);
const char* libxs_lexicon_text(const libxs_lexicon_t* lexicon,
  unsigned int id, int* length, unsigned int* flags);
unsigned int libxs_lexicon_count(const libxs_lexicon_t* lexicon,
  unsigned int id);
int libxs_lexicon_save(const libxs_lexicon_t* lexicon,
  void* buffer, size_t* size);
libxs_lexicon_t* libxs_lexicon_load(const void* buffer, size_t size);
```

Persistent vocabulary of normalized text; id 0 is reserved for unknown text.

- `destroy` accepts NULL. `size` excludes id 0.
- `id` maps text to its id; a non-zero `create` interns missing text, otherwise missing text yields 0.
- `text` returns the text of an id (not terminated, length via `length`) and its flags, or NULL.
- `count` returns how often the text was interned (0 for an unknown id).
- `save` follows `libxs_registry_save`: pass `buffer = NULL` to query the size, `*size` is the capacity on input and the bytes written on output.
- `load` returns NULL on failure, including a vocabulary written by a different library version.

Rules

Rules are fixed-size records that select a built-in template, supply an argument, and name an action. They can be stored in a registry and thereby serialized.

```
typedef struct libxs_textrule_t {
  unsigned char tmpl;
  unsigned char action;
  unsigned char reserved[2];
  unsigned int argument;
  unsigned char pad[8];
} libxs_textrule_t;

typedef struct libxs_textrule_ctx_t {
  const unsigned char* text;
  size_t text_size;
  int byte_pos;
  const libxs_lexeme_t* token;
  const libxs_lexeme_t* prev_token;
} libxs_textrule_ctx_t;
```

```
int libxs_textrule_eval(const libxs_textrule_ctx_t* ctx,
  const libxs_textrule_t* rules, int nrules);
unsigned int libxs_textrule_wordhash(
  const unsigned char* word, int len);
int libxs_textrule_load(const libxs_registry_t* registry,
  libxs_textrule_t* rules, int max_rules);
int libxs_textrule_save(libxs_registry_t* registry,
  const libxs_textrule_t* rules, int nrules);
int libxs_textrule_defaults(libxs_textrule_t* rules, int max_rules);
```

Text rules decide sentence boundaries. `eval` returns 1 (boundary confirmed) or 0 (suppressed); a token without the sentence flag returns 0, and the last matching rule wins. Templates (`tmpl`):

Template	Matches if
`LIBXS_TRULE_PREV_WORD_SHORT_UPPER`	Word before the position is short (<= 4) and uppercase-initial
`LIBXS_TRULE_NEXT_CHAR_UPPER`	Character after the following whitespace is uppercase
`LIBXS_TRULE_NEXT_CHAR_LOWER`	Character after the following whitespace is lowercase
`LIBXS_TRULE_PREV_TOKEN_BREAK`	Previous token has `LIBXS_LEXEME_BREAK`
`LIBXS_TRULE_INSIDE_DELIMITERS`	Position is inside paired delimiters (`argument` is the opening character)

Actions are `LIBXS_TRULE_SUPPRESS` and `LIBXS_TRULE_CONFIRM`. `wordhash` produces the `argument` for word matching.

```
typedef struct libxs_lexrule_t {
  unsigned char tmpl;
  unsigned char action;
  unsigned short flags;
  unsigned int argument;
  unsigned char pad[8];
} libxs_lexrule_t;

typedef struct libxs_lexrule_ctx_t {
  const unsigned char* text;
  int length;
  unsigned int flags;
  unsigned int hash;
} libxs_lexrule_ctx_t;

unsigned int libxs_lexrule_eval(const libxs_lexrule_ctx_t* ctx,
  const libxs_lexrule_t* rules, int nrules);
int libxs_lexrule_load(const libxs_registry_t* registry,
  libxs_lexrule_t* rules, int max_rules);
int libxs_lexrule_save(libxs_registry_t* registry,
  const libxs_lexrule_t* rules, int nrules);
int libxs_lexrule_defaults(libxs_lexrule_t* rules, int max_rules);
```

Lexical rules set (`LIBXS_LRULE_SET`) or clear (`LIBXS_LRULE_CLEAR`) the rule's `flags` on a matching lexeme; `eval` returns the resulting flags. Templates are `LIBXS_LRULE_WORD_HASH` (normalized word by `libxs_textrule_wordhash`), `LIBXS_LRULE_WORD_INITIAL_UPPER` (original spelling starts uppercase), and `LIBXS_LRULE_PUNCT_CHAR` (punctuation by character).

For both rule kinds, `load` returns the number of rules read from the registry (keys prefixed `TRULE:` or `LRULE:`), `save` returns `EXIT_SUCCESS` or `EXIT_FAILURE`, and `defaults` fills in the built-in rules and returns their count.

Text Reflow

```
int libxs_text_reflow(const unsigned char* text, size_t size,
  unsigned char** out, size_t* out_size);
int libxs_text_reflow_map(const unsigned char* text, size_t size,
  unsigned char** out, size_t* out_size, size_t** line_offsets, size_t* nlines);
```

Replace cosmetic newlines (column wrapping) with spaces while keeping structural ones (enumerations, headings, blank lines, verse). `*out` is allocated and must be released with `free`. `reflow_map` also allocates `*line_offsets` with `*nlines` entries, where entry k is the offset in `*out` at which input line k+1 begins; both may be NULL to skip the map. Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

Low-Level Utilities and SIMD

Header: `libxs_utils.h`

Target-attribute machinery, ISA feature gates, intrinsic fix-ups, bit-scan operations, and AVX-512 inline helpers.

Target Attributes

`LIBXS_INTRINSICS(TARGET)`

Expands to a `__attribute__((target(...)))` clause for the given ISA level. Used to compile individual functions for a higher ISA than the baseline without changing compiler flags. The macro is a no-op when the baseline already covers the requested level or when the compiler does not support multi-versioning.

`LIBXS_ATTRIBUTE_TARGET(TARGET)`

Lower-level macro that resolves `TARGET` (an integer ISA constant such as `LIBXS_X86_AVX512`) to the corresponding target string, e.g. `target("avx2,fma,avx512f,...")`.

ISA Feature Gates

Preprocessor symbols defined when the compiler can generate code for a given ISA:

Macro	ISA level
`LIBXS_INTRINSICS_X86`	x86 baseline (SSE2)
`LIBXS_INTRINSICS_SSE3`	SSE3
`LIBXS_INTRINSICS_SSE42`	SSE4.2
`LIBXS_INTRINSICS_AVX`	AVX
`LIBXS_INTRINSICS_AVX2`	AVX2 + FMA
`LIBXS_INTRINSICS_AVX512`	AVX-512 (F + CD + DQ + BW + VL + VNNI)
`LIBXS_INTRINSICS_AMX`	AMX tiles (via target attribute)

`LIBXS_INTRINSICS_INCLUDE` is defined when the x86 intrinsics headers (`immintrin.h`) are included.

`LIBXS_STATIC_TARGET_ARCH` records the highest ISA provided by the compiler flags. `LIBXS_MAX_STATIC_TARGET_ARCH` records the highest ISA reachable via target attributes. Target attributes (`LIBXS_ATTRIBUTE_TARGET_*`) are defined up to `LIBXS_X86_AVX10_512` depending on compiler version.

Intrinsic Fix-ups

Portability wrappers for intrinsics whose signatures differ across compilers:

Macro	Purpose
`LIBXS_INTRINSICS_LOADU_SI128`	`_mm_loadu_si128`
`LIBXS_INTRINSICS_MM512_LOAD_PS/PD`	Unaligned 512-bit float/double load
`LIBXS_INTRINSICS_MM512_STREAM_*`	Non-temporal 512-bit stores
`LIBXS_INTRINSICS_MM256_STORE_EPI32`	256-bit integer store
`LIBXS_INTRINSICS_MM512_SET_EPI16`	Portable 512-bit set from 32 int16 values
`LIBXS_INTRINSICS_MM512_UNDEFINED_*`, `LIBXS_INTRINSICS_MM256_UNDEFINED_SI256`, `LIBXS_INTRINSICS_MM_UNDEFINED_SI128/PD`	Undefined-value initializers (zero in debug)
`LIBXS_INTRINSICS_MM512_EXTRACTI64X4_EPI64`	256-bit extract from 512-bit
`LIBXS_INTRINSICS_MM512_ABS_PS`	Absolute value of packed floats
`LIBXS_INTRINSICS_MM512_MASK_I32GATHER_EPI32`	Masked gather
`LIBXS_INTRINSICS_MM512_STORE/LOAD_MASK*`	Mask register I/O (8/16/32/64-bit)
`LIBXS_INTRINSICS_MM512_CVTU32_MASK*`	Convert uint32 to mask register

Bit-Scan Operations

```
LIBXS_INTRINSICS_BITSCANFWD32(N)   /* count trailing zeros, 32-bit */
LIBXS_INTRINSICS_BITSCANFWD64(N)   /* count trailing zeros, 64-bit */
LIBXS_INTRINSICS_BITSCANBWD32(N)   /* bit index of highest set bit, 32-bit */
LIBXS_INTRINSICS_BITSCANBWD64(N)   /* bit index of highest set bit, 64-bit */
```

Use GCC `__builtin_ctz`/`__builtin_clz` when available, Windows `_BitScanForward`/`_BitScanReverse` intrinsics on MSVC, or portable software fallbacks (`*_SW` variants). All return 0 when `N` is 0.

```
int LIBXS_INTRINSICS_BITSCANFWD32_SW(unsigned int n);
int LIBXS_INTRINSICS_BITSCANFWD64_SW(unsigned long long n);
LIBXS_INTRINSICS_BITSCANBWD32_SW(N)
LIBXS_INTRINSICS_BITSCANBWD64_SW(N)
```

The software variants can be called directly. The backward scans are macros and therefore usable in constant expressions.

Derived Macros

```
LIBXS_NBITS(N)        /* minimum bits to represent N */
LIBXS_ISQRT2_HI(N)    /* approximation of sqrt(N), rounding up to POT */
LIBXS_ISQRT2_LO(N)    /* approximation of sqrt(N), rounding down to POT */

unsigned int LIBXS_ILOG2(unsigned long long n);   /* ceil(log2(n)) */
```

`LIBXS_ILOG2` is a function (not a macro) and handles the n = 0 case.

MXCSR (FP Environment)

Macros for preserving and restoring the x86 MXCSR register (floating-point control/status). On non-x86 platforms these are no-ops.

Macro	Description
`LIBXS_MXCSR_FTZ`	Flush-to-zero flag (0x8000 on x86, 0 otherwise)
`LIBXS_MXCSR_DAZ`	Denormals-are-zero flag (0x0040 on x86, 0 otherwise)
`LIBXS_MXCSR_GET()`	Read the MXCSR register (`_mm_getcsr()` on x86, 0 otherwise)
`LIBXS_MXCSR_SET(VALUE)`	Write the MXCSR register (`_mm_setcsr()` on x86, no-op otherwise)

AVX-512 Inline Functions

Available only when `LIBXS_INTRINSICS_AVX512` is defined.

```
__m512i libxs_mulhi_epu32(__m512i a, __m512i b);   /* inline */
```

Unsigned 32-bit high-multiply for 16 lanes: floor(a * b / 2^32). Emulates the missing `_mm512_mulhi_epu32` via even/odd `_mm512_mul_epu32`.

```
__m512i libxs_mod_u32x16(__m512i x,
  unsigned int p, unsigned int rcp);               /* inline */
```

Vectorized Barrett reduction: x mod p for 16 uint32 lanes. `rcp` is the Barrett reciprocal from `libxs_barrett_rcp()`. This is the SIMD counterpart of the scalar `libxs_mod_u32`.

Appendix

Presentations

- Self-Diagnosing Parameter Prediction
- Practical Ozaki Schemes

Scripts

`tool_normalize.sh`, `tool_checkstruct.py`, and `tool_checkenvars.sh` are policy files: they are maintained in LIBXS and copied into dependent projects by `make documentation`, hence a change belongs in LIBXS.

Build Support

tool_source.sh — Generate `libxs_source.h` (header-only amalgamation). Called by `make headers`.

tool_version.sh — Print the version: `version.txt`, plus the branch and the number of commits since the last release tag (Git). With `-1`, it emits `libxs_version.h`.

tool_pkgversion.sh — Update the RPM spec and the Debian changelog to the version in `version.txt`. Called by `make`.

libxs.pc.in — Template for the installed `.pc` file.

libxsConfig.cmake, **libxsConfigVersion.cmake.in** — CMake package files installed by `make`. **libxsConfig.cmake.in** — the same for the CMake build.

debian/, **fedora/libxs.spec** — Debian and RPM packaging.

Code Quality

tool_analyze.sh — Run cppcheck on `src/`.

tool_checkabi.sh — Compare exported symbols against a reference.

tool_clangformat.sh — Run `clang-format`, or a newer `clang-format-N` if one is found, with the given arguments.

tool_normalize.sh — Run the rules of `.pre-commit-config.yaml` on the tree, or on the given directory. `--install` installs the Git hook.

tool_checkstruct.py — Check the structural rules of the contribution policy (`--list` names the checks). Open findings are listed in `tool_checkstruct.todo`.

Documentation

md2pptx.py — Convert Markdown to PowerPoint. Splits on `---` rules or `##` headings (auto-detected). Requires `python-pptx` and `lxml`, optionally `Pillow` (`pip install -r requirements.txt`).

Development

tool_cpuinfo.sh — Print CPU topology (sockets, cores, threads).

tool_checkenvars.sh — List environment variables used in the source tree (`--list`), or check that the prefixed ones are documented. Names not documented yet are listed in `tool_checkenvars.todo`.

tool_pexec.sh — Parallel command execution with CPU affinity (`-h` for options).

tool_gitaddforks.sh — Add GitHub forks as Git remotes.

tool_gitprune.sh — Aggressive Git housekeeping (gc, fsck, repack).

Contributing

This is the contribution policy for LIBXS and LIBXSTREAM. Both projects follow the same policy, so the text is project-neutral: it is maintained in LIBXS (`documentation/libxs_contrib.md`) and copied into dependent projects by `make documentation`, the same way `Makefile.inc` is shared. Please edit it in LIBXS — the copy is overwritten whenever the original changes.

The code base is small, long-lived, and read far more often than it is written, so uniformity is what keeps diffs reviewable. When a rule below does not answer your case, the strongest rule is: **match the surrounding code**.

Not every rule is derived from first principles. Some are taste, some are habit, and some are plain superstition. They are the policy regardless: a uniform code base is worth more than an individually optimal choice, and rules that are merely arbitrary are still cheap to follow.

Character Encoding and Whitespace

Files	Encoding
Source, headers, kernels, Makefiles, scripts, YAML, plain text	US-ASCII only
Markdown (`*.md`)	any UTF-8

No Unicode in code or build files — no typographic quotes, no em dashes, no non-breaking spaces. Such characters survive editors, diffs, and generators poorly, and neither the compiler nor `sed` is required to handle them.

Markdown has no such restriction: a spaced em dash (—) instead of `--`, an en dash for numeric ranges (1–4), and mathematical notation (alpha, <=, ceil(n/2), x, sum) are all welcome wherever they read better than an ASCII transliteration.

Beyond encoding:

- **No tabs**, except where a Makefile requires them.
- LF line endings. CRLF is rejected.
- No trailing whitespace.
- No whitespace before `#` in a preprocessor directive.
- **No French spacing** in either sense: no space before `,`, `;`, `:`, `!`, or `?`, and a single space after a sentence-ending period. This holds for code, comments, commit messages, and documentation alike.
- A script carrying a shebang is executable in the index (mode 755); every other file is 644.

These are enforced mechanically rather than by review. `.pre-commit-config.yaml` combines the standard pre-commit hooks (trailing whitespace, line endings, byte-order marks, shebang and exec-bit consistency, YAML syntax) with the project-specific rules (US-ASCII, tabs, C++ comments, whitespace before `#`, `exit()` in library code, `sed -i` in scripts, `goto`, a declaration in a `for` initializer, banner comments, `--` in a comment, three blank lines, and column 73 in fixed-form Fortran). Install the Git hook once per clone; the same configuration runs in continuous integration, so a violation fails the pull request:

```
scripts/tool_normalize.sh --install   # once per clone
scripts/tool_normalize.sh             # check and fix the whole tree
scripts/tool_normalize.sh src         # or one directory
```

`.editorconfig` keeps most of it from happening in the first place. Data files (`*.csv`, `*.tsv`, `*.dat`) and generated sources are exempt — tabs and line endings are part of their format, and their producer owns them. Of the rules above only the two spacing conventions are checked partially: a space before a comma or semicolon is caught in C sources, the rest is on the author.

The rules that span more than one line are checked by `scripts/tool_checkstruct.py`: a single function exit, blank lines inside and between functions, stacked single-line comments, the shape of a multi-line comment, where an opening

brace goes, whether a multi-line `if` or loop is braced, whether the sections of a file come in order, and a constant on the left-hand side. It works on the source text, with comments, literals, and preprocessor directives masked out, so a `return` in `#if` and another in `#else` count as one exit. A parser is not used on purpose: the declarations carry `LIBXS_API` and friends, which a preprocessor-less parser turns into an error node every few lines, and a preprocessed compiler dump no longer says which file a construct came from. `scripts/tool_checkenvars.sh` compares the prefixed variables the source reads against what the documentation mentions.

The policy holds everywhere, and so do the hooks. Every rule that `tool_checkstruct.py` checks runs over each C source in the tree, samples and tests included: a contribution is held to the policy wherever it lands. What cannot reasonably be fixed is accepted in the script, per rule and per file, with the reason beside it (a macro that is a lowercase alias on purpose, or a file that configures a kernel by macros and then includes it), or deferred with a count in the to-do list below.

Both keep their open findings in a to-do file beside them, `scripts/tool_checkstruct.todo` and `scripts/tool_checkenvars.todo`, rather than in a file-level exclusion. Three properties follow, and each is the point:

- The list is **per rule and per file**, so a file listed for one rule is still checked by the others.
- The list is **per project and not propagated**. The two scripts are policy files that `make documentation` copies from LIBXS; a count lowered in a copy would be reverted, so the state cannot live inside them.
- The list is a **to-do, not a permission**. Each entry carries what it defers, and going the other way fails as well: one finding more than listed is a regression, one less means the entry outlived its findings, and an entry naming a file that no longer exists is reported too. An exclusion that outlives its cause is how a list starts lying.

The list maintains itself in the one direction that is safe. A fix followed by `tool_normalize.sh` lowers the count, and drops the entry when nothing is left; an entry whose file is gone drops too. The run still fails and says what it changed, exactly as the whitespace hooks do, so the smaller list is reviewed and committed rather than applied silently. Upwards never happens: a count that grew, or a new undocumented variable, is a regression and stays an error. `tool_checkstruct.py --counts` prints the table from scratch if a list needs rebuilding wholesale.

C Source File Structure

A translation unit is strictly grouped, in this order:

1. Includes
2. Macros
3. Types (typedefs, struct definitions)
4. Translation-unit variables (file-scope statics and globals)
5. Prototypes, where forward declarations are needed
6. Functions

No interleaving. A macro used only by the last function still belongs in the macro section, and a type or a table used only by the last function belongs in its own. This makes the shape of every file predictable: what it depends on, what it configures, what state it holds, and what it does — in that order. Grouping is also what keeps the sections reviewable: a macro parked next to its first use hides how many of them the file already has.

Three exceptions, and no others:

- A translation unit split into implementation fragments includes those fragments where their code belongs, not at the top. Such an include is a piece of the implementation rather than a dependency, and hoisting it would reorder the definitions it contains. This one is detected rather than judged, by the one property that separates the two: a fragment carries no include guard, because two of them in one translation unit would define everything twice. A dependency is quoted as well where it is a sibling, so the quotes decide nothing.
- A type may sit immediately above the one entry point it parameterizes, when that is what makes the interface readable. This covers an argument or callback type in a public header, not an internal type shared by several functions.

- In a header the last two sections are one. A header is read as a list of declarations, and an inline definition sits in that list: `LIBXS_API_INLINE` interleaved with `LIBXS_API` is the feature group the reader wants, not an interleaving to fix. The reason is the same one that lets a header open a function body on the right. It applies to those two sections only: a macro, a type or a file-scope variable below either of them is still out of order.

A macro that is undefined again is local to the code between the two, the way a variable is local to a block, so it is not a member of the macro section either. `LIBXS_FPRINT_LOAD` in `libxs_math_fprint.h` is defined above the one function that expands it and undefined below, and moving it into the macro section would separate it from its only use and leave the `#undef` reading as noise. The `#undef` has to follow the definition to mean this: the other order is the defensive one, dropping what a platform header may have set, and those macros (`__STRICT_ANSI__`, `_WIN32_WINNT`) are ordinary members of the section.

What sits inside a conditional is a feature test, and a declaration is no exception. A platform header may not declare what it should, so the declaration goes next to the `#define` that calls it and inside the same condition; a type the platform does not provide is defined next to the `#define` that records its availability. `LIBXS_EXTERN void flockfile(FILE*)` in `libxs_sync.h` and the `__int128` typedef in `libxs_macros.h` are both this, and hoisting either one means writing its condition a second time. So inside an `#if`, only a definition is ranked.

That has a prerequisite: a conditional that **wraps the whole file** must not count, or a translation unit that puts `#if defined(__OPENCL)` around everything it contains would have no rules left inside. The test for it is symmetric, since a header ends with a conditional include of its own implementation, which also has nothing after it: nothing that belongs to a section may precede the opener either, and the include guard is the same shape.

The `section-order` check reads the file as a sequence of top-level constructs and reports where their kind steps back to a section the file has already left. It is approximate by design: it ranks what it recognizes and stays quiet about the rest, so it misses cases rather than inventing them. A construct that is nothing but a macro invocation is left unranked, since what it expands to is not in the text, and so are the statements of an included body fragment, whose control flow sits at brace depth zero without being a definition. One backward step is reported once and not once per construct below it, because a single misplaced typedef does not make every macro under it a separate defect.

The shapes ranked out of the ordering are the ones where the text that looks like a section member is not one: the include guard, whose `#define` is the file's own name; anything but a definition inside an `#if`; a prototype immediately above the definition it repeats, which is how a static definition answers `-Wmissing-prototypes`; an include of a guardless implementation fragment; and a macro that is undefined again.

What is left over is judged, one file at a time, rather than guessed at by a rule reading intent — a type above the one entry point it parameterizes has no signal a checker could read. The judgement is recorded where it stays visible: in the to-do list while the file has structure the rule must keep watching, and in the script's permanent list only where the accepted shape is the one thing that rule could report in that file. `libxs_macros.h` is the second kind, since its standard includes must follow the feature-test macros that configure them, and macros and those includes are all it holds.

Every file carries the SPDX license header (BSD-3-Clause) verbatim as found in existing files.

C Dialect

C89 (ANSI C) in general. The pre-submit configuration (`PEDANTIC=2`) compiles with `-std=c89`, so a C99-only construct breaks the build for everyone else even when it compiles for you:

- Declarations at the beginning of a block, before any statement. No declarations mixed into the middle of a block, and no declaration in a `for` initializer.
- `/* ... */` comments only.
- No variable-length arrays, compound literals, designated initializers, `restrict`, or `//`-style line continuation tricks.
- Newer facilities are reached through the macros and typedefs the public headers already provide, not by raising the dialect locally.

Where a C99 (or later) construct is genuinely required, it is guarded and confined, in the same style as the existing guards.

OpenCL kernels (`*.cl`) follow every rule above except the dialect. They are compiled as OpenCL C, which is C99, so a declaration in a `for` initializer is correct there and the hooks exempt kernels from that one rule. Everything

else holds unchanged: US-ASCII, no tabs, `/* ... */` comments only, a single function exit, two blank lines between functions, capitalized macros, and the SPDX header. A kernel is source, not data.

A 64-bit integer is `uint64_t` or `int64_t`. They are exactly 64 bits wide rather than merely at least that wide, they add no name of ours to the API, and they cost the user nothing: `libxs_macros.h` already includes `<stdint.h>` and `<inttypes.h>`, so every consumer of the public headers has them, and the API already returns `uint64_t` from `libxs_hilbert` and `libxs_morton`. `long long` is kept only where something outside the project spells it — the Intel intrinsics take `long long*` (`_mm256_i64gather_epi64`), the `__atomic_*_8` builtins take `long long`, and a value printed with `%llu` is cast to `unsigned long long`, which is portable without composing the format string out of `PRIu64`. Those are the reasons `-Wno-long-long` is set for a pedantic build, and they are the only ones: new code does not reach for `long long` to hold a number of its own.

Functions

- **A function has a single exit.** No early `return`, no multiple return paths, no `goto`.
- **No trailing underscore on a parameter or a local**, in a definition or a declaration. That mark is reserved for a variable a *macro* declares, so that such a name cannot collide with one the caller already has in scope; a function wearing it anywhere takes the mark away from the one thing it is for. Passing an underscored name *to* a function is a different matter and fine: inside a macro body that is exactly what the macro's own local is for.
- Use a `result` variable, gate subsequent work on `EXIT_SUCCESS == result`, and return `result` at the single exit point.
- Constants go on the left-hand side of a comparison (`EXIT_SUCCESS == result`, `NULL != ptr`), which turns an accidental assignment into a compile error.
- Two blank lines between function definitions.
- At most one blank line inside a function body.

```
int example(const void* input, void** output)
{
  int result = EXIT_SUCCESS;
  if (NULL == input || NULL == output) result = EXIT_FAILURE;
  if (EXIT_SUCCESS == result) {
    result = prepare(input);
  }
  if (EXIT_SUCCESS == result) {
    result = finish(input, output);
  }
  return result;
}
```

Macros

A macro is capitalized, and so are its parameters: `LIBXS_ALIGN(POINTER, ALIGNMENT)`, not `LIBXS_ALIGN(pointer, alignment)`. The parameters are the part that is easy to forget, and they are the part that matters at the point of use: a capitalized argument is what tells the reader that the expression may be evaluated more than once. **A parameter carries no trailing underscore**: that marks a variable the macro declares itself, and the two must stay apart, since the whole point of the underscore is to say "this name is mine, not yours".

A variable a macro declares is the other way round: lowercase, with a trailing underscore, and ideally prefixed by the macro's own name.

```
#define MACRO() { int macro_i_ = 0; }
```

The capitalization tells the reader which names come from the call site and which the macro invented; the trailing underscore and the prefix are what keep the invented one from colliding with a variable the caller already has in scope. A macro that declares plain `i`, `s`, or `p` is a trap for whoever expands it next to their own `i`. The members of an aggregate the macro declares are not locals — in `union { float v; float a[8]; } u_` it is `u_` that carries the underscore — and a `*_DECL(A)` macro that declares a variable named by its own parameter keeps the caller's spelling.

Two kinds of macro name are lowercase on purpose, and both are exempt:

- A trailing lowercase segment is a token pasted onto the name — a type (`LIBXS_TYPECHAR_double`), a lock kind (`LIBXS_LOCK_ACQUIRE_spin`), an address space. It has to match the spelling of what it names.
- A macro standing in for something that is not ours keeps that spelling: a compiler builtin (`__builtin_nan`), a language keyword the OpenCL-on-CPU path defines away (`kernel`, `restrict`), a foreign API (`offloadSuccess`), or the lowercase alias a paste target needs (`libxs_crc32_b8`).

Comments

- **A comment adds value on top of the code.** Never document what the code obviously does — the code already says that. Document what it cannot say: why this way, what breaks otherwise, which assumption is being relied on.
- Prefer no comment at all. Then prefer one line.
- **Never stack comments.** Two comments with no code between them are either one comment — make it one line, or a block if it earns the size — or they describe different subjects, and then the code each one describes belongs between them. **A blank line between them does not separate them**: it is still two comments and no code. This covers blocks as much as single lines; the license header, being the file's leading comment, is exempt.
- **Single-line comments by default.** A multi-line comment has to earn its size: it is reserved for what is absolutely necessary, i.e., a risk or a trap worth spelling out — typically something that has already been gotten wrong once and would be gotten wrong again without the warning. Everything else is one line or nothing.
- API documentation in public headers is the other place a multi-line block belongs.
- **A multi-line block carries a `*` on every line**, under the one in the opening `/*`, the closing line included. Two shapes satisfy that and both are accepted outside a header: the block that opens on its own line, and the compact one whose text starts on the opening line and ends on the closing one. What it rules out is a third shape, a hanging indent with no stars, where nothing marks the lines as comment and a reader scanning the left edge loses them.
- **In a header, a block at file scope opens and closes on its own line.** The reason is the API rather than the comment, and it is the one that lets a header open a function body on the right: the declarations are read as a list, and a block starting and ending mid-line breaks the rhythm of that list. Inside a function body, and in an implementation unit, either shape is fine. Measured before requiring it: 452 blocks at header file scope already open on their own line and 15 do not.
- `/* ... */` only. **C++ comments (`//`) are rejected** in `.c` and `.h`.
- **No decorative or banner-style comment blocks.** No `/*==== Section ====*/`, no boxes, no ASCII rules. The license header is the sole exception.
- Never write `--` inside a comment (the ASCII rule keeps the em dash out, and a double hyphen reads as a typo); rephrase instead.
- These rules govern the comments a change **writes**. Do not restyle existing comments along the way: one that already earned its size keeps it, and reflowing a file's comments buries the actual change exactly as a bulk reformat does.

```
/* one line is the default, and most comments need no more */

/**
 * A block earns its size by naming a trap: what was already gotten wrong once,
 * and would be gotten wrong again without the warning.
 */
```

Blank Lines

- Never three or more consecutive blank lines, anywhere.
- Exactly two blank lines separate function definitions in a `.c` or `.cl` file. **A header may use one**, and the files that carry small `LIBXS_API_INLINE` definitions are uniform about it: `libxs_gemm.h` and `libxs_token.h` use one throughout, `libxs_math.h` for sixteen of its twenty-one. One or two, but not a mixture within a file, which is the surrounding code a change there has to match.

- At most one blank line separates logical blocks inside a function. Two is what separates the functions themselves, so two never appear inside one.

Formatting

A `.clang-format` file is present (LLVM-based, 2-space indent, 96-column limit, never tabs), and `scripts/tool_clangformat.sh` selects the newest available version of the tool.

Do not bulk-reformat as part of a change. Recent clang-format versions reflow entire files, which buries a small change in hundreds of unrelated lines and makes review impossible. This is why clang-format is deliberately absent from the hook set. Format new and edited code by hand to match the file around it; the formatter is a maintenance tool, run deliberately and committed on its own.

Do not mix reformatting, renaming, and behavioural change in one commit.

Where the opening brace goes. It stays on the line of the construct it belongs to — `if (0 < n) {`, `for (...) {`, `} else {` — with two exceptions, and both are checked:

- **A function body opens on its own line in an implementation unit**, a `.c` or a `.cl` file: 1971 definitions do it and 78 do not.
- **A brace whose parentheses were broken across lines opens on its own line**, because the brace is then what tells the reader the condition has ended:

```
if (0 == first &&
    0 != second)
{
  ...
}
```

In a header the brace may trail the signature, and it usually should. What rules a header is a readable API: the declarations are read as a list, and an inline definition sits in that list. It is also outside the ABI and meant to stay a small tool, so a body long enough to want the brace on a line of its own needs a reason — `libxs_gemm.h` has one. Both forms are accepted there, which is the difference from an implementation unit; a broken parameter list still takes the brace onto its own line.

A bare block has no construct line to sit on, so nothing is required of it: `{ int scope_ = n;` and a brace alone on the line are both in use and both fine. A struct, a union and an initializer are left alone as well.

Whatever an `if`, an `else` or a loop controls stays on the keyword's line, or it is braced. Once the construct reaches a second line the braces are what say where it ends, and that includes the case where only the condition wrapped:

```
while (npos < (int)ctx->text_size
  && 0 != isspace(ctx->text[npos]))
{
  ++npos;
}
```

Each keyword is judged on its own, so an `if` with a braced body and a one-line `else` is two decisions rather than one. An `else if` is the inner `if` and is judged there.

Two more places keep the brace on its own line, and the check knows both. One is a construct whose line is followed by a preprocessor directive: the line above the brace is then `#endif`, and moving the brace up would carry it into the branch. The other is a header that is included inside a function body, where the file's own control flow sits at brace depth zero and is not a definition.

```
#if defined(LIBXS_CPUID_ARM_MODEL_FALLBACK)
  if (NULL != info)
#endif
  {
    ...
  }
```

A directive in that position also excuses the braces themselves: the keyword and what it controls then belong to different configurations, and one pair of braces cannot serve both.

Indentation is not reformatted by a hook, but one thing about it is checked: **two closing braces in a row must step left.** Sharing a column means they close blocks that are nested, so a level is missing from the indentation even though the braces balance and the compiler is content. The check is local and makes no assumption about the indent unit, and a preprocessor directive between the two resets it, because which brace belongs to which block then depends on the configuration. `} else {` is not a closer for this purpose: it reopens, so the next closer legitimately shares its column.

Library Code

- Library code does not terminate the process: no direct `exit(...)` in `src/`. Return a status and let the caller decide. The macro that wraps the one unavoidable case is the single exception.
- Environment variables carry the project's own prefix (`LIBXS_*` or `LIBXSTREAM_*`). `scripts/tool_checkenvars.sh --list` lists what the source reads, ours and foreign.
- **Header-only mode must keep working.** The amalgamated header (`*_source.h`, or the corresponding `-D*_SOURCE`) has to be includable from multiple translation units, so a new file-scope symbol in `src/*.c` needs internal linkage or the established macro treatment.
- Fix the cause, not the symptom. When a sample, test, or dependent project runs into a limitation of the library, change the library rather than working around it at the call site.

Fortran

Fortran sources are **fixed-form** (`.f`). Free-form (`.f90`) is not used; please do not propose converting them.

- Statements occupy columns 7 to 72. Columns 73 and beyond are ignored, so text reaching column 73 is **silently truncated** rather than diagnosed.
- A continued line carries `&` in column 6. Sources additionally place a trailing `&` in column 73, so the same file also reads as free-form; that marker is the only thing allowed at column 73.
- `.fi` files are included into fixed-form sources and follow the same rules.

Scripts

- POSIX-portable shell. `sed -i` is rejected: it is not portable (macOS).
- Shell scripts pass `shellcheck`, which the hooks run.
- Python is formatted with `black -l79` and passes `flake8`; `mypy` covers the tooling under `scripts/` and `.theme/`, not sample code.
- Where a hook carries an exclusion, it names the open findings it defers. An exclusion is a backlog item, not a permission.

Documentation

Documentation is terse and written for the person *using* the code, not for the person who wrote it.

- **Every `README.md` is user documentation**: what the thing does, how to enable it, how to run it, which environment variables and build knobs exist. Nothing more.
- Insights stay out. Design rationale, derivations, and performance analysis do not belong in a README — the single exception being a surprising usability implication the user has to know about (a knob that silently changes accuracy, a mode that only works on one vendor). If it does not change how someone uses the code, it is not user documentation.
- A short *why* belongs in the commit message.
- Document every new environment variable. A variable that `tool_checkenvars.sh` reports but the documentation does not mention is a defect, and the hook of the same name says so.
- A new page under `documentation/` needs a `nav` entry in `mkdocs.yml`.

Markdown may use any UTF-8, but the PDF is produced through LaTeX, which cannot render an arbitrary glyph. `PDF_UTF8_SED` in `Makefile.inc` transliterates a known set (Greek letters, arrows, comparison and set operators, ceiling and floor brackets). If `make documentation` fails on a character, add it there rather than removing it from the text.

Parts of `documentation/` are **generated**, and editing the output is lost work: the landing page comes from `README.md`, the development page from `scripts/README.md`, one page per sample from `samples/*/README.md`, one page per test from `tests/*.c`, this page from LIBXS, and the PDFs from all of the above. Version and amalgamated headers are generated too. Edit the source, then:

```
make documentation   # PDFs
make mkdocs          # serve the site with live reload
make mkslides        # serve a slide deck (SLIDES=<topic>)
```

The Makefile is authoritative about what is generated from what.

Build Systems

GNU Make is primary. The `Makefile` and the shared `Makefile.inc` define the defaults, the knobs, and the behaviour; they are the reference. **CMake is secondary**: it has to produce the same artifacts, but it does not get to define anything.

- A new source file, build knob, or changed default lands in the Makefile first, then in `CMakeLists.txt`. `CMakeLists.txt` mirrors the Makefile's defaults and says so where it matters — when a default moves, move both.
- Every step is taken to keep the two interchangeable *from the consumer's side*. In particular, `make` writes the files a CMake consumer expects — the package configuration under `lib/cmake/<project>/` alongside the pkg-config `.pc` files — so a tree built and installed with GNU Make is usable via `find_package()` without CMake ever having run. GNU Make pretending to be CMake is a feature, not a workaround.
- Continuous integration builds and installs both ways and then consumes the result via `find_package()` and pkg-config. A change that only works in one of the two is incomplete.

Building and Testing

Build without `DBG=1` by default. Debug builds are `-O0`, which makes any runtime-bearing sample prohibitively slow — never measure performance with one.

```
make -j $(nproc)                    # default build
make -j $(nproc) test               # test suite
make -j $(nproc) DBG=1 PEDANTIC=2   # correctness check before submitting
```

`PEDANTIC=2` enables strict warnings and `ANALYZE=1` runs the compiler's static analyzer; where a project ships `scripts/tool_analyze.sh`, that runs cppcheck on top. A change is expected to be warning-free under `DBG=1 PEDANTIC=2`, because that is what continuous integration builds: GCC, Intel oneAPI, and macOS, covering release and strict debug configurations as well as a header-only build compiled as C++. See `.github/workflows/` for the exact matrix.

`SANITIZE=<kind>` passes `-fsanitize=<kind>` to both the compiler and the linker. ThreadSanitizer needs no options file: `__tsan_default_options` and `__tsan_default_suppressions` in `libxs_main.c` carry what a run of the library needs, and the environment is parsed after them, so `TSAN_OPTIONS` still overrides either one. They are inert in a shared-library build, where the runtime's own weak definition is found first, and that build needs the environment instead.

```
make SANITIZE=thread
setarch -R ./sample.x    # ASLR off, or the runtime aborts before main()
```

Some kernels abort with `unexpected memory mapping` unless ASLR is off, hence the `setarch`. Exit code 66 means races were reported, not that the program failed.

GNU libgomp is not instrumented, so a race whose two accesses sit on opposite sides of an `omp parallel` region is reported even though the region's own barrier separates them; a fifteen-line correct program reproduces it. The compiled-in `ignore_noninstrumented_modules=1` cuts the volume about fivefold but not the class. **Triage by region position, not by thread**: a report is that artifact unless *both* stacks carry an `_omp_fn` frame. Thread identity is not the test, since the main thread is a member of the team and takes part in the rendezvous. Suppressing the class is not possible, because those stacks run through the code under test; building with Clang against a libomp that has TSan support (it ships `libarcher.so`) is what makes the region boundaries visible.

ABI and Versioning

The version derives from Git tags and `version.txt`; the shared library carries an SOVERSION. `scripts/tool_checkabi.sh` compares exported symbols against the recorded baseline. Do not remove or rename a published symbol — add a new one instead. Run the checker on a build that has symbol information:

```
make STATIC=0 SYM=1
scripts/tool_checkabi.sh
```

A symbol name outside the project's namespace is an error, not a warning.

Commits and Pull Requests

- One concern per commit. Keep unrelated formatting out of it.
- Subject line: short, capitalized, no trailing period, e.g. `Improved device memory allocation`. An area prefix is fine: `CMake: updated to test kernels`.
- Explain *why* in the body when the subject cannot carry it.
- Contributions are accepted under the project's BSD-3-Clause license.

Before submitting:

```
scripts/tool_normalize.sh --install       # once per clone, then automatic
scripts/tool_normalize.sh                 # whitespace, encoding, lint
make -j $(nproc) DBG=1 PEDANTIC=2 test    # strict build and tests
scripts/tool_checkabi.sh                  # only if public symbols changed
```

Workflow with Assistants

If an AI assistant is used, the same policies apply, plus two more:

- Discuss design options and trade-offs before implementing.
- Present options as inline text, not as interactive choice widgets.